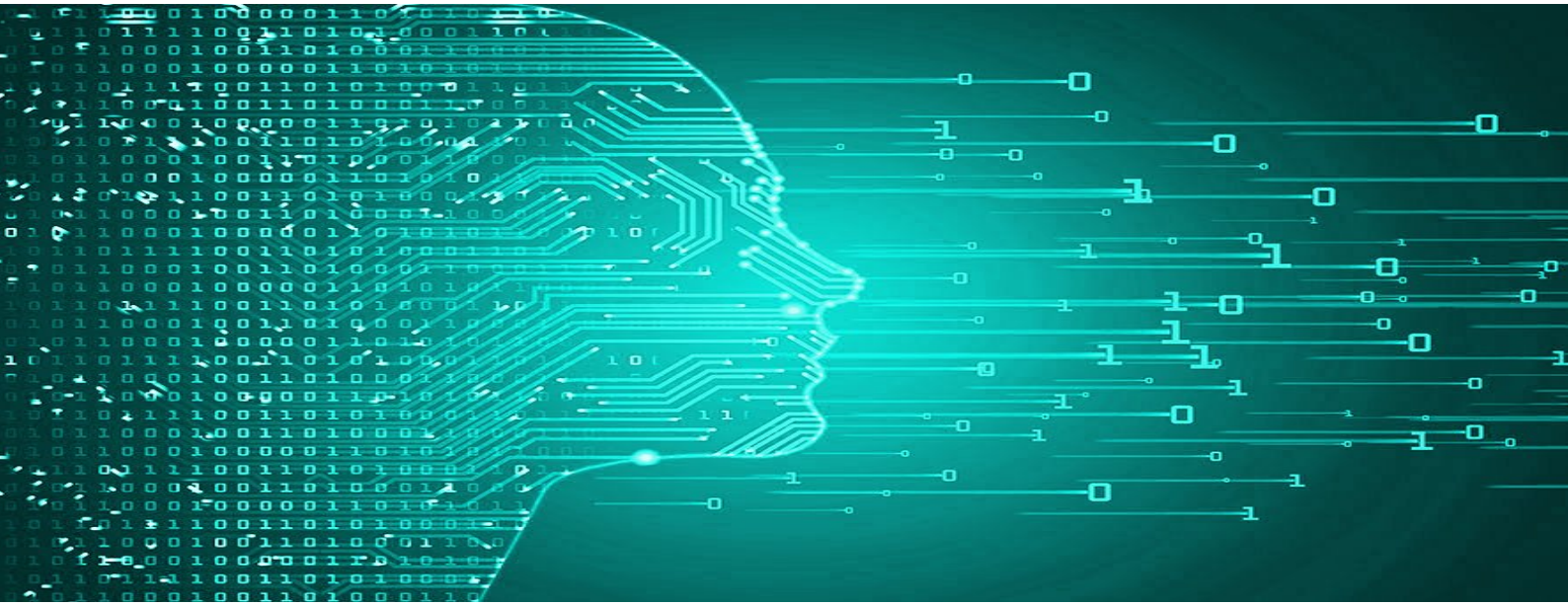


Elmar Wings

AI with an Arduino Nano 33 BLE Sense

Realtime Object Detection with the ArduCAM

Version: 1765
May 3, 2025

Contents

List of Figures

List of Listings

Acronyms

AOP Acoustic Overload Point

GND Ground

GPIO General Purpose Input Output

I²C Inter-Integrated Circuit

IDE Integrated Development Environment

IMU Inertial Measurement Unit

IoT Internet of Things

JST Japan Solderless Terminal

LED Light Emitting Diode

mcd Millicandela

PCB Printed Circuit Board

PDM Pulse Density Modulation

PWM Pulse with Modulation

SCL Serial Clock

SDA Serial Data

SPI Serial Peripheral Interface

UART Universal Asynchronous Receiver Transmitter

VCC Voltage at the Common Collector

1. Introduction

1.1. Introduction

Bézier curves are parameter curves that can be used to represent free-form curves. They are named after the French engineer Pierre Bézier, who developed them in the 1960s at Renault to design car body shapes. During the same period, French physicist and mathematician Paul de Casteljau developed these curves independently of Pierre Bézier at Citroën. Paul de Casteljau's results were available earlier, but they were not published. This is the reason why Pierre Bézier is the namesake of these parameter curves.[Far02]

The Bézier curves are the basis for computer-aided design of models. This is referred to either as Computer Aided Design (CAD) or, when the emphasis is more on a mathematical view, Computer Aided Geometric Design (CAGD). [BN11] Computer requires a mathematical description of shapes to represent them. The most suitable description method for this is the use of parametric curves and surfaces. Here Bézier curves play the central role, because they are the most numerically stable polynomial bases used in CAD/CAGD software.[Far02]

Typography on the computer also uses Bézier curves. There are two ways of representing type with the computer. The simplest way is to save each individual letter with a fixed resolution and size as a bitmap and copy it to the memory area of the screen as needed. These so-called bitmap fonts can be displayed quickly but require a lot of memory if the characters are to be available in different sizes. In this case, an extra bitmap must be created for each size. The quality also decreases when these bitmap fonts are scaled in size and resolution. Vector fonts are an alternative. Their name is derived from the fact that they use curves defined in a two-dimensional vector space to represent the characters. The advantage here is that the characters displayed in this way can be scaled without loss of quality. Two standards have developed for vector fonts. One is the TrueType font and the other is the PostScript font. The TrueType fonts use square Bézier curves while the PostScript fonts use cubic Bézier curves. The cubic Bézier curves have more control points which leads to a better quality.

Another field of application is the control of machine tools. When moving to a corner, the axis must be braked to a standstill at the corner point and then accelerated again after direction correction. This procedure ensures that it is not possible to move at a constant speed, which has negative consequences for the cycle time and quality. A possible solution to this problem is to place a curve between the two sections instead of a sharp corner, which can be run at a constant speed. Bézier curves are suitable for this purpose because only two points and two tangents are needed to form them. In the case of a corner, the points as well as the tangents would lie on the sections that form the corner. The resulting error would be analytically controllable and would allow an adjustment of the curve tolerance.[SS14]

1.2. Challenges and Solutions

1.3. Structure

Part I.

Arduino Nano 33 BLE Sense - Onboard Sensoren

2. Arduino Nano 33 BLE Sense

Arduino is an open-source electronics platform based on flexible, easy-to-use hardware and software. It provide us on board microcontroller and microprocessor kits for making digital and analogue devices. The microcontroller, oftenly called as tiny computer embed on the arduino board, normally in order to run these microcontroller we need some type of electronics e.g; diodes, resisters, capacitors, and transistors for making the voltage and current balancing. But the arduino team make a user friendly environment for getting rid of these electronics complication to run the hardware on software, just power the board as per the required voltage write the desired programm and upload it in a few seconds, it will bring everything on the board and make us independent from any worry about the electronics complication. By the technological advancement in semiconductor and electronics industry, the control problems are now being solved by using these small size microcontroller instead of mechanical and electrical swithes. All Arduino boards have one thing in common which is a microcontroller, it is basically a really small computer, which help us to make a edge computing application.[Ard21]

WS:Advertisement! Rewrite more as an engineer!

2.1. Arduino Nano 33 BLE Sense

The Arduino Nano Family is a set of boards with a tiny footprint, packed with features. It ranges from the inexpensive, entry model Nano , to the more feature-packed Nano BLE Sense / Nano RP2040 Connect which comprises of Bluetooth / Wi-Fi radio modules. These boards also have a set of embedded sensors, such as temperature, humidity, pressure, gesture, microphone and more. They can also be programmed with MicroPython and supports Machine Learning. [Raj19].

Arduino Nano 33 BLE Sense is one type of arduino family board, which is come up with Bluetooth Low Energy (BLE) capability for communication and set of sensors. This compact and reliable Nano board is built around the NINA B306 module for BLE and Bluetooth 5.0 communication; Bluetooth 5.0 is the latest version of the Bluetooth wireless communication standard. Use of microcontrollers has become inevitable in almost every field of engineering. The Arduino Nano 33 BLE Sense module is based on Nordic nRF52480 processor that contains a powerful Cortex M4F and the board has a rich set of sensors that allow the creation of innovative and highly interactive designs. Its reduced power consumption, compared to other same size boards, together with the Nano form factor opens up a wide range of applications.

The model name Arduino Nano 33 BLE Sense, itself puts out some important information. It is called “Nano” because of its compact nano size. “33” is included in the model name to indicate that the board operates on 3.3V. Then the name “BLE” indicates that the module supports Bluetooth Low Energy and the name “Sense” indicates that it has on-board sensors like accelerometer, gyroscope, magnetometer, temperature and humidity sensor, Pressure sensor, Proximity sensor, Colour sensor, Gesture sensor, and even a built-in microphone. [Raj19].

Also, for making the RGB color and person detection, we need to make the interface of BLE 33 Sense with camera shield Arducam OV2640. The Arducam OV2640 camera use to detect the RGB, object detetion and also the gesture too. Arduino Nano 33 BLE Sense and camera shield Arducamis a perfect match for making ML and AI application, by having the set of sensors on board we just need to install the respective

WS:Advertisement! Rewrite more as an engineer!

library on Arduino boar and it support the funcnality of sensors and Machine learning application.
The following figure 2.1 shows a Arduino Nano 33 BLE Sense Revision 2, shows how compact it is and small in size.

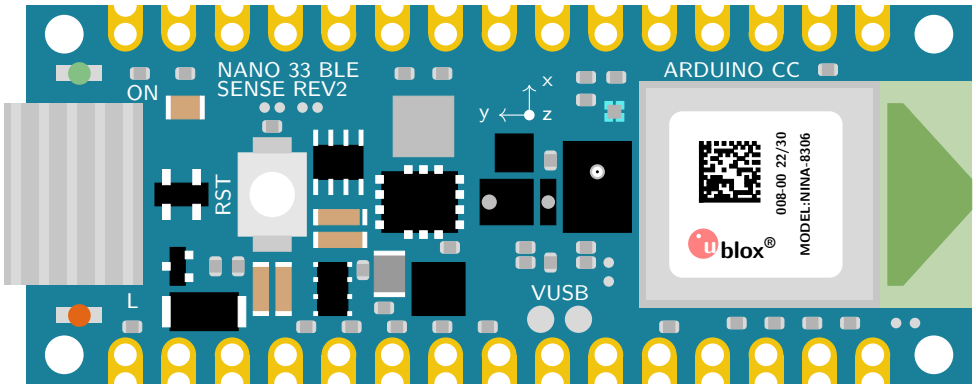


Figure 2.1.: Arduino Nano 33 BLE Sense Rev. 2, see Arduino Store

The BLE (Bluetooth Low Energy) compact and reliable Nano board are built on NINA B306 module for BLE and Bluetooth 5 communication; the NINA B306 module based on Nordic nRF52480 processor that contains a powerful Cortex M4F CPU, Its architecture fully compatible with Arduino IDE Online and Offline. The Arduino Nano BLE 33 Sense have a following set of sensors on board, ADPS-9960, LPS22HB, HTS221, LSM9DS1,and MP34DT05-A. It is small in size, having all the required sensor on board. [Ard21]
The Arduino Nano 33 BLE Sense have the following set of Sensors, BLE module and its functionality below.

- The Bluetooth is managed by a NINA B306 module.
- The ADPS-9960 is a digital proximity, ambient light, RGB and gesture sensor.
- The LSM9DS1 is a system-in-package featuring a 3D digital linear acceleration sensor, a 3D digital angular rate sensor, and a 3D digital magnetic sensor.
- The LPS22HB reads barometric pressure and environmental temperature.
- The HTS221 senses relative humidity.
- The MP34DT05 is support the sound detection.

2.2. Unterschied zwischen dem Arduino Nano 33 BLE Sense Rev1, dem Arduino Nano 33 BLE Sense Rev2 und dem Arduino Nano 33 BLE Sense Lite

2.2.1. What is the difference between Rev1 and Rev2?

The differences between the Arduino Nano 33 BLE Sense and the Arduino Nano 33 BLE Sense Rev2 concern the IMU, the temperature and humidity sensor, the microphone, the crypto chip and the voltage converter. In the second revision, the LSM9DS1 IMU has been replaced by two modules: the BMI270 acceleration and angular rate sensor and the BMI150 magentometer. The HTS221 humidity sensor has been replaced by the HS3003, with the latter promising greater accuracy. The values

of the microphones have remained the same despite the change from the MP34DT05 to the MP34DT06JTR. Similarly, only the component designations of the voltage converters differ. The first generation uses the MPM3610, the Rev2 uses the MP2322. However, the crypto chip is no longer present in the Rev2.

The changes are summarized below:

- Replacement of the IMU of LSM9DS1 (9 axes) with a combination of two IMUs (BMI270 - 6 axes IMU and BMM150 - 3 axes IMU).
- Replacement of the temperature and humidity sensor from HTS221 to HS3003.
- Replacing the microphone from MP34DT05 to MP34DT06JTR.

In addition, some components and changes have been made to increase user-friendliness:

- Replacing the power supply from MPM3610 to MP2322.
- Add a VUSB solder pad to the top of the board.
- New test points for USB, SWDIO and SWCLK.

The figure 2.2 presents the layout of the first version of the Arduino Nano 33 BLE,

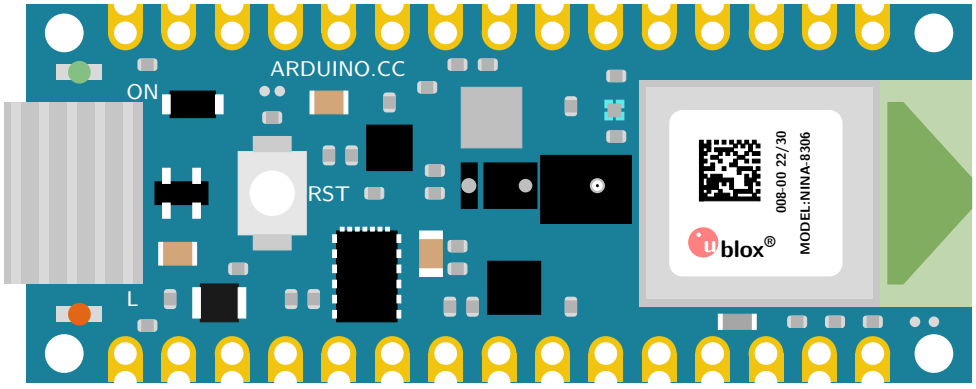


Figure 2.2.: Arduino Nano 33 BLE Sense Rev. 1

2.2.2. Changes in the Sketch

For sketches that were created with libraries such as LSM9DS1 for the IMU or HTS221 for the temperature and humidity sensor, these libraries must be changed to the following for the new revision:

- Arduino_BMI270_BMM150 for the new combined IMU, and
- Arduino_HS300x for the new temperature and humidity sensor.

Rev 1 TensorFlow Lite lib

2.2.3. Arduino Nano 33 BLE Sense Lite

WS:citation The Tiny Machine Learning Kit contains only the Arduino Nano 33 BLE Sense Lite.

The Arduino Nano 33 BLE Sense Lite differs only slightly from the Arduino Nano 33 BLE Sense Revision 1. The only difference between the two models is that the Lite version does not have the HTS221, the sensor for relative humidity and temperature. The LPS22HB sensor, which is on the board, is a pressure sensor, but it can also be used to measure the temperature. It is therefore not possible to measure humidity with the Arduino Nano 33 BLE Lite. To measure relative humidity, a separate sensor must be connected. The reason for this decision is that the Arduino company has difficulties maintaining the stock. [FD22]
The figure 2.3 presents the layout of the Arduino Nano 33 BLE Lite,

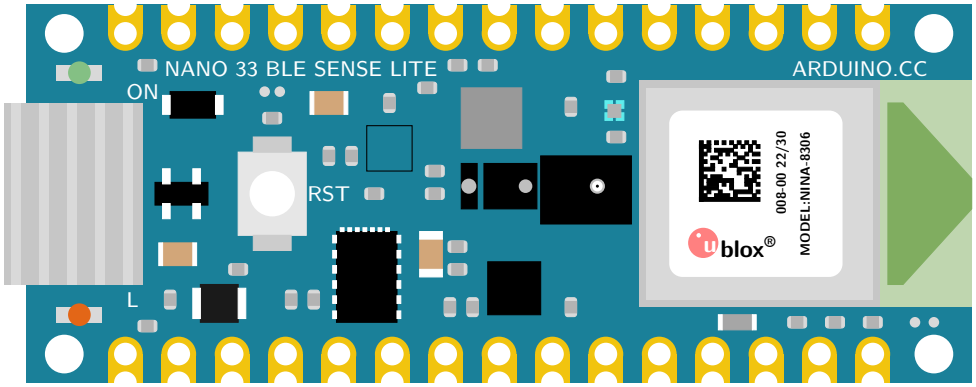


Figure 2.3.: Arduino Nano 33 BLE Sense Lite; the empty blavk square marks the postion of the missing sensor HTS221

2.3. On-Board Sensor Description

Arduino Nano 33 BLE sense come up with the set of embed sensor on the board. The available embed sensors are commonly use for measuring both the analog and digital values around the sorrounding. Arduino Nano 33 BLE sense is very small 45mm × 18mm in size, which makes it very usefull for Internet of things (IOT) and Artificial intelligence (AI) application as a embed device where space is the main constrained issue. It is low power consumption board and operate normally on 3.3 V, we can say that this small size low power consumption board can operate on small batteries even for many months. Due to on-board available sensor, the low power consumption and mini architecture we can use this nano board anywhere. The Arduino Nano 33 BLE Sense is a completely new board on a well-known form factor. For getting detail information about each component of Arduino Nano 33 BLE Sense and data sheets of each sensor the following links give us a detail information [Ard21]. The short description of each sensor are as follow.

- The ADPS-9960 is a digital proximity, ambient light, RGB and gesture sensor. it can measure the proximity distance, light, color and gestures when moving close with the borad.
- The sensor LSM9DS1 is a 9 axis Inertial Measurement Unit (IMU) use as a accelerometre, gyroscope, and magnatometre, this 9 axis sensor is ideal for wearable devices.

- The sensor LPS22HB is a barometric pressure sensor, it measures the environmental pressure which is usefull for simple weather station monitoring.

- The sensor HTS221 senses the relative humidity, and temperature, to get highly accurate measurements of the environmental conditions.

- The sensor MP34DT05 is the digital microphone. it is usefull for capturing, analyzing and detecting the sound in real time.

- The USB port allows you to connect Arduino Nano 33 BLE sense to your machine.

- There are 3 different LEDs that can be accessed on the Nano BLE Sense: RGB Programmable LED , the built-in orange Programmable LED and the Power LED.

The figure 2.4 show the embed sensors on the board, with powerful processor as compared to other arduino boards the nRF52840 from Nordic Semiconductors, a 32-bit ARM® Cortex™-M4 CPU processor running at 64 MHz are as follow.

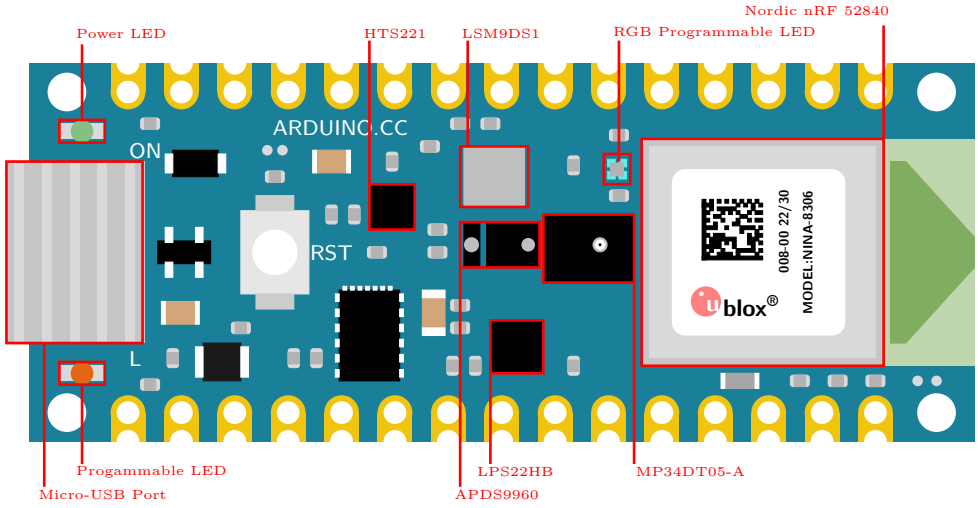


Figure 2.4.

Another point to bear in mind is the overall 'trueness' of sensor readings based on where do you place the actual sensors in the environment, as this could be quite critical. The operating temperature should not exceed 85^0C and not lower than -40^0C. Other factors like humidity level and air pressure values should also be kept in check.

WS:Auch mit dem Rev

MICROCONTROLLER	nRF52840
OPERATING VOLTAGE	3.3V
INPUT VOLTAGE (LIMIT)	21V
DC CURRENT PER I/O PIN	15 mA
CLOCK SPEED	64MHz
CPU FLASH MEMORY	1MB
SRAM	256KB
LED_BUILTIN	13
IMU (Accelerometer, Gyroscope, Magnetometer)	LSM9DS1
MICROPHONE	MP34DT05
GESTURE, LIGHT, PROXIMITY, COLOUR	APDS9960
BAROMETRIC PRESSURE	LPS22HB
TEMPERATURE, HUMIDITY	HTS221

Table 2.1.: Technical Specifications of Arduino Nano 33 BLE Sense [Ard21]

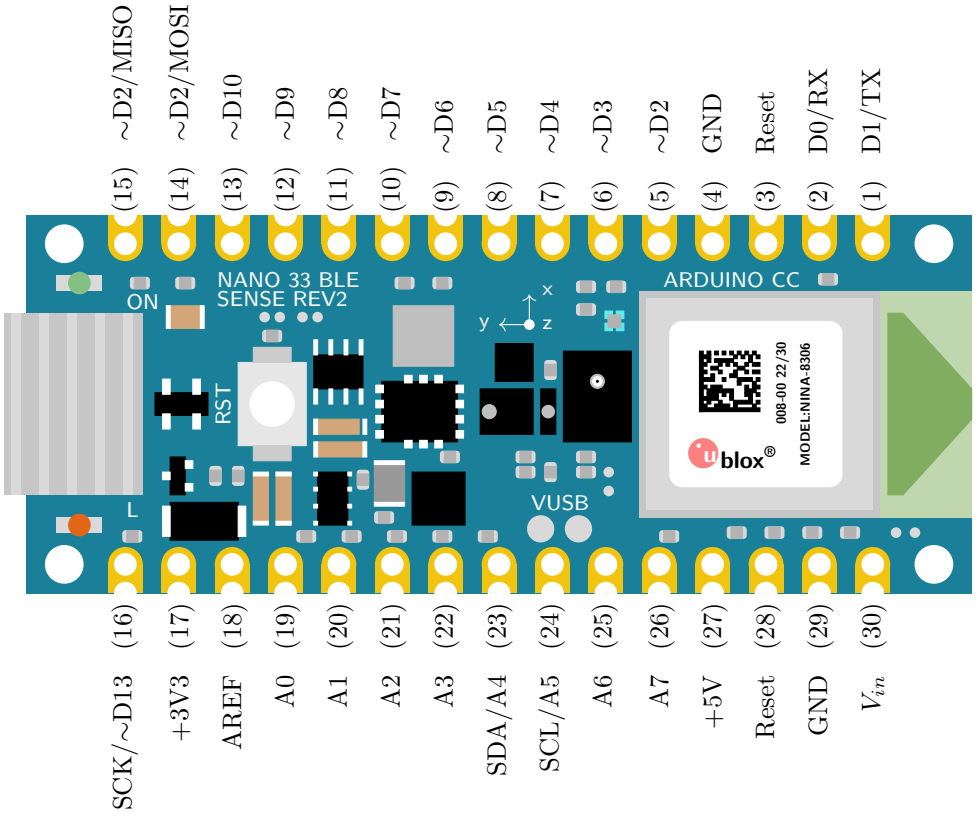


Figure 2.5.: Pin assignment of the Arduino Nano 33 BLE Sense and for the Arduino Nano 33 BLE Sense Rev 2; note that the orange built-in LED is connected to pin D13 and the power LED to pin D1. The built-in RGB LED occupies pins D18, D19 and D20.

WS:Beide Diagramme

2.3.1. Gesture, Proximity, and Color Detection Sensor ADPS-9960

The APDS-9960 device features advanced Gesture detection, Proximity detection, Digital Ambient Light Sense (ALS) and Color Sense (RGB). [Ard21] Gesture detection

utilizes four directional photodiodes to sense reflected IR energy (sourced by the integrated LED) to convert physical motion information (i.e. velocity, direction and distance) to a digital information.
For the following Applications the sensor is in use:

- Gesture Detection
- Color Sense
- Ambient Light Sensing
- Proximity Sensing

2.3.2. Accelerometer, Gyroscope, and Magnetometre Sensor LSM9DS1

The LSM9DS1 is a system-in-package featuring a 3D digital linear acceleration sensor, a 3D digital angular rate sensor, and a 3D digital magnetic sensor. IMU's work by detecting rotational movements across the 3 axis known as Pitch, Roll and Yaw. To achieve the same, it depends on Accelerometer, Gyroscope and Magnetometer. The accelerometer gives the velocity at which the IMU module moves. The gyroscope measure the rotational movement rate on the IMU. Magnetometer measures the force of gravity acting on the IMU.
For the following Applications the IMU is in use:

- Indoor navigation
- Advanced gesture recognition
- Gaming and virtual reality input devices
- Display/map orientation and browsing
- Consumer electronics; Smartphones, tablets, fitness trackers for motion sensing and orientation.
- Compact transportation solutions like Segway.
- Sports Technology - helping athletes to know how they can improve their movements.

There are also certain disadvantages of using the IMU and points that need to be kept in mind while using an IMU sensor. Accumulated error or *'Drift'* is the main disadvantage of IMUs, present due to its constant measuring of changes and rounding off its calculated values off. When such a process happens for a prolonged period of time, it can lead to significant errors. The best way to avoid the *drift* factor is to use a good quality IMU Sensor and make sure the IMU sensor is calibrated. [Stma]

WS:The explanation of drift is to small; citatio

Calibration of an IMU

On research it was found that there are various methods to calibrate the sensors involved, the time period between each calibration is also not defined specifically, however it is advised that regular calibration is done especially when there are strange outputs noticed. Few methods of calibration are briefed below:

WS:What is calibration citations!

Low and High Limit Method

In this method the sensor is rotated in circles along each axis a few times. The midpoint is then found between the two extremes. If there is no offset, the midpoint is close to zero, but if there is a slight deviation from zero, this figure is the hard iron offset, which is the result of the distortion caused by the Earth's magnetic field. This method is mainly used to calibrate the Magnetometer [Mal15]

Magneto V1.2

In this method, the raw magnetometer data is pre-processed with axis specific gain correction to convert the raw output into nanoTesla:

```
Xm_nanoTesla = rawCompass.m.x*(100000.0/1100.0);
Gain X [LSB/Gauss] for selected input field range
Ym_nanoTesla = rawCompass.m.y*(100000.0/1100.0);
Zm_nanoTesla = rawCompass.m.z*(100000.0/980.0);
```

This converted data is saved into the file `Mag_raw.txt` that you open with the Magneto program. To start using this method, we first need to replace the (100000.0/1100.0) scaling factors with values that convert your specific sensors output into nanoTesla. Rather than simply finding an offset and scale factor for each axis, Magneto creates twelve different calibration values that correct for a whole set of errors: bias, hard iron, scale factor, soft iron and misalignment.
A side benefit of this is that it can be used to calibrate accelerometers as well. You might again need to pre-process your specific raw accelerometer output, taking into account the bit depth and G sensitivity, to convert the data into milliGalileo. Then enter a value of 1000 milliGalileo as the "norm" for the gravitational field. [Mal15]

WS:Here, we need more information because we want to use it:

- General infomration about imus
- angle to roation matrix with example!
- calibration
- drift
- ...
- specials for this imu
- description of the parameters for coding
- example code, see Code/-Nano33BLESense/IMU/Arduino

2.3.3. Pressure Sensor LPS22HB

The LPS22HB is an ultra-compact piezoresistive absolute pressure sensor which functions as a digital output barometer.
For the following Applications the sensor is in use:

- Altimeters and barometers for portable devices
- Weather station equipment
- Sports watchs

A sensor element is installed as well as an IC interface that communicates via an I^2C or SPI bus. The function is given in a temperature range from $-40°C$ to $+85°C$. [STM17]

- Absolutdruckbereich: 260 bis 1260 hPa
- Versorgungsspannung: 1,7 bis 3,6 Volt
- 24-bit Druckdatenausgabe
- 16-bit Temperaturdatenausgabe

2.3.4. Relative Humidity and Temperature Sensor HTS221

The HTS221 is an ultra-compact sensor for relative humidity and temperature. It includes a sensing element and a mixed signal to provide the measurement information through digital serial interfaces.
For the following Applications the sensor is in use:

- Air conditioning, heating and ventilation
- Air humidifiers
- Refrigerators
- Smart home automation
- Industrial automation

Dieser Sensor kommuniziert über den I^2C- und SPI-Bus. Dieser Sensor ist einsetzbar in einem Temperaturbereich von $-40°C$ bis $+120°C$. Zur Spannungsversorgung werden 1,7 bis 3,3 Volt benötigt. Eine Temperaturmessung erfolgt mit einer Genauigkeit von $\pm 5°C$. [STM23]

- GND - Ground
- DRDY - Data ready output signal
- SCL/SPC - I2C serial clocl (SCL) & SPI serial port clock (SPC)
- VDD - Stromversorgung
- SDA/SDI/SDO - I^2C serial Data (SDA) & 3 wire-SPI serial data input/output (SDI/SDO)
- SPI enable - I^2C/SPI mode selection

2.3.5. Digital Microphone MP34DT05-A

The MP34DT05-A is an ultra-compact, low-power, omni directional, digital microphone built with a capacitive sensing element and an IC interface. The sensing element, capable of detecting acoustic waves, is manufactured using a specialized silicon micromachining process dedicated to producing audio sensors. The MP34DT05 is a low-distortion microphone with a signal-to-noise ratio of 64 dB. The sensitivity is -26 dBFS ± 3 dB. The Acoustic Overload Point (AOP) is 122.5 dBSPL.
The output signal of the microphone is a PDM signal. This signal is a binary signal modulated by Pulse Density Modulation (PDM) from the analogue signal. [Stmb]
For the following Applications the sensor is in use:

- Speech recognition
- Portable media player
- Mobile Terminal

The Arduino nano 33 BLE Sense has a built-in microphone that uses PDM (Pulse-Density Modulation) to convert sound into digital data. You can use the PDM library to access the microphone data and perform various tasks with it. Here is a simple example of using the builtin microphone for an Arduino nano 33 BLE Sense:
The code 2.1 will print the sound level of the microphone to the serial monitor every 100 milliseconds.

```
1000        // Include the PDM library
            #include <PDM.h>
1002
            // Buffer to store the microphone data
1004        short sampleBuffer[256];

1006        // Variable to store the sound level
            int soundLevel = 0;
1008
            // Callback function for PDM data
1010        void onPDMdata() {
                // Read the PDM data
1012            int bytesAvailable = PDM.available();
                PDM.read(sampleBuffer, bytesAvailable);
1014
                // Calculate the sound level
1016            soundLevel = 0;
                for (int i = 0; i < bytesAvailable / 2; i++) {
1018                soundLevel += abs(sampleBuffer[i]);
                }
1020            soundLevel /= bytesAvailable / 2;
            }
1022
            void setup() {
1024            // Initialize serial communication
                Serial.begin(9600);
1026            while (!Serial);

1028            // Initialize PDM with a sample rate of 16 kHz and 16-bit
    resolution
                PDM.begin(1, 16000);
1030            PDM.onReceive(onPDMdata);
            }
1032
            void loop() {
1034            // Print the sound level to the serial monitor
                Serial.println(soundLevel);
1036            delay(100);
            }
1038
```

Listing 2.1.: Simple example using of the builtin microphone of the Arduino Nano 33 BLE Sense

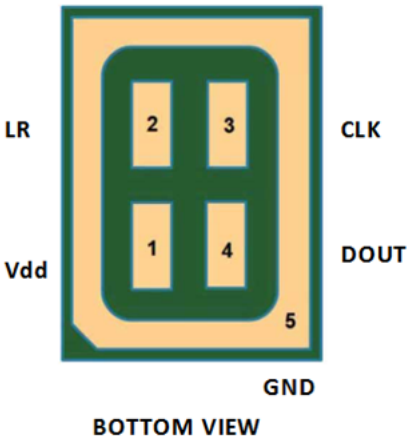


Table 1. **Pin description**

Pin #	Pin name	Function
1	Vdd	Power supply
2	LR	Left/Right channel selection
3	CLK	Synchronization input clock
4	DOUT	Left/Right PDM data output
5 (ground ring)	GND	Ground

Figure 2.6.: Circuit diagram microphone [Stmb]

You can modify this code to perform other tasks with the microphone data, such as controlling LEDs, playing sounds, or sending data to other devices. For more information and examples, you can check out the PDM library documentation or the Arduino nano 33 BLE Sense page.

WS:How to install the library PDM description of the lib functions

2.3.6. Bluetooth Module nRF52840

The nRF52840 is an advanced, highly flexible single chip solution for today's increasingly demanding Ultra Low Power (ULP) wireless applications for connected devices on our person, connected living environments and the Internet of Things (IoT) at large. It is designed ready for the major feature advancements of Bluetooth 5 and takes advantage of Bluetooth 5's increased performance capabilities. [Ard21]

Applications

- Smart Home products
- Industrial mesh networks
- Smart city infrastructure
- Connected watches
- Advanced personal fitness devices
- Wearables with wireless payment
- Connected Health

2.4. Bluetooth Module INA B306

The module is a Bluetooth Low Energy (BLE). The connection with our smartphone is possible, to do this we have to use application “Nordic Semiconductors nRF Connect” They used 5.0 generation bluetooth. 2.4

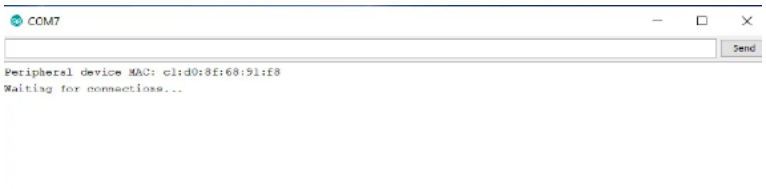


Figure 2.7.: Connection with BLE and smartphone

We can use for example to share Arduino batterie on smartphone, 2.8 here you see an code example for how we can do this.

Figure 2.8.: Simple sketch to seen Arduino battery

```
1000 #include <ArduinoBLE.h>

1002 BLEService batteryService("1101");
     BLEUnsignedCharCharacteristic batteryLevelChar("2101"
1004 BLERead | BLENotify);

1006 void setup() {
       Serial.begin(9600);
1008   while (!Serial);

1010   pinMode(LED_BUILTIN, OUTPUT);
       if (!BLE.begin()) { //Searche connection but they wasn't function
1012     Serial.println("Starting BLE failed!");
         while (1);
1014   }

1016   BLE.setLocalName("BatteryMonitor");
       BLE.setAdvertisedService(batteryService);
1018   batteryService.addCharacteristic(batteryLevelChar);
       BLE.addService(batteryService);
1020   BLE.advertise();
       Serial.println("Bluetooth device active, waiting for  connections...")
         ;
1022   }

1024 void loop() {
       BLEDevice central = BLE.central();
1026   if (central) {
         Serial.print("Connected to central: ");
1028     Serial.println(central.address());
         digitalWrite(LED_BUILTIN, HIGH);
1030     while (central.connected()) {
           int battery = analogRead(A0);
1032       int batteryLevel = map(battery, 0, 1023, 0, 100);
           Serial.print("Battery Level is now: ");
1034       Serial.println(batteryLevel);
           batteryLevelChar.writeValue(batteryLevel);
1036       delay(200);
         }
1038     digitalWrite(LED_BUILTIN, LOW);
         Serial.print("Disconnected from central: ");
1040     Serial.println(central.address());
       }
1042 }
```

../../Code/Nano33BLESense/Bluetooth/bluetooth.ino

2.5. Arduino Nano 33 BLE Pin Configuration

Arduino Nano 33 BLE is an advanced version of Arduino Nano board that is based on a powerful processor the nRF52840. The figure 2.9 shows that the board has the following pin configuration. Pin Configuration

Digital pin The number of digital I/O pins are 14 which receive only two values HIGH or LOW. These pins can either be used as an input or output based on the requirement. When these pins receive 5V, they are in a HIGH state and when they receive 0V they are in a LOW state.

Analog pin Total 8 analog pins available on the board A0 – A7. These pins get any value as opposed to digital pins that only receive two values HIGH or LOW. These pins are used to measure the analog voltage ranging between 0 to 5V.

PWM pin All digital pins can be used as PWM pins. These pins generate analog results with digital means.

SPI pin The board supports serial peripheral interface (SPI) communication protocol. This protocol is employed to develop communication between a controller and other peripheral devices like shift registers and sensors. Two pins are used for SPI communication i.e. Master Input Slave Output (MISO) and Master Output Slave Input (MOSI) are used for SPI communication. These pins are used to send or receive data by the controller.

I2C pin The board carries the I2C communication protocol which is a two-wire protocol. It comes with two pins SDL and SCL.

UART pin The board features a UART communication protocol that is used for serial communication and carries two pins Rx and Tx. The Rx is a receiving pin used to receive the serial data while Tx is a transmission pin used to transmit the serial data.

External Interrupts pin All digital pins can be used as external interrupts. This feature is used in case of emergency to interrupt the main running program with the inclusion of important instructions at that point.

LED at Pin 13 and AREF pin There is an LED connected to pin 13 of the board. And AREF is a pin used as a reference voltage for the input voltage.

2.6. Was fehlt

- Beschreibung der Sensoren
  - Hintergrundwissen
  - Beschreibung
  - Eigenschaften
  - Kalibrierung
- Beschreibung der Bibliothek
  - Beschreibung
  - Installation
  - Funktionen

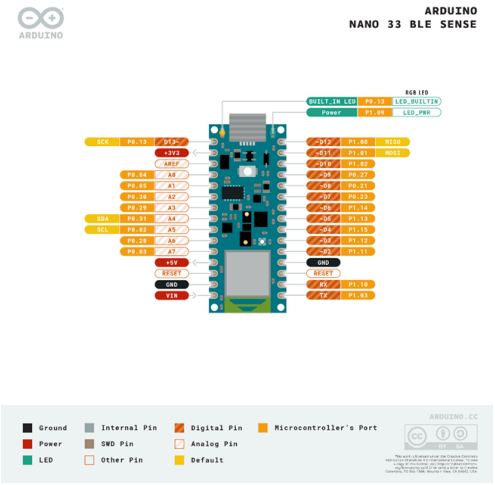


Figure 2.9.: Arduino Nano 33 BLE Pin Configuration

- Laufzeit, Speicherplatz, ...
- Software-Dokumentation
- Verwendung von Werkzeugen, z.B. Doxygen
  - Beschreibung, inklusive im Quellcode; z.B. Header
  - Handbuch
  - Ablaufdiagramm
  - ...
- Interpretation der Ergebnisse
- Literatur, Bilder
- ...

3. Reset Button on the Arduino Nano 33 BLE Sense

The reset button on the Arduino Nano 33 BLE Sense plays a crucial role in managing the board's operation. It allows for restarting the board, entering Bootloader Mode, and addressing issues related to the program or functionality. The following section outlines its functions, appropriate use cases, and technical specifications.

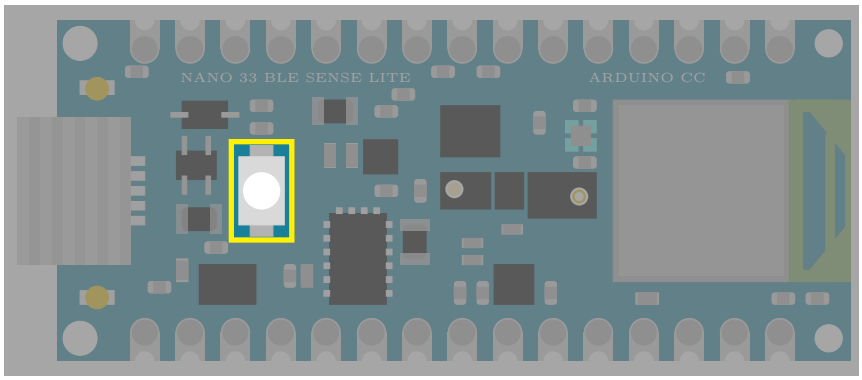


Figure 3.1.: Arduino Nano 33 BLE Sense's Reset Button

3.1. Purpose and Functionality

The reset button on the Arduino Nano 33 BLE Sense is positioned on the top side of the board, close to the USB connector. This small tactile push button communicates with the nRF52840 microcontroller, performing specific actions depending on the timing and sequence of the presses.

3.1.1. Functions of the Reset Button

- System Reset: A single press of the reset button triggers a system reset. This action momentarily interrupts the power to the microcontroller, restarting the board and reinitializing the uploaded program.
- Bootloader Mode: The reset button also allows entry into Bootloader Mode, which is essential for uploading new sketches or recovering the board when it becomes unresponsive due to a faulty program.
  - Activation: To activate Bootloader Mode, press the reset button twice in quick succession (within approximately 1 second). The double-press sequence triggers the bootloader, which prepares the board to receive new firmware or sketches via the USB interface.
  - Indication: When Bootloader Mode is active, the built-in LED blinks quickly, indicating that the board is ready for a firmware upload.
  - Purpose: Bootloader Mode is particularly helpful for uploading sketches when the Arduino IDE cannot recognize the board and for recovering from unresponsive or faulty programs that disrupt normal functionality.

-

3.2. Timing and Sequence

The reset button's behavior depends on the timing and sequence of presses:

- Single Press: Executes a system reset, restarting the microcontroller and the loaded sketch.
- Double Press: Activates Bootloader Mode, readying the board for new firmware uploads.
  - The timing of the double-press must be precise (quickly within 1 second); otherwise, the board will perform only a standard reset.

3.3. Use Cases

The reset button provides critical functionality in several scenarios:

1. Restarting the Board: During development, a single press allows developers to restart the board and observe the program from its initial state.
2. Uploading Sketches: When the Arduino IDE fails to detect the board or cannot upload a sketch, entering Bootloader Mode manually resolves the issue.
3. Recovering from Errors: If a faulty sketch causes the board to freeze or become unresponsive, Bootloader Mode enables users to bypass the issue and upload a new program.

3.4. Conclusion

The reset button on the Arduino Nano 33 BLE Sense is a vital feature for developers, providing tools to restart the board, enter Bootloader Mode, and recover from programming errors. Its dual functionality—system reset and Bootloader Mode activation—ensures flexibility and reliability during development and debugging. The requirement for a precise double-press to activate Bootloader Mode enhances user control, making it a robust tool for managing the board.

4. Built-inLED

Some Arduino boards have a LED on board which can use for the application. [Ard24a; Ard23a; Ard23b]

4.1. General Information

LEDs are used in the applications. Depending on the action performed by a user or the sketch, corresponding feedback can be provided. This can take the form of the LED switching on, switching off or flashing. [Kur21]

4.2. Built-in LED

The built-in LED of the Arduino Nano 33 BLE Sense is a single orange LED that is connected to pin 13 on the board. The built-in LED is active-high, which means that setting the pin to `HIGH` will turn the LED on, and setting the pin to `LOW` will turn the LED off. The built-in LED can be used for various purposes, such as indicating the status of the board, displaying sensor data, creating visual effects, or debugging. [Ard23a; Ard23b; Arda]

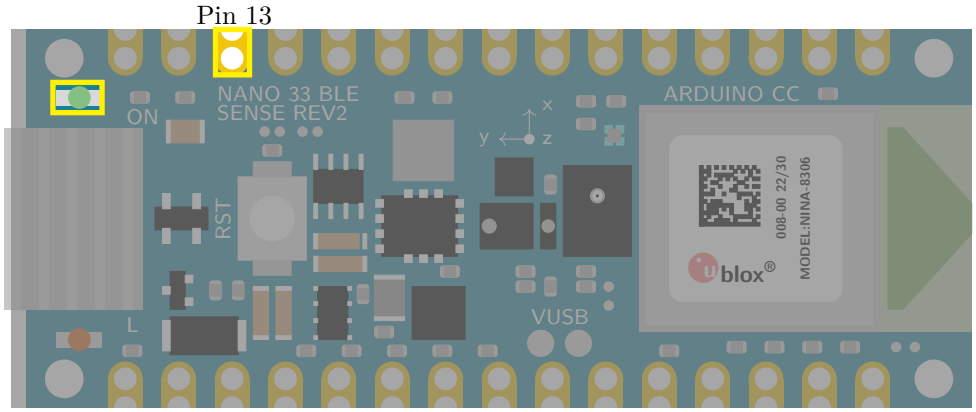


Figure 4.1.: Arduino Nano 33 BLE Sense's built-in LED with Pin 13

4.3. Specification

4.3.1. Pin Assignment

The built-in LED is a orange LED and connected to pin 13. The brightness can not be controlled. [Ard23a; Ard23b]

Built-in LED: `LED_BUILTIN = 13u`

Built-in LED is active-high and connected to pin 13.
The built-in LED can be controlled programmatically by setting the pin to `HIGH` or `LOW`.

The pin 13 must be defined as an output in the function `setup` by setting `pinMode (13, OUTPUT)`, otherwise the LED cannot be switched on.

The pin 13 can also be used otherwise. Then the LED is not in use.

4.3.2. Power Comsumption

The power comsumption has to be considered. It is the same value as for the power LED, see section 5.3.2.

4.4. Simple Code

4.4.1. Simple Code for the Built-in LED

For using the built-in LED, a simple library is introduced here.

In the header file `BuiltinLED.h`, see code **??**, are the declaration of the variables and the functions. A variable is connected to pin 13. The pin 13 is defined as an output in the function `BuiltinLEDinit`. There are 2 functions:

1. `BuiltinLEDinit`: This funtion initializes the digital pin 13 of the built-in LED as an output.
2. `BuiltinLED`: This funtion switches the built-in LED on or off.

Listing 4.1.: Header file without comments for using the Built-in LED

```
1000 #ifndef LEDBuiltin_h
     #define LEDBuiltin_h
1002
1004
     /**
1006
     /**
1008
     #endif
```

../../Code/Nano33BLESense/LEDs/LEDBuiltin.h

In the file `BuiltinLED.cpp`, see code 4.2, the functions are implemented.

Listing 4.2.: Code file without comments for using the Built-in LED

```
1000 #include <LEDGeneral.h>
     #include <LEDBuiltin.h>
1002 *   @return 0 - success
     *   @return 1 - error
1004 */
     int LEDBuiltinsetup()
1006 {
       // initialize digital pin LED_BUILTIN as an output.
1008   LEDSetup(LED_BUILTINLED);
     *
1010 *   @return 0 - success
     *   @return 1 - error
1012 */
     int LEDBuiltin(bool On)
1014 {
```

../../Code/Nano33BLESense/LEDs/LEDBuiltin.cpp

4.4.2. Simple Code for Testing the Built-in LED

In the sketch 4.3, a variable is connected to pin 13. The pin is defined as an output in the function `setup`. In the function `loop`, the LED is switched on for 1 second and switched off for 1 second so that the LED flashes accordingly.

Listing 4.3.: Simple sketch without comments to control the built-in LED

```
1000 *  Turns the built-in LED on for one second, then off for one second,
         repeatedly.
     *
1002 *  The LED is switched on for 1 second and switched off
     *
1004 *  Initialization of the pin LED_BUILTIN as output
     *
1006 *  @param ---
     *
1008 *  swichting the led on / off for 1sec.
     *
1010 *  @param ---
     *
1012 *  @return void
     */
1014 void loop() {
         // Turn the LED on
1016     LEDBuiltin(false);
         // Wait for one second
```

../../Code/Nano33BLESense/LEDs/examples/TestLEDBuiltin/TestLEDBuiltin.ino

This is just a simple example. The variable `LED_BUILTIN` is already defined, so the assignment is not necessary. The command `delay` should be avoided in an Arduino sketch. Instead, variables of the type `elapsedMillis` should be used.

4.5. Tests

4.5.1. Simple Function Test

The simplest test is the flashing of the LED at 2 Hz, see sketch 4.4.

Listing 4.4.: Simple sketch to test the built-in LED

```
1000 *  Turns the built-in LED on for one second, then off for one second,
         repeatedly.
     *
1002 *  The LED is switched on for 1 second and switched off
     *
1004 *  Initialization of the pin LED_BUILTIN as output
     *
1006 *  @param ---
     *
1008 *  swichting the led on / off for 1sec.
     *
1010 *  @param ---
     *
1012 *  @return void
     */
1014 void loop() {
         // Turn the LED on
1016     LEDBuiltin(false);
         // Wait for one second
```

../../Code/Nano33BLESense/LEDs/examples/TestLEDBuiltin/TestLEDBuiltin.ino

4.5.2. Test all Functions

As the built-in does not allow any special manipulations, all functions are covered with the sketch 4.4.

4.6. Simple Application

In many applications, it is necessary to save power so that the LED is not switched on continuously. Nevertheless, feedback is required as to whether the system, e.g. a fire detector, is switched on and functional. In this case, the LED is switched on for 1 second at a defined time interval, in this case 30 seconds. This is implemented in the example 4.5.

Listing 4.5.: A simple watch dog: A simple watchdog: the built-in LED is switched on for 1 second every 30 seconds.

```
1000 #include <../LEDs/BuiltinLED.h>
     #include <../LEDs/LED.h>
1002 #include <../LEDs/SignsOfLife.h>


1004
     #define CycleTimeOn 1000 /*< Duty cycle[ms] */
1006
     #define CycleTimeOff 29000 /*< Switch-off time [ms] */
1008 void setup() {
       // Initialize the function SignsOfLife
1010   SignsOfLifeInit(LED_BUILTIN, CycleTimeOn, CycleTimeOff)

1012   // Application
       // ...
1014 }
     void loop() {
1016   // Switch builtin LED on/off
       SignsOfLife();
1018
       // Application
1020   // ...
     }
```

../../Code/Nano33BLESense/Test/TestLEDBuiltinApplication.ino

4.7. Further Readings

- Babu, Neelakandan Ramesh and Chenchireddy, Kalagotla and Reddy, V. Harsha Vardhan and Samhitha, Dr. and Apparao, P. and Kalyan, Ch. Pavan: *Case Study on Ni-MH Battery*. 2023 2nd International Conference on Applied Artificial Intelligence and Computing (ICAAIC), 2023. [Bab+23]
- Scherer, Thomas: *Elektor special: Stromversorgung und Batterien*. Elektor Special. 2022. [Sch22]

5. Power LED

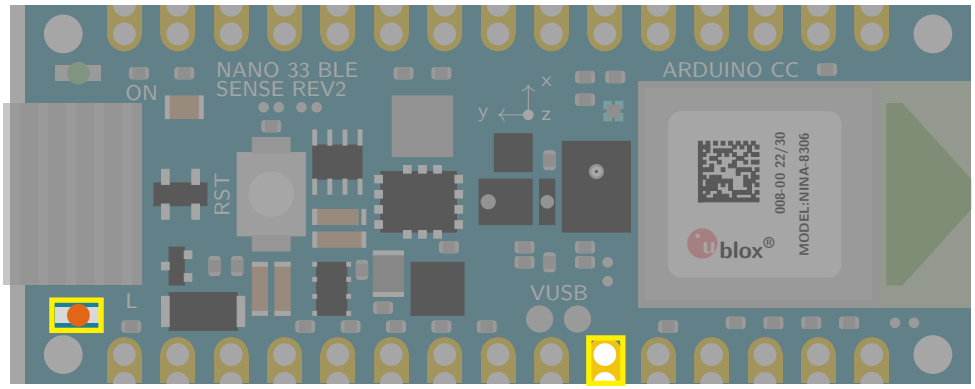
Arduino boards have a power LED on board. Normally, the power LED indicates the status of the board.

5.1. General Information

The power LED on an Arduino board is a small, usually red or green, Light Emitting Diode (LED) that indicates whether the board is receiving power. It is typically located near the USB connector on the board. When the board is powered on, the LED will illuminate. The specific color and behavior of the power LED may vary depending on the Arduino board model. For example, on the Arduino Uno, the power LED is red and illuminates steadily when the board is powered on. On the Arduino Nano, the power LED is green and blinks slowly when the board is powered on. The power LED is a helpful visual indicator that can be used to troubleshoot power supply issues. If the power LED is not illuminated, it means that the board is not receiving power. This could be due to a number of reasons, such as a loose USB connection, a damaged power supply, or a problem with the board itself. By checking the power LED, you can quickly identify and resolve power supply problems with your Arduino board. [Ard24a; Ard23a; Ard23b; Arda; Kur21]

5.2. Power LED

While labeled as a power LED, the single green LED on the Nano 33 BLE Sense isn't solely for power indication. It has multi-functionality depending on board state and user code interaction. During typical operation, it lights steadily green when powered, similar to other Arduino models. However, it flashes rapidly during serial communication and bootloader mode. Notably, when user code enters deep sleep mode, the LED turns off entirely for power saving. This includes power status, communication activity, and sleep mode activation. Understanding these LED behaviors can aid in troubleshooting and code debugging. The green power LED's primary function remains indicating power and basic board states. [Ard24a; Ard23a; Ard23b; Arda]



Pin 25

Figure 5.1.: Arduino Nano 33 BLE Sense's Power LED with Pin 25

5.3. Specification

5.3.1. Pin Assignment

The power LED is a green LED and connected to pin 25. The brightness can be controlled via Pulse with Modulation (PWM). [Ard23a; Ard23b]

Power LED: `LED_PWR = 25u`

Power LED is active-high and connected to pin 25.
The power LED can also be controlled programmatically by setting the pin to `HIGH` or `LOW`.
The pin must be defined as an output in the function `setup` by setting `pinMode (LED_PWR, OUTPUT)`, otherwise the LED cannot be switched on.

5.3.2. Power Comsumption

The power comsumption has to be considered. There, the power and current in a simple circuit using a resistor and an LED has to calculated.
The led onboard works in a circuit with an 200 Ohm resistor, a 5V power source from an Arduino, and an LED with 2V across it. To find out how much power is being used by the LED and the resistor, the current must be calculated in the first step. Using Ohm's Law $V = I \cdot R$ we can find the current flowing through the resistor. Since 3V is across the resistor, we can plug in the values:

$$\frac{3\text{V}}{200\Omega} = 0.015\text{A} = 15\text{mA}$$

This means that 15mA of current is flowing through the resistor and the LED. In the next step, we calculate the power. Now that we know the current, we can calculate the power dissipated in the LED and the resistor.

- For the LED: $2\text{V} \times 15\text{mA} = 30\text{mW}$
- For the resistor: $3\text{V} \times 15\text{mA} = 45\text{mW}$
- Total Power:

  To find the total power coming out of the Arduino, we multiply the voltage by the current:

  $5\text{V} \times 15\text{mA} = 75\text{mW}$

Notice that the total power (75mW) is equal to the sum of the power dissipated in the LED (30mW) and the resistor (45mW). This makes sense, since all the power coming out of the Arduino is being used by either the LED or the resistor.

The pin 25 can also be used otherwise. Then the LED is just switched on if the board is connected to a power source.

5.4. Simple Code

5.4.1. Simple Code for LEDs

Using of LEDs is a standard problem for Arduino's programmer. Therefore, a simple library is introduced here.

In the header file `LED.h`, see code 5.1, are the declaration of the functions. There are 3 functions:

1. LEDinit: This funtion initializes a digital pin of the LED as an output.
2. LED: This funtion switches the LED on or off.
3. LEDBrightness: This funtion sets the LED's brightness.

Listing 5.1.: Header file without comments for using a LED

```
1000 #define LED_h
1002
1004 /**
     *  @brief initialize digital pin of the LED as an output.
1006 *  @brief function for switching on or off the LED
     *
1008 /**
     *  @brief setting the brightness
1010
     #endif
```

../../Code/Nano33BLESense/LEDs/LEDGeneral.h

In the file LED.cpp, see code 5.2, the functions are implemented.

Listing 5.2.: Code file without comments for using a LED

```
1000 #include "LEDGeneral.h"
     #include <Arduino.h>
1002 *
     *  @return 0 - success
1004 *  @return 1 - error
     */
1006 int LEDSetup(int PinNo) {
       // initialize digital pin LED_BUILTIN as an output.
1008   pinMode(PinNo, OUTPUT);
     *
1010 *  @return 0 - success
     *  @return 1 - error
1012 */
     int LED(int PinNo, bool On)
1014 {
       if (On)
1016   {
         digitalWrite(PinNo, HIGH);    // turn the LED on (HIGH is the voltage
          level)
1018   } else
       {
1020     digitalWrite(PinNo, LOW);     // turn the LED off by making the
         voltage LOW
       }
1022 *
     *  @return 0 - success
1024 *  @return 1 - error
     */
1026 int LEDBrightness(int PinNo, int setBrightness)
     {
1028   int b;

1030   if (setBrightness <= 0) {
         b = 0;
1032   } else
       if (setBrightness >= 255) {
1034     b = 255;
       } else
1036   {
         b = setBrightness;
1038   }
```

```
1040   analogWrite(PinNo, b);
```

../../Code/Nano33BLESense/LEDs/LEDGeneral.cpp

5.4.2. Simple Code for the Power LED

For using the power LED, a simple library is introduced here.

In the header file `PowerLED.h`, see code 5.3, are the declaration of the variables and the functions. A variable is connected to pin 25. The pin 25 is defined as an output in the function `PowerLEDinit`. There are 2 functions:

1. `PowerLEDinit`: This funtion initializes a the digital pin 25 of the power LED as an output.
2. `PowerLED`: This funtion switches the power LED on or off.

Listing 5.3.: Header file with doxygen comments for using the Power LED

```
1000 #ifndef LEDPower_h
     #define LEDPower_h
1002
1004
     /**
1006
     /**
1008
     #endif
```

../../Code/Nano33BLESense/Nano33BLESenseLED/LEDPower.h

In the file `PowerLED.cpp`, see code 5.4, the functions are implemented.

Listing 5.4.: Code file with doxygen comments for using the Power LED

```
1000 #include <LEDGeneral.h>
     #include <LEDPower.h>
1002
     #define LED_PWR 25 /*< Define the pin for the power LED */
1004 *  @return 1 - error
     */
1006 int LEDPowersetup()
     {
1008   // initialize digital pin LED_BUILTIN as an output.
       LEDSetup(LED_PWR);
1010 *  @return 0 - success
     *  @return 1 - error
1012 */
     int LEDPower(bool On)
1014 {
       LED(LED_PWR, On);    // turn the LED on = true or off = false
```

../../Code/Nano33BLESense/Nano33BLESenseLED/LEDPower.cpp

5.4.3. Simple Code for Testing the Power LED

In the sketch 5.5, the power LED is tested. First, the function `PowerLEDinit` is called in the function `setup`. In the function `loop`, the power LED is switched on for 1 second and switched off for 1 second so that the power LED flashes accordingly.

Listing 5.5.: Simple sketch to control the power LED

```
1000 *
     */
1002  * @brief Setup function runs once when the sketch starts.
      *
1004  * Initializes the power LED for use in the main loop. The pin for the
        LED is configured here.
      */
1006 void setup() {

1008   LEDPower.turnOff(); ///< Turns off the power LED
       delay(500);         ///< Wait for 500 milliseconds (LED is OFF)
1010 }
```

../../Code/Nano33BLESense/LEDs/examples/TestLEDPower/TestLEDPower.ino

This is just a simple example. The variable `LED_PWR` is already defined, so the assignment is not necessary. The command `delay` should be avoided in an Arduino sketch. Instead, variables of the type `elapsedMillis` should be used. [Ard24b]

5.4.4. Test all Functions

The brightness of the power LED can be controlled. This is demonstrated in the example sketch 5.6.
Using the pulse width modulation, the brightness is gradually increased to the maximum value and then gradually reduced to 0 again.

Listing 5.6.: Simple sketch to check the battery state using the power LED

```
1000 #incude <../LEDs/PowerLED.h>
     #incude <../LEDs/LED.h>
1002
     int Brightness = 0; /*< Define the initial brightness values  (0-255) */
1004
     int redStep = 5; /*< Define the increment/decrement value  */
1006 void setup() {
         // Initialize the pin as an output
1008     PowerLEDinit();
     }
1010 void loop() {
       // Write the PWM values to the LED pin
1012   LEDBrightness(LED_PWR, redBrightness);

1014   // Update the brightness values
       Brightness += Step;
1016
       // Check if the brightness values are out of range and reverse the
         direction
1018   if (Brightness <= 0 || Brightness >= 255) {
         Step = -Step;
1020   }

1022   // Wait for 10 milliseconds
       delay(10);
1024 }
```

../../Code/Nano33BLESense/Test/TestLEDPowerBrightness.ino

5.5. Simple Application

There are different situations where it might be useful to program the power LED of the Arduino Nano. For example, you could use it to:

- Indicate the status of the board, such as whether it is connected to a power source, a computer, or a sensor.
- Display the battery level of the board, by changing the brightness or color of the power LED.
- Create a visual alarm or notification, by making the power LED blink or flash in a certain pattern.

The next sketch is a frame for applications with a sign of life. The power LED is switched on for 1 sec and off for 29 sec. The file `SignsOfLife.h`, see 5.8, contains the declarations and the file `SignsOfLife.cpp` contains the implementations.

Listing 5.7.: Simple code for signs of life

```
1000 #ifndef LEDSignsOfLife_h
     #define LEDSignsOfLife_h
1002
1004
     /**
1006  *  @brief initialize digital pin of the LED as an output.
      *
1008  *  call the function once, in the function setup, when you press reset
          or power the board
      *
1010  *  Initialization of the pin  as an output
      *
1012  *  The program doesn't check if the parameter PinNo is reasonable.
      *
1014  *  @param PinNo - number of pin for the LED
      *  @return false - LED's actual state is off
1016  */
     extern bool SignsOfLife();
```

../../Code/Nano33BLESense/LEDs/LEDSignsOfLife.h

The library contains one function for initialization and one function `SignsOfLife`, which controls the LED.

Listing 5.8.: Simple code for signs of life

```
1000 #include "LEDGeneral.h"
     #include "LEDSignsOfLife.h"
1002 #include <Arduino.h>

1004 /**
      *  @brief initialize digital pin of the LED as an output.
1006  *
      *  call the function once, in the function setup, when you press reset
          or power the board
1008  *
      *  Initialization of the pin  as an output
1010  *
      *  The program doesn't check if the parameter PinNo is reasonable.
1012  *
      *  @param PinNo - number of pin for the LED
1014  *  @param CycleTimeOn - duration of the LED is on
      *  @param CycleTimeOff - duration of the LED is off
1016  *
      *  @return true - LED's actual state is on
1018  *  @return false - LED's actual state is off
      */
1020 bool SignsOfLifeSetup(int PinNo, int CycleTimeOn, int CycleTimeOff)
     {
1022   LEDSetup(PinNo);
```

```
1024   SignsOfLifePinNo = PinNo;

1026   LED(SignsOfLifePinNo, SignsOfLifeState);

1028   return SignsOfLifeState;
}
1030
/**
1032 *  @brief function for blinking the LED
 *
1034 *
 *  The function checks the duration of the state. If the state has to
     change, then the function changes the state.
1036 *

1038 *
 *  @return true - LED's actual state is on
1040 *  @return false - LED's actual state is off
 */
1042 bool SignsOfLife()
{
1044
   // Check if CycleTimeOff have passed since the last LED turn on
```

../../Code/Nano33BLESense/LEDs/LEDSignsOfLife.cpp

A simple application is to check the condition of the battery. The sktech 5.9 demonstrates, if the voltage drops too low, the power LED flashes, see [Sch22] for more information.

Listing 5.9.: Simple sketch to check the battery state using the power LED

```
1000 #include <Arduino.h>
#incude <../LEDs/PowerLED.h>
1002 #incude <../LEDs/LED.h>

1004 const int BATTERY_PIN = A0; /*< Battery measurement pin */

1006 const float REFERENCE_VOLTAGE = 3.3; /*< Reference voltage for 3.3V (
     adjust if using external power) */
void setup() {
1008
   SignsOfLifeInit(LED_PWR, 500, 500);
1010   PowerLED(true);
}
1012 int BatteryState(bool SendMessages)
{
1014   int ret;
   // Read battery voltage from analog pin
1016   float rawVoltage = analogRead(BATTERY_PIN) * (REFERENCE_VOLTAGE /
     1023.0);

1018   // Calculate percentage based on reference voltage (adjust based on
     battery specs)
   float batteryPercentage = (rawVoltage / 4.2) * 100.0;
1020
   if (SendMessages)
1022   {
     // Print battery voltage and percentage (for debugging)
1024     Serial.print("Battery voltage: ");
     Serial.print(rawVoltage);
1026     Serial.println(" V");
     Serial.print("Battery percentage: ");
1028     Serial.print(batteryPercentage);
     Serial.println("%");
1030   }

1032   // Optional: blink LED based on battery level (adjust thresholds)
```

```
  if (LED_PWR >= 0) {
1034    if (batteryPercentage < 20) {
          digitalWrite(LED_PWR, HIGH);
1036      delay(500);
          digitalWrite(LED_PWR, LOW);
1038      delay(500);
        ret = 1;
1040    } else {
          digitalWrite(LED_PWR, HIGH);
1042    ret = 0;
        }
1044  }
  else
1046  {
  ret = 0;
1048  }
  return ret;
1050 }

1052 void loop() {

1054   BatteryState(false);

1056   // Delay between measurements
  delay(5000);
1058 }
```

../../Code/Nano33BLESense/Test/TestLEDPowerBattery.ino

5.6. Further Readings

- Kurniawan, Agus: *IoT Projects with Arduino Nano BLE Sense: Step-By-Step Projects for Beginners*. Apress, 2021. [Kur21]
- Kühnel, Claus: *Arduino - Das umfassende Handbuch*. Rheinwerk Verlag GmbH, 2024. [Küh24]
- Smythe, Richard J.: *Advanced Arduino Techniques in Science - Refine Your Skills and Projects with PCs or Python-Tkinter*. Apress, 2021. [Smy21a]
- Smythe, Richard J.: *Arduino in Science - Collecting, Displaying, and Manipulation Sensor Data*. Apress, 2021. [Smy21b]

6. Built-in RGB-LED

Some Arduino boards have a RGB-LED on board which can use for the application. ARG-LED is at least thre LEDs in on.

6.1. General Information

An RGB LED is a type of LED that can emit different colors of light. RGB stands for red, green, and blue, which are the three primary colors of light. An RGB LED consists of three individual LEDs inside a single package, each with its own color and pin. By varying the brightness of each LED using PWM signals, an RGB LED can produce a wide range of colors by mixing the primary colors. An RGB LED is usually active-low, which means that setting the pin to LOW will turn the LED on, and setting the pin to HIGH will turn the LED off. [Ber18; HEG21; KKV14]

6.2. Specific Sensor

The onboard LED of the Arduino Nano 33 BLE Sense is a RGB-LED that consists of three individual LEDs: red, green, and blue. Each LED is connected to a different pin on the board:

- Pin 22 for red,
- Pin 23 for green, and
- Pin 24 for blue.

The RGB-LED can produce different colors by varying the brightness of each LED using PWM signals. The RGB-LED is active-low, which means that setting the pin to `LOW` will turn the LED on, and setting the pin to `HIGH` will turn the LED off. This is different from the power LED and the built-in LED, which are both active-high. [Ard24a; Ard23a; Ard23b]

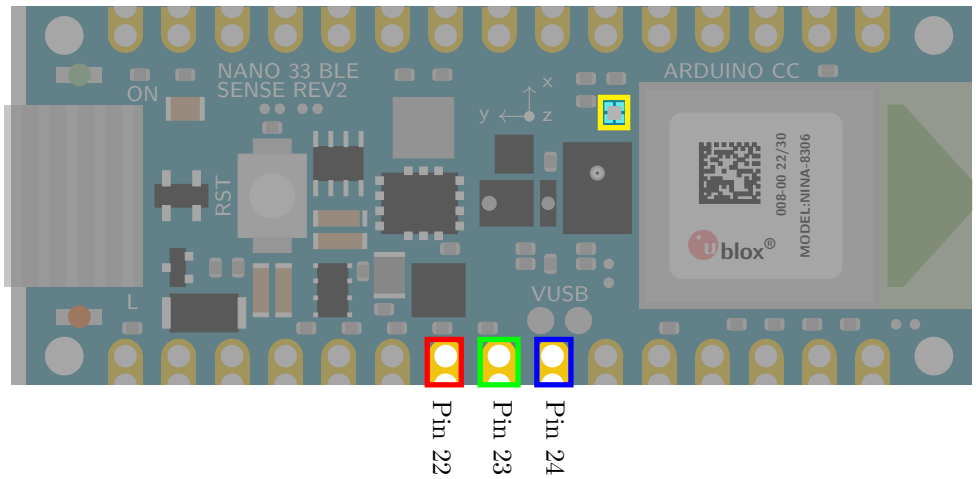


Figure 6.1.: Arduino Nano 33 BLE Sense's RGB LED with Pin 22, Pin 23, and Pin 24

The brightness of the LED varies between 5000-6000 Millicandela (mcd) at a current of 20 mA. The color temperature is controllable and can be set in a range of 5000-6000 Kelvin. The LED should be stored and used between -40°C and +85°C.

6.3. Specification

6.3.1. Pin Assignment

The RGB LED occupies pins on the board internally. These pins are defined via variable names `LEDR`, `LEDG`, and `LEDB`. [Ard23a; Ard23b]
This must be observed when using the pins.

Red LED: LEDR = Pin 22

Green LED: LEDG = Pin 23

Blue LED: LEDB = Pin 24

RGB LED is active-low and connected to pin 22, 23, and 24.
The RGB LED can be controlled programmatically by setting the pin to `LOW` or `HIGH`. The pins 22, 23, and 24 must be defined as an output in the function `setup` by setting `pinMode (22, OUTPUT)`, `pinMode (23, OUTPUT)` and `pinMode (22, OUTPUT)`, otherwise the LED cannot be switched on.

The pins 22, 23, 24 can also be used otherwise. Then the LED is not in use.

6.3.2. Power Comsumption

The power comsumption has to be considered. It is the same value as for the power LED, see section 5.3.2. Here, a RGB-LED has 3 LEDs inside, therefore the poswer consumption must be consiederd for each color.

6.4. Simple Code

In the header file `RGBLED.h`, see code 6.1, are the declaration of the variables and the functions. Variables are connected to pin 22, pin 23, and pin 24. The pins are defined as output in the function `RGBLEDinit`. There are the following functions:

1. `RGBLEDinit`: This funtion initializes the digital pin 22, pin 23, and pin 24 of the RGB-LED as output.
2. `RGBLED_Red`: This funtion switches the red part of the RGB-LED on or off.
3. `RGBLED_Green`: This funtion switches the green part of the RGB-LED on or off.
4. `RGBLED_Blue`: This funtion switches the blue part of the RGB-LED on or off.
5. `RGBLED_Color`: This functions combines the colors.

Listing 6.1.: Header file without comments for using the RGB-LED

```
1000 #ifndef LEDRGB_h
     #define LEDRGB_h
1002
1004
     /**
1006
```

```
/**

/**
```

../../Code/Nano33BLESense/LEDs/LEDRGB.h

In the file RGBLED.cpp, see code 6.2, the functions are implemented.

Listing 6.2.: Code file without comments for using the RGB-LED

```
#include <LEDGeneral.h>
#include <LEDRGB.h>
#include <Arduino.h>
*  @param ---
*
*  @return 0 - success
*  @return 1 - error
*/
int LEDRGBsetup()
{
*
*  swichting the red led on / off
*
*  @param On - true: switch on, false: switch off
*
*  @return 0 - success
```

../../Code/Nano33BLESense/LEDs/LEDRGB.cpp

6.5. Tests

6.5.1. Simple Function Test

The simplest test is the flashing of the LEDs at 2 Hz, see sketch **??**.

Listing 6.3.: Simple sketch to test the RGB LED

```
/**
*  @file TestLEDRGB.ino
*
*  @brief Simple program for testing the RGB-LED
*
*
*  Turns the RGB-LED on for one second, then off for one second,
    repeatedly.
*
*  The LED is switched on for 1 second and switched off
*  for 1 second so that the LED flashes accordingly.
*
*  On the Arduino Nano 33 BLE Sense, it is attached to digital pin 22,
    23, 24
*
*/


#incude <../LEDs/RGBLED.h>
#incude <../LEDs/LED.h>

/**
*  @brief the setup function runs once when you press reset or power the
     board
*
*  Standard function of Arduino sketches
*
*  Initialization of the pin  22, pin 23, and 24 as output
```

```
1026 *
     *  @param ---
1028 *
     *  @return void
1030 */
     void setup() {
1032     // Initialize the pin as an output
         RGBLEDinit();
     }
1034


1036

     /**
1038  *  @brief the setup function runs once when you press reset or power the
           board
1040  *
      *  Standard function of Arduino sketches
1042  *
      *  In each loop, the red part is switched on for 1 sec,
      *  then the green part is switched on for 1 sec, and
1044  *  at last the blue part is switched on.
      *
1046  *  @param ---
      *
1048  *  @return void
      */
1050  void loop() {
         // Turn the red LED on
1052     RGBLED_Red(SET_ON);
         // Wait for one second
1054     delay(1000);
         // Turn the LED off
1056     RGBLED_Red(SET_OFF);
         // Turn the green LED on
1058     RGBLED_Green(SET_ON);
         // Wait for one second
1060     delay(1000);
         // Turn the LED off
1062     RGBLED_Green(SET_OFF);
         // Turn the blue LED on
1064     RGBLED_Blue(SET_ON);
         // Wait for one second
1066     delay(1000);
         // Turn the LED off
1068     RGBLED_BLue(SET_OFF);
         // Wait for one second
1070     delay(1000);
     }
```

../../Code/Nano33BLESense/Test/TestLEDRGB.ino

6.5.2. Test all Functions

Different Colors

Colors

A RGB LED is a device that can emit light of different colors by mixing the primary colors of red, green, and blue. The color of the light depends on the relative brightness of each LED, which can be controlled by PWM signals. By varying the brightness of each LED, the RGB LED can produce a wide range of colors, such as yellow, cyan, magenta, white, and more. Some examples of the colors and their corresponding brightness values are:

Red: red = 255, green = 0, blue = 0

Green: red = 0, green = 255, blue = 0

Blue: red = 0, green = 0, blue = 255

Yellow: red = 255, green = 255, blue = 0

Cyan: red = 0, green = 255, blue = 255

Magenta: red = 255, green = 0, blue = 255

White: red = 255, green = 255, blue = 255

Black: red = 0, green = 0, blue = 0

The RGB LED can also create intermediate colors by using different brightness values for each LED. For example, to create

- **orange**, one can use red = 255, green = 127, blue = 0.
- To create **pink**, one can use red = 255, green = 192, blue = 203.
- To create **purple**, one can use red = 128, green = 0, blue = 128.

The sketch 6.4 tests different colors of the LED. The color of the LED changes every 1 second.

Listing 6.4.: value between 0 and 255 to write to the RGB LED

```
1000 /**
     *  @file TestLEDRGBColors.ino
1002 *
     *  @brief Simple program for testing the RGB-LED
1004 *
     *
1006 *  Turns the RGB-LED on for one second, then off for one second,
         repeatedly.
     *
1008 *  The LED is switched on for 1 second and switched off
     *  for 1 second so that the LED flashes accordingly.
1010 *
     *  On the Arduino Nano 33 BLE Sense, it is attached to digital pin 22,
         23, 24
1012 *
     */
1014

1016 #incude <../LEDs/RGBLED.h>
     #incude <../LEDs/LED.h>
1018
     /**
1020 *  @brief the setup function runs once when you press reset or power the
         board
     *
1022 *  Standard function of Arduino sketches
     *
1024 *  Initialization of the pin  22, pin 23, and 24 as output
     *
1026 *  @param ---
     *
1028 *  @return void
     */
1030 void setup() {
       // Set the LED pins as outputs
1032   RGBLEDinit();
     }
```

```
1034
1036
/**
1038 *  @brief the setup function runs once when you press reset or power the
       board
*
1040 *  Standard function of Arduino sketches
*
1042 *  In each loop, different colors are tested
*
1044 *  @param ---
*
1046 *  @return void
*/
1048 void loop() {
  // red
1050   RGBLED_Color(255,0,0);
    // Wait for one second
1052     delay(1000);
  // green
1054   RGBLED_Color(0,255,0);
    // Wait for one second
1056     delay(1000);
  // blue
1058   RGBLED_Color(0,0,255);
    // Wait for one second
1060     delay(1000);
  // yellow
1062   RGBLED_Color(255,255,0);
    // Wait for one second
1064     delay(1000);
  // cyan
1066   RGBLED_Color(0,255,255);
    // Wait for one second
1068     delay(1000);
  // magenta
1070   RGBLED_Color(255,0,255);
    // Wait for one second
1072     delay(1000);
  // white
1074   RGBLED_Color(255,255,255);
    // Wait for one second
1076     delay(1000);
  // black
1078   RGBLED_Color(0,0,0);
    // Wait for one second
1080     delay(1000);
  // orange
1082   RGBLED_Color(255,127,0);
    // Wait for one second
1084     delay(1000);
  // pink
1086   RGBLED_Color(255,192,203);
    // Wait for one second
1088     delay(1000);
  // purple
1090   RGBLED_Color(120,0,128);
    // Wait for one second
1092     delay(1000);
}
```

../../Code/Nano33BLESense/Test/TestLEDRGBColors.ino

Brightness of the RGB-LED

The RGB-LED can also create gradients of colors by changing the brightness values gradually over time. This can create a smooth transition from one color to another, such as from red to green to blue and back to red.
This sketch 6.5 will make the RGB-LED change colors smoothly by varying the brightness of each LED with different speeds. You can adjust the initial brightness values and the increment/decrement values to get different effects.

Listing 6.5.: Different brightness levels for the RGB-LED colors

```
/**
*  @file TestLEDRGBBrightness.ino
*
*  @brief Simple program for testing the RGB-LED
*
*
*  Turns the RGB-LED on for one second, then off for one second,
    repeatedly.
*
*  The LED is switched on for 1 second and switched off
*  for 1 second so that the LED flashes accordingly.
*
*  On the Arduino Nano 33 BLE Sense, it is attached to digital pin 22,
    23, 24
*
*/


#incude <../LEDs/RGBLED.h>
#incude <../LEDs/LED.h>

int redBrightness = 0;   /*< Define the initial brightness values for
    red (0-255)   */
int greenBrightness = 0; /*< Define the initial brightness values for
    green (0-255) */
int blueBrightness = 0;  /*< Define the initial brightness values for
    blue (0-255)  */


int redStep = 5;   /*< Define the increment/decrement value for red   */
int greenStep = 3; /*< Define the increment/decrement value for green */
int blueStep = 7;  /*< Define the increment/decrement value for blue  */


/**
*  @brief the setup function runs once when you press reset or power the
     board
*
*  Standard function of Arduino sketches
*
*  Initialization of the pin  22, pin 23, and 24 as output
*
*  @param ---
*
*  @return void
*/
void setup() {
  // Set the LED pins as outputs
  RGBLEDinit();
}


        /**
*  @brief the setup function runs once when you press reset or power the
     board
*
```

```
*  Standard function of Arduino sketches
*
*  In each loop, the color is changing
*
*  @param ---
*
*  @return void
*/
void loop() {
    // Write the PWM values to the LED pins
    RGBLED_Colors(redBrightness, greenBrightness, blueBrightness);

    // Update the brightness values for each color
    redBrightness += redStep;
    greenBrightness += greenStep;
    blueBrightness += blueStep;

    // Check if the brightness values are out of range and reverse the
    direction
    if (redBrightness <= 0 || redBrightness >= 255) {
       redStep = -redStep;
    }
    if (greenBrightness <= 0 || greenBrightness >= 255) {
        greenStep = -greenStep;
    }
    if (blueBrightness <= 0 || blueBrightness >= 255) {
        blueStep = -blueStep;
    }

    // Wait for 10 milliseconds
    delay(10);
}
```

../../Code/Nano33BLESense/Test/TestLEDRGBBrightness.ino

6.6. Simple Application

In many applications, it is necessary to save power so that the LED is not switched on continuously. Nevertheless, feedback is required as to whether the system, e.g. a fire detector, is switched on and functional. In this case, the LED is switched on for 1 second at a defined time interval, in this case 30 seconds. This is implemented in the example **??**.

Listing 6.6.: A simple watch dog: A simple watchdog: the built-in RGB-LED is switched on for 1 second every 30 seconds.

```
/**
*
*  @file TestLEDRGBApplication.ino
*
*  @brief Simple application using the built-in RGB-LED of the Arduino
    Nano 33 BLE Sense
*
*
*  @details The red part of the RGB-LED is switched on for 1 second and
    switched off for 29 second so that red part of the LED flashes
    accordingly.

*
*/

#include <../LEDs/RGBLED.h>
#include <../LEDs/LED.h>
#include <../LEDs/SignsOfLife.h>
```

```
1016
#define CycleTimeOn 1000 /*< Duty cycle[ms] */
1018
#define CycleTimeOff 29000 /*< Switch-off time [ms] */
1020


1022
/**
1024 *  @brief the setup function runs once when you press reset or power the
       board
*
1026 *  standard function of Arduino sketches
*
1028 *  Initialization of the pin LED_RED, the red part of the builtin RGB-
     LED for the signs of life
*
1030 *  @param ---
*
1032 *  @return void
*/
1034 void setup() {
     // Initialize the function SignsOfLife
1036   SignsOfLifeInit(LED_RED, CycleTimeOn, CycleTimeOff)

1038   // Application
   // ...
1040 }


1042


1044 /**
*  @brief the loop function runs over and over again forever
1046 *
*  standard function of Arduino sketches
1048 *
*  swichting the led on / off for the signs of life
1050 *
*  @param ---
1052 *
*  @return void
1054 */
void loop() {
1056   // Switch red part of the RGB LED on/off
   SignsOfLife();
1058
   // Application
1060   // ...
}
```

../../Code/Nano33BLESense/Test/TestLEDRGBApplication.ino

6.7. Further Readings

- Schanda, Janos: *Colorimetry: Understanding the CIE System*. Wiley, 2007. [Sch07]
- Hiller, Gabrielle: *Color measurement - the CIE color space*. Datacolor, 2019. [Hil22]
- Lukac, Rastislav and Plataniotis, Konstantinos N.: *Color Image Processing: Methods and Applications*. CTC Press, 2018. [LP18]

7. TinyML Shield: Built-in Push Button

7.1. General

A push button is a simple switch mechanism used to control various devices and processes. It is typically made of hard materials like plastic or metal. The surface of a push button is designed to be easily depressed or pushed by the human finger or hand. When you press a push button, it either closes or opens an electrical circuit.
In industrial and commercial applications, push buttons can be linked together so that pressing one button releases another. Emergency stop buttons, often with large mushroom-shaped heads, enhance safety in machines and equipment. Pilot lights are sometimes added to push buttons to draw attention and provide feedback when the button is pressed. Color-coding is common to associate push buttons with their specific functions (e.g., red for stopping, green for starting). [Din]

7.2. Built-in Push Button

The Arduino Nano 33 BLE Sense features an onboard push button. This button is a simple electrical switch that can be activated by pressing it. When you press the button, it completes an electrical circuit. The push button is designed for user interaction and can be used for various purposes.
The built-in button `BUTTON_B` is connected with pin 11. Using the function `pinMode(BUTTON_PIN, INPUT_PULLUP)` the pin is declared as an input. As can be seen in the sketch, pressing the button can be used to trigger actions; typical actions include switching on an LED, changing modes, or initiating sensor readings. Overall, the push button provides a convenient way to interact with the Arduino Nano 33 BLE Sense and create responsive projects. [Ard23a; Ard23b; Arda]

WS:Push Button hier nochmal mit tikz-code hervorheben

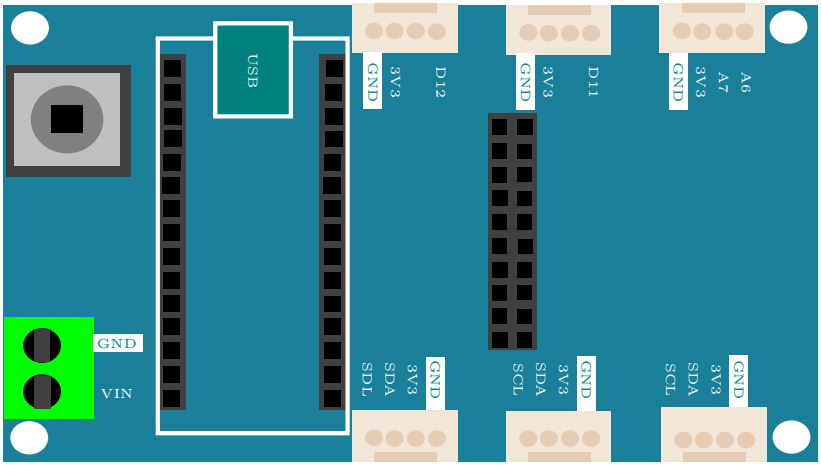


Figure 7.1.: Tiny Machine Learning Shield [Gua21]

7.3. Specification

The built-in button is a small white button and connected to pin 11.

Built-in Button: `BUTTON_B = 11u`

If the pin is declared as input in the function `setup`, then it can be used.
The pin 11 must be defined as an input in the function `setup` by setting `pinMode (11, INPUT_PULLUP)`, otherwise the button cannot be read.

The pin 11 can also be used otherwise. Then the button is not in use. [Ard23a; Ard23b; Arda]

WS:

- cite data sheet
- Circuit Diagram

7.4. Simple Code

As soon as the button is connected, it can be used. It is not necessary to install a special library. Programming takes place in two steps:

1. In the first step, the pin is configured in the function `setup`:

Listing 7.1.: Defining the built-in button's pin as an input.

```
1000        pinMode(BUTTON_B, INPUT_PULLUP)
```

2. In the second step, the button can be used in the function `loop`. To read in the value, use the function `digitalRead`:

Listing 7.2.: Read the built-in button's state

```
1000        buttonState = digitalRead(BUTTON_B);
```

7.5. Tests

The simplest test is the flashing of an LED for 2 seconds, if the button is pressed, see sketch 7.3.

Listing 7.3.: Simple sketch to test the push button and the built-in LED

```
1000 #include <LEDGeneral.h>
     #include <LEDPower.h>
1002 #include <LEDRGB.h>
     #include <LEDSignsOfLife.h>
1004 #include <LEDbuiltin.h>
     #include <Arduino.h>
1006
     /**
1008 *
     *  @file TestPushButton.ino
1010 *
     *  @brief Simple application reading in the built-in push button states.
1012 *
     *
1014 *  @details If the built-in push button is pressed, the the internal
         built-in LED is turn on for 2 seconds
```

```
1016 *
     */
1018
     #define BUTTON_B 11 /*<   Button_B is here defined as pin 11 */
1020 #define BUTTON_PIN  BUTTON_B /*< Use the onboard push button (BUTTON_B)
         */
1022 #define SET_ON  true  /*< Define flag for switching on  */
     #define SET_OFF false /*< Define flag for switching off */
1024
     int buttonState = 0; /*< state of the built-in push button */
1026
     /**
1028 *  @brief the setup function runs once when you press reset or power the
         board
     *
1030 *  standard function of Arduino sketches
     *
1032 *  Initialization of the built-inb LED and the push button
     *
1034 *  @param ---
     *
1036 *  @return void
     */
1038
     void setup() {
1040   // Initialize the pin as an output
       LEDBuiltinsetup();
1042   // Initialize the pin as an input
       pinMode(BUTTON_PIN, INPUT_PULLUP);
1044 }

1046
     /**
1048 *  @brief the loop function runs over and over again forever
     *
1050 *  standard function of Arduino sketches
     *
1052 *  swichting the built-in led on for 2 sec, if the push button is
         pressed
     *
1054 *  @param ---
     *
1056 *  @return void
     */
1058 void loop()
     {
1060   buttonState = digitalRead(BUTTON_PIN);
       if (buttonState == HIGH)
1062   {
         // Turn the LED on
1064     LEDBuiltin(SET_ON);
         // Wait for two second
1066     delay(2000);
         // Turn the LED off
1068     LEDBuiltin(SET_OFF);
         // Wait for one second
1070     delay(1000);
         buttonState = LOW;
1072   }
     }
```

../../Code/Nano33BLESense/PushButton/TestPushButton/TestPushButton.ino

7.6. Interrupt Function on the Arduino Nano 33 BLE Sense

An interrupt is a function that allows the microcontroller on the Arduino Nano 33 BLE Sense to immediately respond to an external event, such as pressing a button, without the need for the program to continuously check for that event. Normally, in a program, the microcontroller would repeatedly check if a button is pressed by running through a loop, which consumes processing power and can slow down other tasks. This is not efficient, especially when the program needs to perform other actions at the same time.
With an interrupt, the program can continue running other tasks, and only when the specified event (like a button press) occurs, the program is temporarily paused to execute a special function known as the Interrupt Service Routine (ISR). The ISR is a small, efficient function designed to handle the event, such as changing a variable or triggering another action. After the ISR is executed, the program returns to where it left off, continuing its normal operation.
Using interrupts in this way helps make the program more efficient and responsive, as it doesn't waste time constantly checking for events and can focus on other tasks until something important happens. This is especially useful for real-time applications where quick responses to external events are necessary, such as in embedded systems, robotics, or sensor monitoring.

7.7. Simple Application

This code sketch 7.4 shows how to use an interrupt to control the built-in LED on the Arduino Nano 33 BLE Sense when the built-in push button is pressed.
The program sets up the built-in LED and push button. When the button is pressed, an interrupt runs a special function called an Interrupt Service Routine. The ISR is very short and only sets a flag variable `pushPressed` to `true`, indicating that the button has been pressed.
The main program `loop` checks this flag. If it is set to `true`, the program turns on the LED for 2 seconds, then turns it off and resets the flag to `false`. This ensures the system is ready to detect the next button press. Using the `true and false` values makes it easy to manage the button's state.
This method avoids constantly checking the button's state, making the program more efficient. Additionally, the Arduino Nano 33 BLE Sense allows all its pins to be used for interrupts. This makes it easy to attach interrupt functions to any pin, allowing quick and efficient responses to inputs like buttons or sensors. [Ardb]

WS:andere Überschrift für simple Application

Listing 7.4.: Simple sketch connects the push button with an interrupt. Here, pushing the built-in button is handled by an interrupt. Then the built-in LED switch on for 2 sec.

```
1000 #include <LEDGeneral.h>
     #include <LEDPower.h>
1002 #include <LEDRGB.h>
     #include <LEDSignsOfLife.h>
1004 #include <LEDbuiltin.h>

1006 #include <LEDGeneral.h>
     #include <LEDPower.h>
1008 #include <LEDRGB.h>
     #include <LEDSignsOfLife.h>
1010 #include <LEDbuiltin.h>

1012 /**
```

```
1014 *
     *  @file TestPushButtonInterrupt.ino
1016 *
     *  @brief Simple application reading in the built-in push button states
          using an interrupt
1018 *
     *
1020 *  @details If the builtin push button is pressed, the built-in LED is
          switched on  for 2 second and switched off again.
     *  But in this example, an interrupt is used.
1022 *
     */

1024 #include <LEDGeneral.h>
     #include <LEDBuiltin.h>
1026


1028
     #define BUTTON_B 11 /*<   Button_B is here defined as pin 11 */
1030 #define BUTTON_PIN  BUTTON_B

1032
     #define SET_ON  true  /*< Define flag for switching on  */
1034 #define SET_OFF false /*< Define flag for switching off */


1036



1038
     // Initialize variables
1040 //
     volatile bool pushPressed = false; /*< // Flag, whether the button is
          pressed. Declare as volatile for interrupt.  safety */
1042
     int ledState = 0; /*<  LED-Status zur Verarbeitng */

1044
     /**
1046 *  @brief the setup function runs once when you press reset or power the
          board
     *
1048 *  standard function of Arduino sketches
     *
1050 *  Initialization of the built-inb LED and the push button
     *
1052 *  @param ---
     *
1054 *  @return void
     */
1056 void setup() {
       // Initialize the pin as an output
1058   LEDBuiltinsetup();
       // Initialize the pin as an input
1060   pinMode(BUTTON_PIN, INPUT_PULLUP);
       // Initialize the interrupt function
1062   attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), buttonPressed,
         CHANGE);
     }
1064


1066 /**
     *  @brief Interrupt service function
1068 *
     *  Attention: as short as possible
1070 *
     *  The function changes just one flag.
1072 */
     void buttonPressed()
1074 {
       if (pushPressed == false)
1076   {
```

```
    pushPressed = true;
  }
}

/**
*  @brief the loop function runs over and over again forever
*
*  standard function of Arduino sketches
*
*  swichting the built-in led on for 2 sec, if the push button is
    pressed
*
*  @param ---
*
*  @return void
*/
void loop()
{
  if (pushPressed)
  {
    // Turn the LED on
    LEDBuiltin(SET_ON);
    // Wait for one second
    delay(2000);
    // Turn the LED off
    LEDBuiltin(SET_OFF);
    pushPressed = false;
  }
  // ...
}
```

../../Code/Nano33BLESense/PushButton/TestPushButtonInterrupt/TestPushButtonInterrupt.ino

This is just a simple example. The variable `BUTTON_B` is already defined, so the assignment is not necessary. The command `delay` should be avoided in an Arduino sketch. Instead, variables of the type `elapsedMillis` should be used.

7.8. Further Readings

- Boxall, John: *Arduino Workshop - A Hands-On Introduction with 65 Projects*. No Starch Press, 2021. [Box21]
- Voš, Andreas: *Volumio mit Drehgebern erweitern*. Make Magazin, 2024. [Voš24]
- Kühnel, Claus: *Arduino - Das umfassende Handbuch*. Rheinwerk Verlag GmbH, 2024. [Küh24]

Part II.

Arduino Nano 33 BLE Sense - External Sensors and Actors

8. Tiny Machine Learning Kit

The Tiny Machine Learning Kit will equip you with all the tools which are needed to start with machine learning. [Ardc]
The kit consists of

- an Arduino Nano 33 BLE Sense Lite,
- a camera module OV7675,
- USB A to USB Micro B cable, and
- a custom Arduino shield to make it easy to attach components.

Figure 8.1.: Das Tiny Machine Learning Kit [Ardc]

8.1. Das Arduino Tiny Machine Learning Shield

Das Tiny Machine Learning Shield wird von Arduino für das Tiny Machine Learning Kit hergestellt, um das Verbinden von Komponenten, wie beispielsweise der Kamera, zu vereinfachen.
Bei dem Tiny Machine Learning Shield handelt es sich um ein Breadboard, was speziell für den Arduino Nano 33 BLE Sense ausgelegt ist. Komponenten können entweder wie die Kamera direkt in passende Slots gesteckt oder mit Grove-Kabeln verbunden werden. Außerdem verfügt das Tiny Machine Learning Shield über eine Anschlussklemme, um externe Spannungsquellen anzuschließen und über einen Taster.

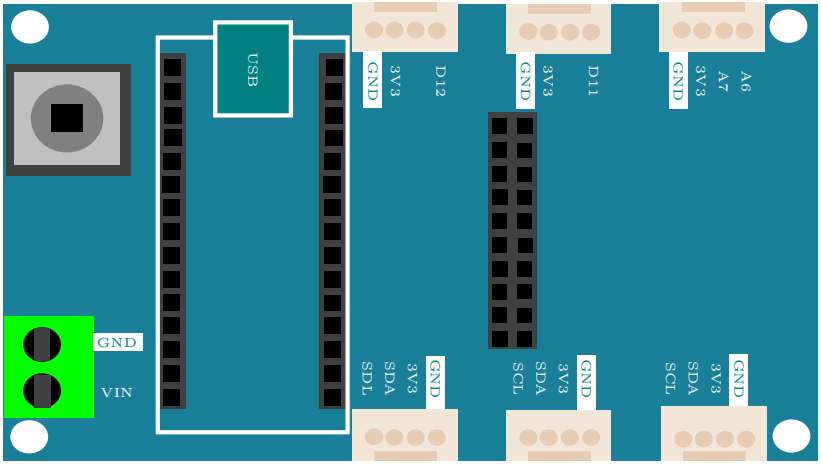


Figure 8.2.: Das Tiny Machine Learning Shield [Gua21]

Die Pinbelegung für die Grove-Schnittstellen sind auf dem Tiny Machine Learning Shield beschriftet. Die Belegung für den 10-poligen Steckplatz für die Kamera ist in der Grafik 8.3 dokumentiert.

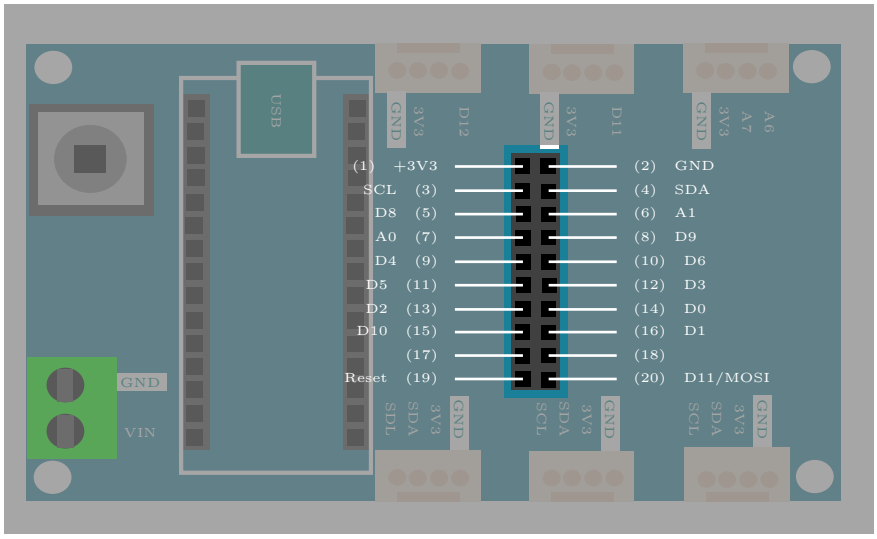


Figure 8.3.: Pin-Belegung des Tiny Machine Learning Shields [Gua21]

8.2. Die OV7675 Kamera

Da die auf dem Arduino fest verbauten Lichtsensoren nur Helligkeit und Farben messen können, ist in dem Kit eine Kamera enthalten. Diese kann verwendet werden, um Objekte zu erkennen oder einen Videostream auszugeben. In unserem Projekt findet die Kamera allerdings keine Verwendung.
Die Kamera kann direkt auf den mittleren Anschluss des Tiny Machine Learning Shields gesteckt werden. Es sind also keine zusätzlichen Kabel zum Verbinden notwendig.

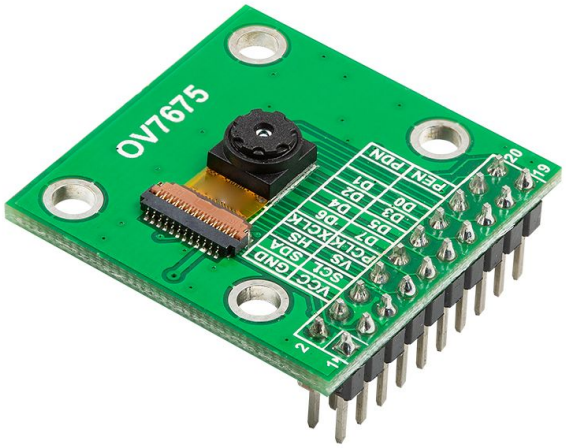


Figure 8.4.: Das OV7675 Kameramodul [Ardc]

8.3. USB-Micro-B- auf USB-A-Kabel

Zur Spannungsversorgung ist in der Box ein USB-Micro-B- auf USB-A-Kabel enthalten. Mit diesem können außerdem die Programme auf den Arduino aufgespielt sowie Daten vom Arduino auf das Programmiergerät übertragen werden.

Figure 8.5.: Ein USB-A auf Micro-USB Kabel

WS:Quelle fehlt

9. Powering the TinyML Shield

Now that you have your Arduino development board up and running, let's talk about how you could deploy it independent of your computer! While some embedded systems call on AC-DC converters (or "wall warts," colloquially) to provide low voltage power to their electronics (the Google home speaker, for example), others are battery powered. Both of these paradigms are applicable to real-world deployment of tinyML and both are achievable using your TinyML kit.

WS:Komplettes Kapite
muss überarbeitet werd
Sortierung, Quellen,
Diagramm,...

9.0.1. USB Power Delivery

To this point, we have provided power over USB to our microcontroller via the microB port on the Nano 33 BLE Sense. The 5V that USB carries is then down regulated on the development board to 3.3V, the logical reference for the MCU. While there's nothing wrong with this in development, a prototype of your application ought not depend on drawing power from your computer. Instead, you could call on an AC-DC converter with USB output. This has the added benefit of likely raising the current capacity of the 5V power rail, from at most 500 mA via a computer to whatever the specification happens to be for a given wall-bound converter. This could be meaningful for driving certain power hungry actuators, like speakers.

9.1. Battery

While the above solution removes the need for the computer, you're still tethered to a wall. To go fully mobile you'll want to call on a battery. So what are our options? The idea of using a voltage regulator (specifically, the MPM3610) to cut down an input voltage to a nice, stable 3.3V level applies here as well. If you were to take a closer look at the linked datasheet, you'd find that the MPM3610 accepts input voltages from 4.5V to 21V. The 5V delivered over USB is within this range, and any compatible battery will need to be as well. This unfortunately eliminates the possibility of calling on single cell 3.7V lithium batteries, but makes the selection of a 9V alkaline battery fairly obvious.
You might be wondering how any battery might connect to the boards in front of you, but never fear, we've got you covered. At one corner of the Tiny Machine Learning Shield you'll find a green terminal block with silkscreen labels that read, "VIN" and "GND," where GND is our reference voltage and as such should be connected to the negative terminal of any compatible battery. This green terminal block is where you'd want to screw in wires carrying 4.5V to 21V, and we'll add that 9V clip, like this, that terminates in pre-stripped hook-up wire makes this quite easy!

Assembly Steps

1. Screw down a wire leading from the negative battery terminal (black) to GND (Most < 3mm flat-head screwdrivers will suffice here).
2. Repeat this process for the positive battery terminal (red) to VIN. And that's it you're all set to power your Arduino from a battery!

Important Notes

1. While there is clever circuitry on board to handle such an exception, it is generally good practice to avoid having competing power sources, so we'd recommend that you unplug the Nano 33 BLE Sense from USB power before connecting a battery

2. With about 550 mAh capacity, a 9V battery can source 15 mA for about 37 hours before you will need to get a new one.

VS:tikz-Bild des Shields fehlt.

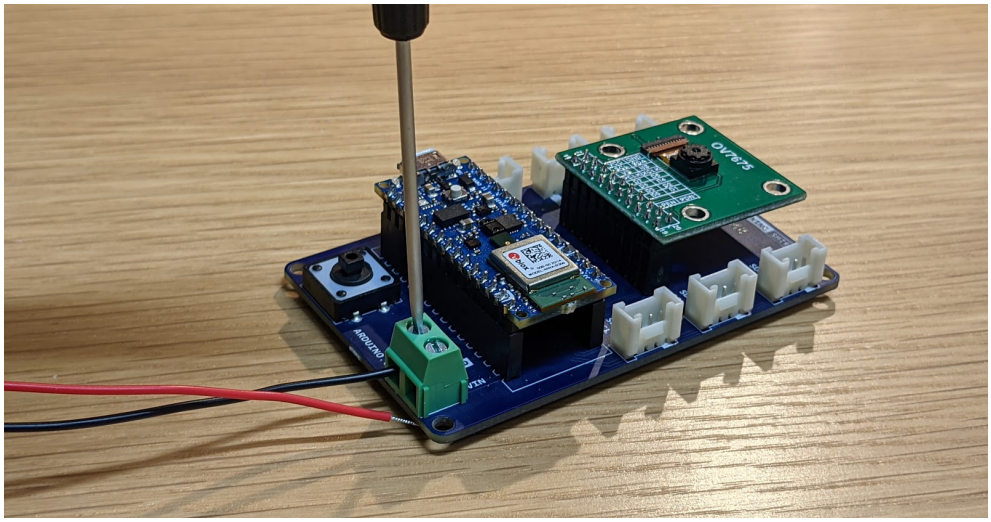


Figure 9.1.: Montage des Clips

An image of screwing down the black negative terminal to attach a wire.

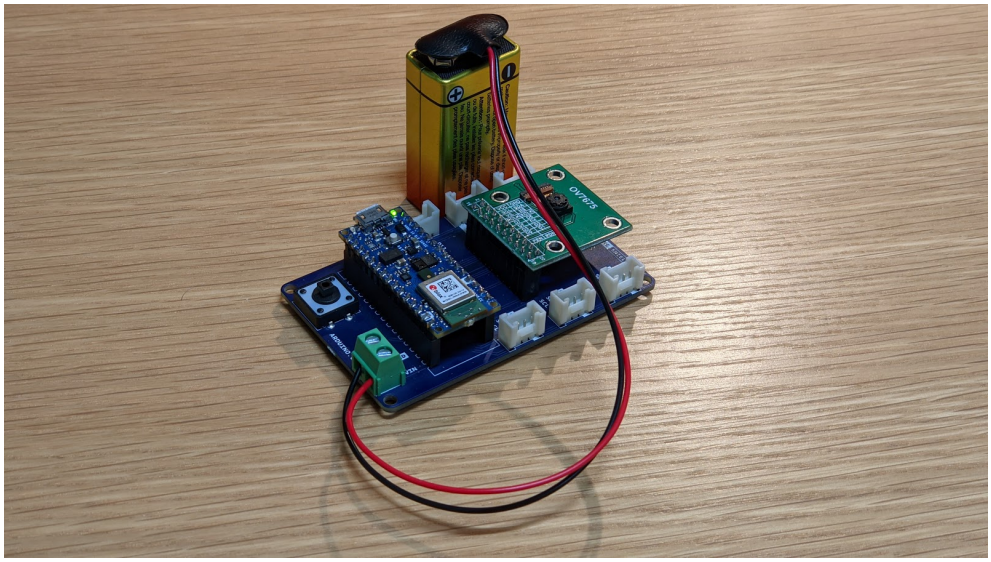


Figure 9.2.: Komplettes System mit Batterie

An image of the attached battery powering the device.

9.2. Checking the Battery Voltage

We use an analog input pin to read the voltage. As we are running from a 3.7V volt battery, we need to adjust the reference voltage used by the pin as otherwise it would

be comparing the voltage to itself. The statement `analogReference(INTERNAL)` sets the pin to compare the input voltage to a regulated 1.1V. We therefore need to reduce the voltage on the input pin to less than 1.1V for this to work. This is done by dividing the voltage using 2 resistors, 1m and 330k ohms. This divides the voltage by approximately 4 so when the battery is fully charged, which is 4.2V, the voltage at the pin input is 4.2/4 = 1.05V.

```
// Battery Monitor
#define MONITOR_PIN  A0                  // Pin used to monitor supply voltage
const float voltageDivider  = 4.0;       // Used to calculate the actual voltage fRom
                                         // Using 1m and 330k ohm resistors divids th
voltage by approx 4
                                         // You may wany to substitute
actual values of resistors in an equation (R1 + R2)/R2
                                         // E.g. (1000 + 330)/330 = 4.03
                                         // Alternatively  take the voltage reading a
                                         //  the 2 resistors to ground and divide one

// Read the monitor pin and calculate the voltage
float BatteryVoltage()
{
    float reading = analogRead(MONITOR_PIN);
    // Calculate voltage - reference voltage is 1.1v
    return 1.1 * (reading/1023) * voltageDivider;
}
```

The function `BatterVoltage()`, reads the analog pin, which will range from 0 for 0V to 1,023 for 1.1V and using this reading calculates the actual voltage coming form the battery.
The function `DrawScreenSave()` function calls this then selects the appropriate bitmap to display based on the following:

- If voltage is greater then 3.6V - full
- Voltage between 3.5 and 3.6V - 3/4
- Voltage between 3.4 and 3.5V - half
- Voltage between 3.3 and 3.4V - 1/4
- Voltage < 3.3V - empty

9.3. Batterieclip

Der Batterieclip in Abb. 9.3, der vom Hersteller *reichelt* ist, kann vertikal an einen 9-Volt-Block angeschlossen werden. Die dazugehörigen Anschlussdrähte haben eine Länge von 150 mm. Der Anschlussclip ist in der I-Form ausgeführt, weshalb er sich platzsparend ins Gehäuse einbinden lässt [Rei].

9.4. Spannungssensor

Der Spannungssensor in Abb. **??** von dem Hersteller *Shenzhen Global Technology Co., Ltd* kann bei der Versorgungsspannung von 3,3 V Spannungen in dem Bereich von 0 V bis 16,5 V messen. Dieser wird genutzt, um den Ladestand der Batterie zu überwachen. Die analoge Auflösung des Sensors liegt bei 10 Bit. Damit kann bei dem angegebenen Messbereich die Spannung mit der Auflösung von 0,016 V gemessen werden. Zur

Figure 9.3.: Batterieclip für 9-Volt-Block[Rei24b]

Eingangsschnittstelle gehört der Voltage at the Common Collector (VCC)-Anschluss und der Ground (GND)-Anschluss [She]. Die Bauteilmaße betragen 13 mm x 27 mm.

Figure 9.4.: Voltage Sensor 170640 von dem Hersteller *Shenzhen Global Technology Co., Ltd*[She]

Mit dem Sketch `TestBattery.ino`, siehe **??**TestBattery, soll der Spannungssensor getestet werden.

9.4.1. Durchführung

Für den Test werden die folgenden Hardware-Komponenten benötigt:

- Arduino Nano 33 BLE Sense Lite
- Tiny Machine Learning Shield
- USB-A auf USB-Mikro Verbindungskabel
- Grove Jumper zu Grove 4 Pin Kabel
- Spannungssensor
- Batterie
- Batterieclip

Listing 9.1.: Beispiel-Sketch zum Testen der Batterie

```
1000 /**
     * @file TestBattery.ino
1002 * @author Wings
     * @date 20.05.2024
1004 * @details The sketch is used to test the voltage sensor. For the test,
       the voltage is measured on a 9 V block battery and output on the
       serial monitor.
     *
1006 **/

1008 /** Pin A7 is referenced as the voltage pin for the voltage sensor. */
     #define VoltagePin A7
1010
     /** The parameter SignalEingang is later assigned the value read in from
         the VoltagePin pin. */
1012 long SignalEingang;

1014 /** The  parameter Spannung stands for the voltage that is measured with
         the voltage sensor and has the unit volt. */
     double Voltage;
1016
     /**
1018 * @brief Starting serial communication
     *
1020 * @details The function is executed when the programme is started. The
       serial interface to the computer is initialised at 9600 baud
     * There is no return value.
1022 */
     void setup() {
1024   Serial.begin(9600);
     }
1026
     /**
1028 * @brief Display of the measured voltage
     *
1030 * @details The function is executed continuously. The signal at the pin
       VoltagePin is read out and a voltage value is converted.
     * The value read in at the voltage pin is between 0 and 1023. The
       voltage measurement range is between 0 and 16.5 V. The read-in value
        can be converted into a voltage using the function map().
1032 * The voltage is displayed in volts on the serial monitor. There is a 5-
       second wait after each cycle to avoid flooding the serial monitor.
       There is no return value.
     */
1034 void loop() {
       SignalEingang = analogRead(VoltagePin);
1036   Voltage       = map(SignalEingang, 0, 1023, 0, 165); // Converting the
         input signal 165 for 16.5 V
       Voltage       = Voltage/10; //  Converting the input signal 165 for
         16.5 V
1038   Serial.print(Voltage);
       Serial.println(" V");
1040   delay(5000);
     }
```

../../Code/Arduino/Battery/TestBattery.ino

Die Hardware-Komponenten werden zusammengebaut, aber die Batterie wird noch nicht angeschlossen. Dann wird der Arduino Nano 33 BLE Sense Lite mit einem Computer verbunden. Anschließend wird der Sketch `TestBattery.ino` auf den Arduino Nano 33 BLE Sense Lite geladen und der serielle Monitor in der Arduino IDE geöffnet. Die Batterie wird während des Tests an den Batterieclip angeschlossen und der gemessene Wert bei dem seriellen Monitor ausgelesen.

9.4.2. Ergebnisse

Zu Beginn des Tests zeigt der serielle Monitor die Spannung 0 V an. Nach dem Anschließen der Batterie an den Spannungssensor wird die Spannung 9,6 V angezeigt. Abbildung 9.5 zeigt den gemessenen Spannungsverlauf. Die angezeigte Spannung liegt in dem erwarteten Bereich einer 9 V Batterie.

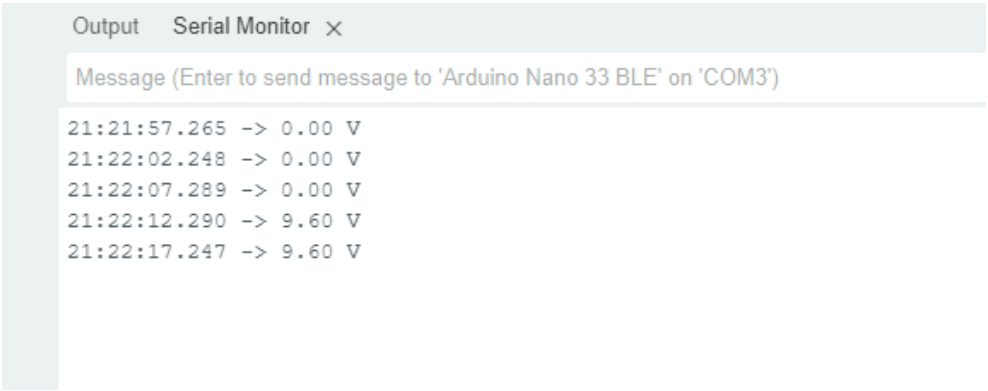


Figure 9.5.: Testoutput des Spannungssensors

10. Connectors of Type JST

10.1. Connector Series

JST connectors are electrical connectors manufactured to the design standards originally developed by J.S.T. Mfg. Co. (Japan Solderless Terminal).[Jsta] JST manufactures numerous series (families) and pitches (pin-to-pin distance) of connectors.

JST manufactures a large number of series (families) of connectors. The PCB (wire-to-board) connectors are available in top (vertical) or side (horizontal) entry, and through-hole or surface-mount. [Jstb]

This description refers exclusively to the mechanical structure and electrical behaviour. The pin assignment is not standardised.

Attention: The term "JST" is sometimes incorrectly used as a vernacular term meaning any small white electrical connector mounted on Printed Circuit Board (PCB)s.

10.2. JST VH

The technical data of the JST VH series is given in this section. They are taken from the data sheet for the series. [Jstd]

10.2.1. Product Profile

Series	VH connector
Category	Crimp Style Connectors (Wire-to-Board type)
Type	Crimp style, Compact type, With locking device Disconnectable type
Feature	With secure locking device

10.2.2. Specification

Pitch	3.96mm
Pin rows	1
Current rating	10A (AWG#16)
Voltage rating	250V
PC board mounting direction	Top entry, Side entry

Figure 10.1.: JST connector type VH https://www.jst-mfg.com/product/index.php?series=262

10.3. JST PH

The technical data of the JST PH series is given in this section. They are taken from the data sheet for the series. [Jstc]

10.3.1. Product Profile

Series	PH connector
Category	Crimp Style Connectors (Wire-to-Board type)
Type	Crimp style, Thin type
	Disconnectable type

10.3.2. Specification

Pitch	2.0mm
Current rating	2A (AWG#24)
Voltage rating	100V
PC board mounting direction	Top entry, Side entry

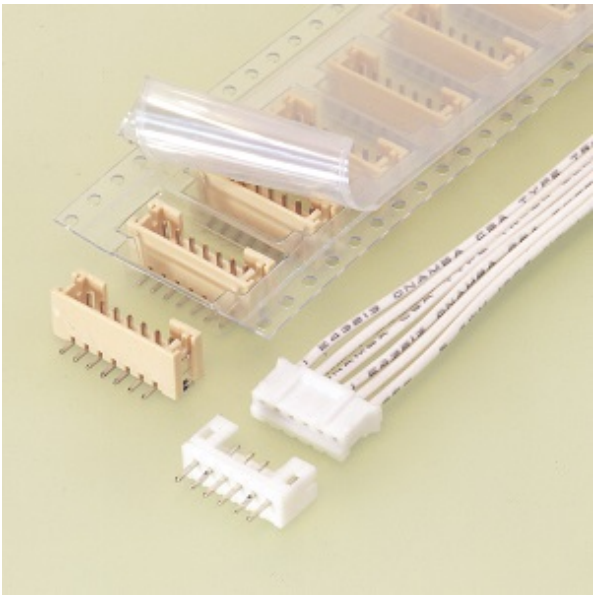


Figure 10.2.: JST connector type VH [Jstc]

10.3.3. JST PHR-2

One variant of the connector series is the JST PHR-2 type, which has 2 pins. Figure 10.3 shows a JST PHR-2 connector.

Figure 10.3.: JST connector type PHR-2 [Jstc]

11. Grove

Grove, ein Produkt von Seeed Studio, ist eine Hardware-Schnittstelle für Technikinteressierte. Es vereinfacht die Kommunikation zwischen Mikrocontrollern und Sensoren und reduziert das Löten. Diese Schnittstelle wurde im Jahr 2010 von Seeed Studio entwickelt, um die Zusammenarbeit zwischen verschiedenen Technologien zu vereinfachen und die Entwicklung neuer Projekte zu erleichtern.[See12; See13]

Mit der Verfügbarkeit eines Steckverbinder-Typs bietet diese Schnittstelle eine Vielzahl von Möglichkeiten für die Integration von Sensoren, Aktuatoren und Mikrocontrollern. Diese einfachen Kommunikationsverbindungen ermöglichen es den Nutzern, ihre Projekte ohne großen Zeitaufwand zu realisieren und zu testen. [See16] Mit jedem neuen Sensor, der mit der Grove-Familie auf dem Markt erscheint, wird die Flexibilität und kreative Möglichkeit für die Entwicklung von Projekten weiter gesteigert.

Die meisten Mikrocontroller verfügen nicht unmittelbar über die Grove-Schnittstelle. Damit Komponenten, die die Grove-Schnittstelle haben, werden häufiger Shields eingesetzt, die diese dann zur Verfügung stellt. Zum Beispiel verfügt der Arduino Nano 33 BLE Sense Lite über keine Grove-Anschlüsse. Das Tiny Machine Learning Shield ermöglicht die Verwendung von Grove-Komponenten für den Arduino Nano 33 BLE Sense, indem es als Schnittstelle zwischen dem Mikrocontroller und den Grove-Modulen fungiert. Der Arduino Nano 33 BLE Sense wird direkt auf das Shield gesteckt, welches über standardisierte Grove-Anschlüsse verfügt. Auf der Abbildung 11.1 ist dargestellt, wie der Mikrocontroller Arduino Nano 33 BLE Sense mit dem Shield verbunden ist.

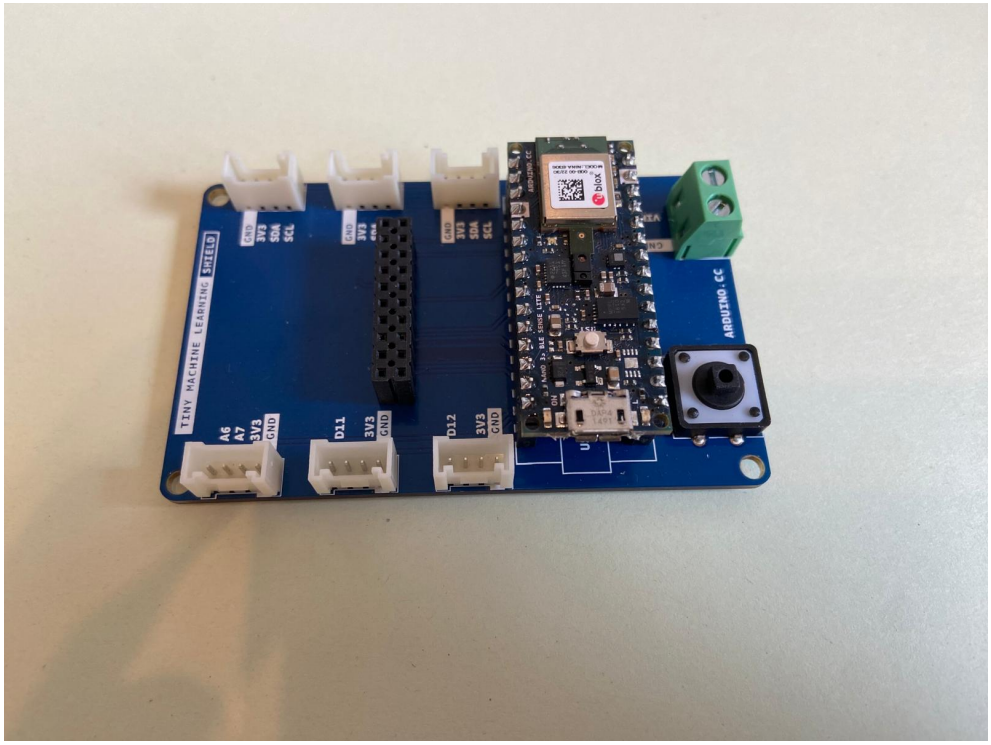


Figure 11.1.: Mikrocontroller gesteckt auf dem Shield

11.1. Grove-Universal-Kabel

Üblicherweise werden Sensoren und Mikrocontroller über ein Grove-Universal-Kabel verbunden. In der Abbildung 11.2, vom Hersteller *seeed studio* ermöglicht die Verbindung mit allen Modulen des Grove-Systems. Für ein Grove-System wird somit nur eine Kabelart benötigt. Die Schnittstellen des Kabels sind beidseitig Japan Solderless Terminal (JST)-VH-Stecker. In der Abbildung 11.2 beträgt beispielsweise die Kabellänge 50 mm [See16].

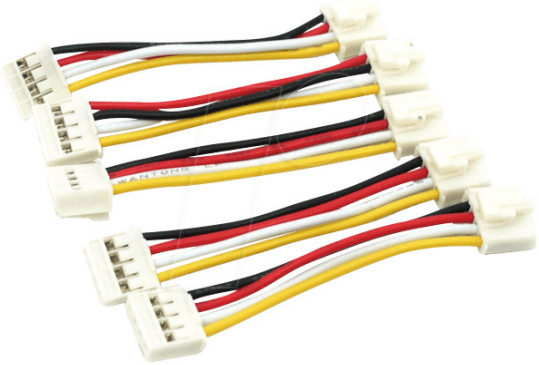


Figure 11.2.: Grove-Universal-Kabel[Rei24a]

11.2. Verbindung Grove-Schnittstelle zu 4-Pin-Lösungen

Da nicht alle Sensoren und Mikrocontroller über eine Grove-Schnittstelle verfügen, existieren Kabel, die einseitig die Grove-Schnittstelle haben. In der Abbildung 11.3 ist das Kabel dargestellt, das auf einer Seite die Grove-Schnittstelle zur Verfügng stellt und auf der anderen Seite 4 Pins. In diesem Beispiel hat das Kabel eine Länge von 30 cm [Rei24c].

Figure 11.3.: Grove Jumper zu Grove 4 Pin-Kabel[Rei24c]

11.3. Pinbelegung

Die Pinbelegung ist von seeed studio standardisiert. In der Tabelle 11.1 ist die Belegung erläutert. [See16]

Table 11.1.: Pinbelegung der Grove-Schnittstelle

Farbe	Pin	Funktion
Gelb	1	frei belegbar, D0 oder A0
Weiß	2	frei belegbar, D1 oder A1
Rot	3	VCC, 3,3V oder 5V
Schwarz	4	GND

Grove stellt über Pin 3 und 4 die Stromversorgung sicher. Der gelbe und der weiße Pin ist in Abhängigkeit vom Sensor belegt. Falls der Sensor nur einen Pin benötigt, so wird Pin 1 verwendet. Als Schnittstelle zum Mikrocontroller muss die Belegung für jede Anwendung definiert werden.
Für Protokolle, wie zum Beispiel I^2C und Serial Peripheral Interface (SPI), sind die Belegung eindeutig festgelegt. Die Tabelle11.2 listet die Pinbelegung der Grove-Schnittstelle für I^2C-Grove-Stecker auf.

Table 11.2.: Pinbelegung der Grove-Schnittstelle für I^2C-Grove-Stecker

Farbe	Pin	Funktion
Gelb	1	frei belegbar, zum Beispiel SCL auf I^2C-Grove-Steckern
Weiß	2	frei belegbar, zum Beispiel SDA auf I^2C-Grove-Steckern
Rot	3	VCC, 3,3V oder 5V
Schwarz	4	GND

Die Tabelle 11.3 enthält die Pinbelegung für die serielle Schnittstelle UART.

Table 11.3.: Pinbelegung der Grove-Schnittstelle für UART

Farbe	Pin	Funktion
Gelb	1	RX
Weiß	2	TX
Rot	3	VCC, 3,3V oder 5V
Schwarz	4	GND

12. BlueTooth

- https://www.instructables.com/Arduino-NANO-33-Made-Easy-BLE-Sense-and-IoT/
- https://www.hackster.io/sridhar-rajagopal/control-arduino-nano-ble-with-blue
- https://docs.arduino.cc/tutorials/nano-33-ble-sense/ble-device-to-device
- https://projecthub.arduino.cc/8bitkick/sensor-data-streaming-with-arduino-a6
- https://www.okdo.com/getting-started/get-started-with-arduino-nano-33-sense/
- https://rootsaid.com/arduino-ble-example/
- https://learn.sparkfun.com/tutorials/bluetooth-basics
- https://ladvien.com/arduino-nano-33-bluetooth-low-energy-setup/

12.1. Quick Start

This tutorial shows you how to use the free pfodDesignerV3 V3.0.3774+ Android app to create a general purpose Bluetooth Low Energy (BLE) and WiFi connection for Arduino NANO 33 boards without doing any programming. There are three (3) Arduino NANO 33 boards, the NANO 33 BLE and NANO 33 BLE Sense, which connect by BLE only and the NANO 33 IoT which can connect via BLE or WiFi.
Connecting using pfodApp is the most flexible way to connect via BLE (or WiFi). See
Using pfodApp to connect to the NANO 33 BLE (Step 4) and
Using pfodApp to connect to the NANO 33 IoT via BLE (Step 7) and
Using pfodApp to connect to the NANO 33 IoT via WiFi (Step 9) below.
owever simple sketches are also provided to send user defined command words via Telnet, for WiFi, or via the free Nordic nRF UART 2.0 for BLE. See
Using the Nordic nRF UART 2.0 app to connect to NANO 33 BLE (Step 5) and
Using the Nordic nRF UART 2.0 app to connect to the NANO 33 IoT via BLE (Step 8) and
Using a Telnet terminal program to connect to the NANO 33 IoT via WiFi (Step 10) below
This tutorial is also available on-line at Arduino NANO 33 Made Easy.

12.2. Supplies

Arduino NANO 33 – either BLE or Sense or IoT
optionally pfodDesignerV3 and pfodApp

12.3. Step 1: Introduction

There are a number of problems with BLE. See this page for BLE problems and solutions and there are some BLE trouble shooting tips. The learning curve is steep and the specification has hundreds of specialise connect services each of which requires its own mobile application to connect to. This tutorial shows you how to generate

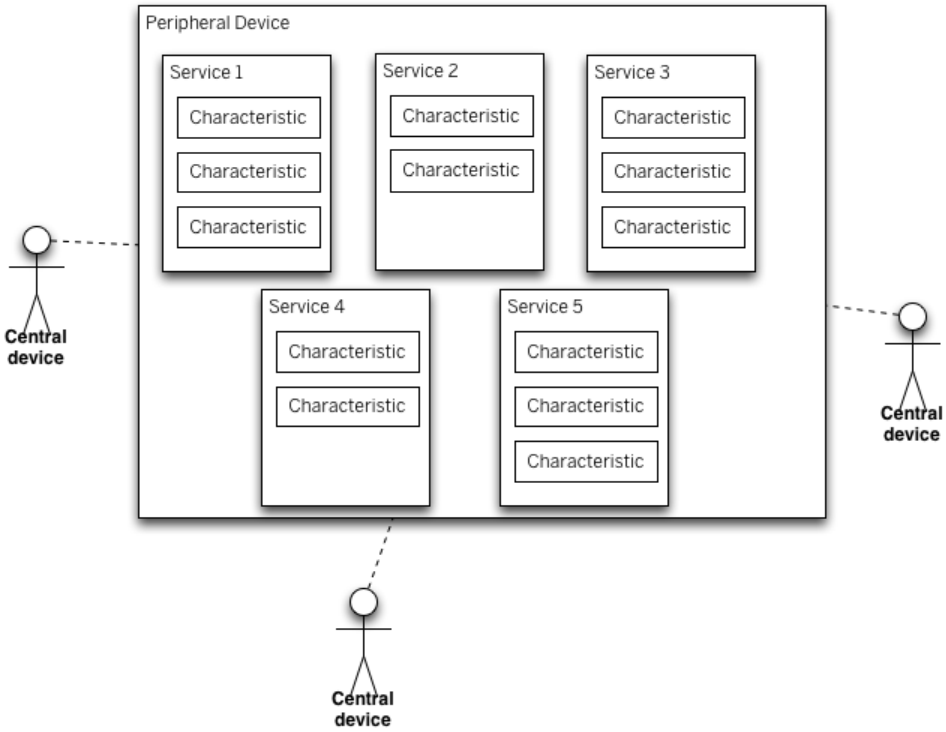


Figure 12.1.: BLE Devices – Peripheral and Central Devices

Arduino code for a general purpose Nordic UART BLE connection over which you can send and receive a stream of commands and data to a general purpose BLE UART mobile application. The free pfodDesignerV3 Android application is used to generate the Arduino code. The output is designed to connect to the paid pfodApp Android application which can display menus, send command, log data and show charts. No Android programming is required. The Arduino code has complete control over what is displayed by pfodApp.
For the NANO 33 IoT you can also connect via WiFi. Again the pfodDesignerV3 generates all the Arduino code and by default is designed to connect to pfodApp with optional 128bit security.
However you do not need to use pfodApp, you can connect to the generated code using the free Nordic nRF UART 2.0 or a Telnet programs (for the WiFi connection). Sketches are included which provide command words to control the boards.
The free pfodDesigner V3.0.3774+ will generate Arduino code for a wide range of boards and connection types including Serial connections, Bluetooth Low Energy (BLE), WiFi, SMS, Radio/LoRa, Bluetooth Classic and Ethernet. For examples Arduino code for of a wide range of BLE boards see Bluetooth Low Energy (BLE) made simple with pfodApp. Here we will be using a BLE connection for NANO 33 BLE, Sense and IoT and a WiFi connection for the IoT.
Each of the NANO 33 boards has extra sensor components that differ between the three (3) boards. The pfodDesignerV3 generates code to read/write the digital outputs and perform analogReads and analogWrites. In the examples below we will turn the board LED on and off and read the voltage at A0 and log and plot it. Once that sketch is running you can add each board's specialised sensor libraries and sent their data in place of the analogRead(A0).
pfodDesigner generates simple menus and charts, however you can also program custom graphical interfaces in your Arduino sketch. Above is an example of slider control

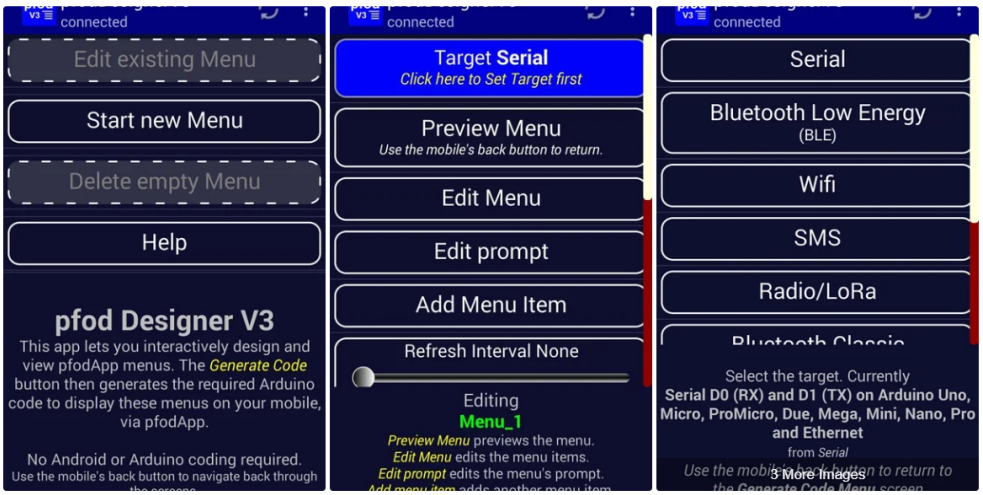


Figure 12.2.: Step 2: Creating the Custom Android Menus and Generating the Code

adjusting a guage. All the code for this control is in your Arduino sketch. No Android programming necessary. See Custom Arduino Controls for more examples.

12.4. How pfodApp is optimised for short BLE style messages

Bluetooth Low Energy (BLE) or Bluetooth V4 is a completely different version of Bluetooth. BLE has been optimised for very low power consumption. pfodApp is a general purpose Android app whose screens, menus, buttons, sliders and plots are completely defined by the device you connect to.
BLE only sends 20 bytes in each message. Fortunately the pfod Specification was designed around very small messages. Almost all of pfod's command are less then 20 bytes. The usual exception is the initial main menu message which specifies what text, menus, buttons, etc. pfodApp should display to the user, but the size of this message is completely controlled by you and you can use sub-menus to reduce the size of the main menu.
The pfod specification also has a number of features to reduce the message size. While the BLE device must respond to every command the pfodApp sends, the response can be as simple as (an empty response). If you need to update the menu the user is viewing in response to a command or due to a re-request, you need only send back the changes in the existing menu, rather then resending the entire menu. These features keep the almost all messages to less than 20 bytes. pfodApp caches menus across re-connections so that the whole menu only needs to be sent once. Thereafter short menu updates can be sent.

12.5. Step 2: Creating the Custom Android Menus and Generating the Code

Before looking at each of these BLE modules, pfodDesignerV3 will first be used to create a custom menu to turn a Led on and off and plot the voltage read at A0. pfodDesignerV3 can then generate code tailored to the particular hardware you select. You can skip over this step and come back to it later if you like. The section on each module, below, includes the completed code sketch for this example menu generated

for that module and also includes sketches that do not need pfodApp

The free pfodDesignerV3 is used to create the menu and show you an accurate preview of how the menu will look on your mobile. The pfodDesignerV3 allows you to create menus and sub-menus with buttons and sliders optionally connected to I/O pins and generate the sketch code for you (see the pfodDesigner example tutorials) but the pfodDesignerV3 does not cover all the features pfodApp supports. See the pfodSpecification.pdf for a complete list including data logging and plotting, multi- and single- selections screens, sliders, text input, etc.

Create the Custom menu to turn the Arduino LED on and off and Plot A0 Start a new menu and select as a target Bluetooth Low Energy (BLE) and then select NANO 33 BLE (and Sense). pfodDesignerV3.0.3770+ has support for NANO 33 boards.

Then follow the tutorial Design a Custom menu to turn the Arduino Led on and off for step by step instructions for creating a LED on/off menu using pfodDesignerV3.

If you don't like the colours of font sizes or the text, you can easily edit them in pfodDesignerV3 to whatever you want and see a WYSIWYG (What You See Is What You Get) display of the designed menu.

Now we will add a Chart button to display the A0 reading. The steps to does this in pfodDesignerV3 are shown in Adding a Chart and Logging Data The AtoD range is 0 to 1023 for 0 to 3.3V

Generating the code from pfodDesignerV3 gives the this sketch `Nano33BLE_Led_-A0.ino`

12.6. BLE Mobile App "Nordic Semiconductors nRF Connect"

- On your mobile, install the App "Nordic Semiconductors nRF Connect". There are Android and IOS versions.

- In the Arduino IDE open Serial Monitor by clicking on the magnifying glass icon on the top right.

The Nano will start sending BLE advertising packets and wait for a Central (Mobile Phone) to connect.

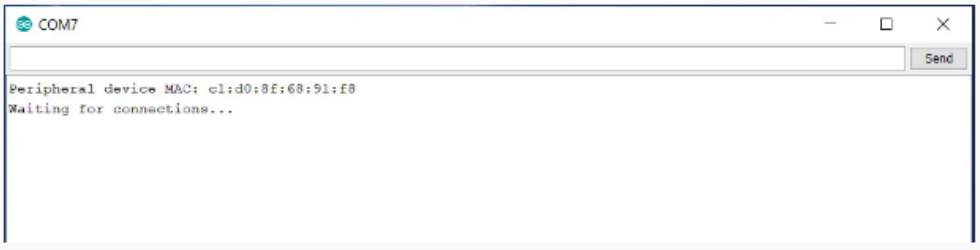


Figure 12.3.: App nRF is searching the Arduino Nano BLE Sense

In the App nRF on your mobile select **Scan** and you should see **Nano33BLE** as a device you can connect to.

Attention: BLE Peripherals can only connect to one device at a time – if the Nano33BLE is already connected it could be to another device nearby.

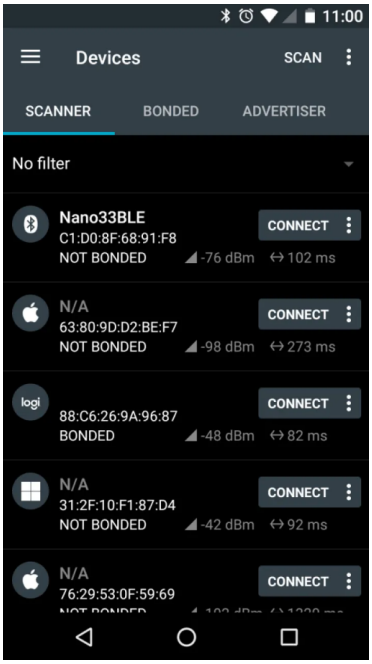


Figure 12.4.: App nRF is searching the Arduino Nano BLE Sense

Connect to the Nano and select the Unknown Service.

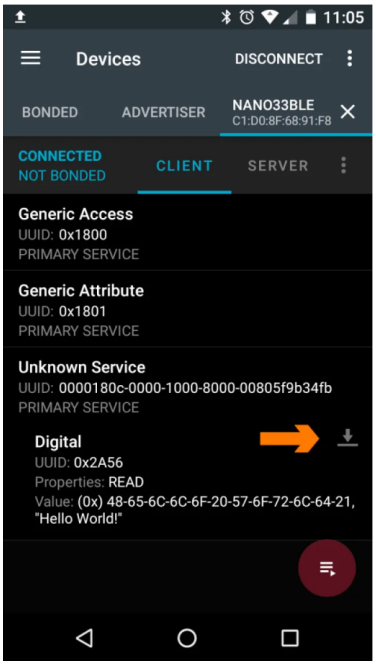


Figure 12.5.: App nRF is getting data the Arduino Nano BLE Sense

Touch the **down arrow** next to the characteristic **Digital** and it will read the message "Hello World!" being broadcast by the Nano33BLE.

12.7. Arduino BLE Example – Explained Step by Step

12.7.1. Arduino BLE Example Code Explained

In this tutorial series, I will give you a basic idea you need to know about Bluetooth Low Energy and I will show you how you can make Arduino BLE Chipset to send and receive data wirelessly from mobile phones and other Arduino boards. Let's Get Started.

Arduino Nano 33 BLE Sense, Nano with BLE connectivity focussing on IOT, which is packed with a wide variety of sensors such as 9 axis Inertial Measurement Unit, pressure, light, and even gestures sensors and a microphone.

It is powered by Nina B306 module that supports BLE as well as Bluetooth 5 connection. The inbuilt Bluetooth module consumes very low power and can be easily accessed using Arduino libraries. This makes it easier to program and enable wireless connectivity to any of your projects in no time. You won't have to use external Bluetooth modules to add Bluetooth capability to your project. Save space and power.

12.7.2. Arduino BLE – Bluetooth Low Energy Introduction

BLE is a version of Bluetooth which is optimized for very low power consuming situations with very low data rate. We can even operate these devices using a coin cell for weeks or even months.

WS:cite Arduino have a wonderful introduction to BLE but here in this post, I will give you a brief introduction for you to get started with BLE communication.

Basically, there are two types of devices when we consider a BLE communication block.

- The Peripheral Device
- The Central Device

Peripheral Device is like a Notice board, from where we can read data from various notices or pin new notices to the board. It posts data for all devices that needs this information.

Central Devices are like people who are reading notices from the notice board. Multiple users can read and get data from the notice board at the same time. Similarly multiple central devices can read data from the peripheral device at the same time.

The information that is given by the Peripheral devices are structured as Services. And These services are further divided into characteristics. Think of Services as different notices in the notice board and services as different paragraphs in each notice board.

If Accelerometer is a service, then their values X, Y and Z can be three characteristics.

Now let's take a look at a simple Arduino BLE example.

12.7.3. Arduino BLE Example 1 – Battery Level Indicator

In this example, I will explain how you can read the level of a battery connected to pin A0 of an Arduino using a smartphone via BLE. This is the code here. This is pretty much the same as that of the example code for Battery Monitor with minor changes. I will explain it for you.

First you have to install the library ArduinoBLE from the library manager.

Just go to `Sketch -> Include Library -> Manage Library` and Search for `ArduinoBLE` and simply install it.

```
#include <ArduinoBLE.h>
BLEService batteryService("1101");
BLEUnsignedCharCharacteristic batteryLevelChar("2101", BLERead | BLENotify);

void setup() {
    Serial.begin(9600);
    while (!Serial);

    pinMode(LED_BUILTIN, OUTPUT);
    if (!BLE.begin())
    {
        Serial.println("starting BLE failed!");
        while (1);
    }

    BLE.setLocalName("BatteryMonitor");
    BLE.setAdvertisedService(batteryService);
    batteryService.addCharacteristic(batteryLevelChar);
    BLE.addService(batteryService);

    BLE.advertise();
    Serial.println("Bluetooth device active, waiting for connections...");
}

void loop()
{
    BLEDevice central = BLE.central();

    if (central)
    {
        Serial.print("Connected to central: ");
        Serial.println(central.address());
        digitalWrite(LED_BUILTIN, HIGH);

        while (central.connected()) {

            int battery = analogRead(A0);
            int batteryLevel = map(battery, 0, 1023, 0, 100);
            Serial.print("Battery Level % is now: ");
            Serial.println(batteryLevel);
            batteryLevelChar.writeValue(batteryLevel);
            delay(200);

        }
    }
    digitalWrite(LED_BUILTIN, LOW);
    Serial.print("Disconnected from central: ");
    Serial.println(central.address());
}
```

Listing 12.1.: Arduino BLE Tutorial Battery Level Indicator Code

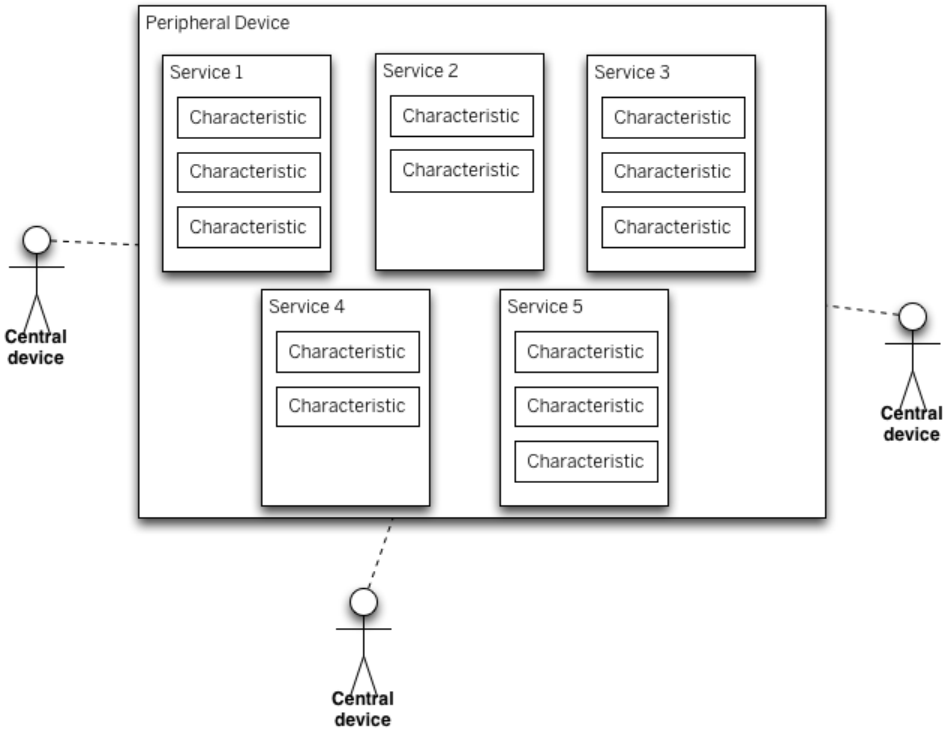


Figure 12.6.: BLE Devices – Peripheral and Central Devices

12.7.4. Arduino BLE Tutorial Battery Level Indicator Code

12.7.5. Arduino Bluetooth Battery Level Indicator Code Explained

```
#include <ArduinoBLE.h>
BLEService batteryService("1101");
BLEUnsignedCharCharacteristic batteryLevelChar("2101", BLERead | BLENotify);
```

The first line of the code is to include the file `ArduinoBLE.h`. Then we will declare the Battery Service as well the battery level characteristics here. Here we will be giving two permissions – `BLERead` and `BLENotify`.
`BLERead` will allow central devices (Mobile Phone) to read data from the Peripheral device (Arduino). And BLENotify allows remote clients to get notifications if this characteristic changes.
Now we will jump on to the Setup function.

```
Serial.begin(9600);
while (!Serial);
pinMode(LED_BUILTIN, OUTPUT);
if (!BLE.begin()) {
    Serial.println("starting␣BLE␣failed!");
    while (1);
}
```

Here it will initialize the Serial Communication and BLE and wait for serial monitor to open.
Set a local name for the BLE device. This name will appear in advertising packets and can be used by remote devices to identify this BLE device.

```
BLE.setLocalName("BatteryMonitor");
```

```
BLE.setAdvertisedService(batteryService);
batteryService.addCharacteristic(batteryLevelChar);
BLE.addService(batteryService);
```

Here we will add and set the value for the Service UUID and the Characteristic.

```
BLE.advertise();
Serial.println("Bluetooth device active, waiting for connections...");
```

And here, we will Start advertising BLE. It will start continuously transmitting BLE advertising packets and will be visible to remote BLE central devices until it receives a new connection.

```
BLEDevice central = BLE.central();
if (central) {
    Serial.print("Connected to central: ");
    Serial.println(central.address());
    digitalWrite(LED_BUILTIN, HIGH);
```

And here, the loop function. Once everything is setup and have started advertising, the device will wait for any central device. Once it is connected, it will display the MAC address of the device and it will turn on the builtin LED.

```
    while (central.connected()) {
        int battery = analogRead(A0);
        int batteryLevel = map(battery, 0, 1023, 0, 100);
        Serial.print("Battery Level % is now: ");
        Serial.println(batteryLevel);
        batteryLevelChar.writeValue(batteryLevel);
        delay(200);
    }
```

Now, it will start to read analog voltage from A0, which will be a value in between 0 and 1023 and will map it with in the 0 to 100 range. It will print out the battery level in the serial monitor and the value will be written for the batteryLevelchar charecteristics and waits for 200 ms. After that the whole loop will be executed again as long as the central device is connected to this peripheral device.

```
    digitalWrite(LED_BUILTIN, LOW);
    Serial.print("Disconnected from central: ");
    Serial.println(central.address());
    Once it is disconnected, a message will be shown on the central device and LE
```

12.7.6. Installing the App for Android

In your Android smartphone, install the app “nRF Connect”. Open it and start the scanner. You will see the device “Battery Monitor” in the device list. Now tap on connect and a new tab will be opened.
Go to that and you will see the services and characteristics of the device.Tap on Battery service and you will the battery levels being read from the Arduino.

12.7.7. Example 2 – Arduino BLE Accelerometer Tutorial

I showed you a very easy example to show you jpw you can send simple data via Bluetooth. If you are new to this, try this example first. It will help to give you a better understanding of the BLE.

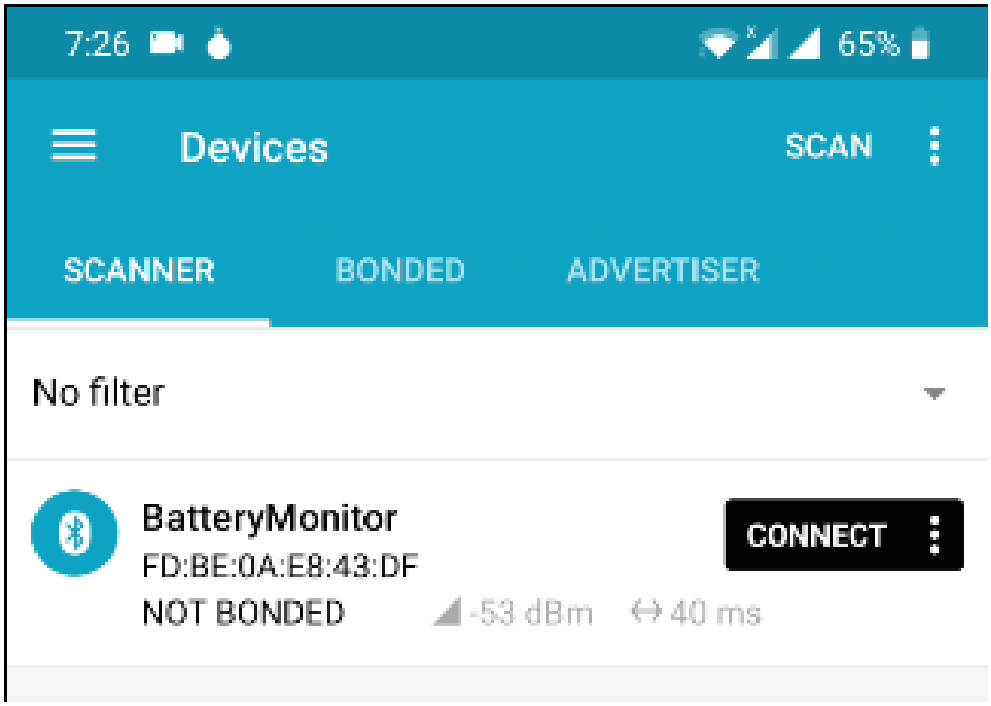


Figure 12.7.: Battery app “nRF Connect”

Step 1 – Installing Libraries

For this example, we will need two libraries

- `ArduinoBLE` – To Send Data via Bluetooth
- `LSM9DS1` – To Read data from inbuilt Accelerometer

Both these libraries are available in library manager. Simply search for that using the name and click on install.

Step 2 – Test Accelerometer Code (Optional)

Now we will try running the accelerometer code to make sure that these data are being read properly. For that, use the below code.

```
#include <Arduino_LSM9DS1.h>

void setup() {
    Serial.begin(9600);
    while (!Serial);
    Serial.println("Started");

    if (!IMU.begin()) {
        Serial.println("Failed to initialize IMU!");
        while (1);
    }

    Serial.print("Accelerometer sample rate = ");
    Serial.print(IMU.accelerationSampleRate());
    Serial.println(" Hz");
```

```
    Serial.println();
    Serial.println("Acceleration␣in␣G's");
    Serial.println("XtYtZ");
}

void loop() {
    float x, y, z;

    if (IMU.accelerationAvailable()) {
        IMU.readAcceleration(x, y, z);

        Serial.print(x);
        Serial.print('t');
        Serial.print(y);
        Serial.print('t');
        Serial.println(z);
    }
}
```

Once uploaded, start the serial monitor. You will see the data being populated. Try tilting the board in all direction and you will see the value changes accordingly.

Step 3 – Upload the Code

Now its time to upload the complete code. Copy the code below and paste it in the IDE.

```
#include <ArduinoBLE.h>
#include <Arduino_LSM9DS1.h>

int accelX=1;
int accelY=1;
float x, y, z;

BLEService customService("1101");
BLEUnsignedIntCharacteristic customXChar("2101", BLERead | BLENotify);
BLEUnsignedIntCharacteristic customYChar("2102", BLERead | BLENotify);

void setup() {
    IMU.begin();
    Serial.begin(9600);
    while (!Serial);

    pinMode(LED_BUILTIN, OUTPUT);

    if (!BLE.begin()) {
        Serial.println("BLE␣failed␣to␣Initiate");
        delay(500);
        while (1);
    }

    BLE.setLocalName("Arduino␣Accelerometer");
    BLE.setAdvertisedService(customService);
    customService.addCharacteristic(customXChar);
    customService.addCharacteristic(customYChar);
    BLE.addService(customService);
```

```
    customXChar.writeValue(accelX);
    customYChar.writeValue(accelY);

    BLE.advertise();

    Serial.println("Bluetooth device is now active, waiting for connections...");
}


void loop() {

    BLEDevice central = BLE.central();
    if (central) {
        Serial.print("Connected to central: ");
        Serial.println(central.address());
        digitalWrite(LED_BUILTIN, HIGH);
        while (central.connected()) {
            delay(200);
            read_Accel();

            customXChar.writeValue(accelX);
            customYChar.writeValue(accelY);

            Serial.print("At Main Function");
            Serial.println("");
            Serial.print(accelX);
            Serial.print(" - ");
            Serial.println(accelY);
            Serial.println("");
            Serial.println("");
        }
    }
    digitalWrite(LED_BUILTIN, LOW);
    Serial.print("Disconnected from central: ");
    Serial.println(central.address());
}

void read_Accel() {

    if (IMU.accelerationAvailable()) {
        IMU.readAcceleration(x, y, z);
        accelX = (1+x)*100;
        accelY = (1+y)*100;

    }
}
```

Select the right port and board. Click on upload.

Step 4 – Testing Arduino BLE Accelerometer

In your Android smartphone, install the app "nRF Connect". Open it and start the scanner. You will see the device "Arduino Accelerometer" in the device list. Now tap on connect and a new tab will be opened.
Go to that and you will see the services and characteristics of the device.

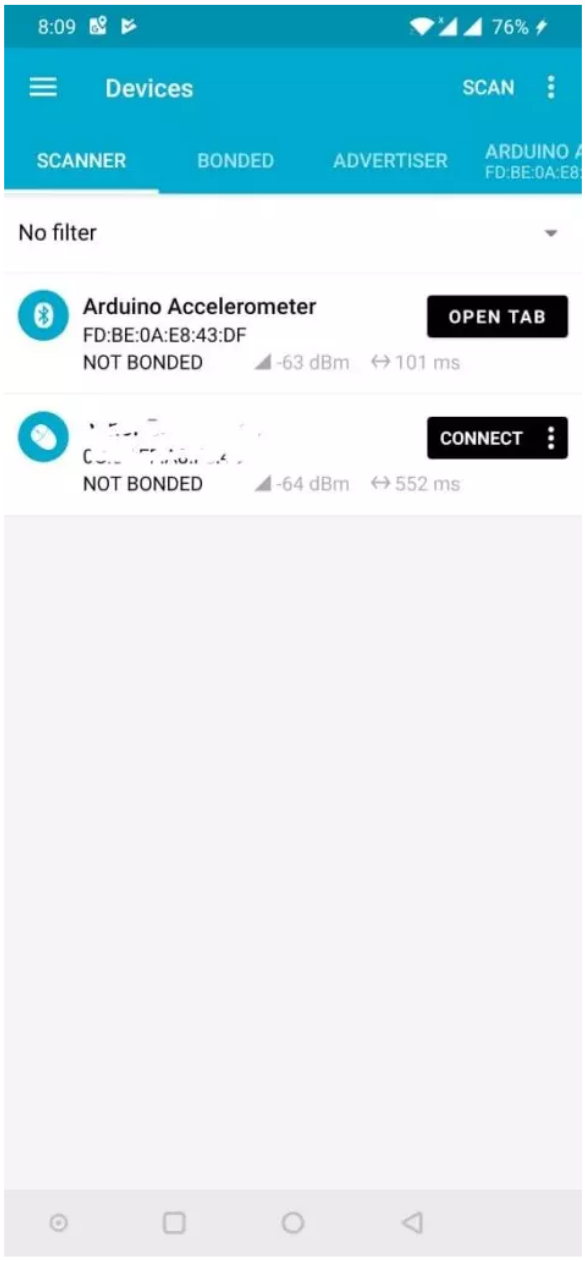


Figure 12.8.: nRF Connect Screen

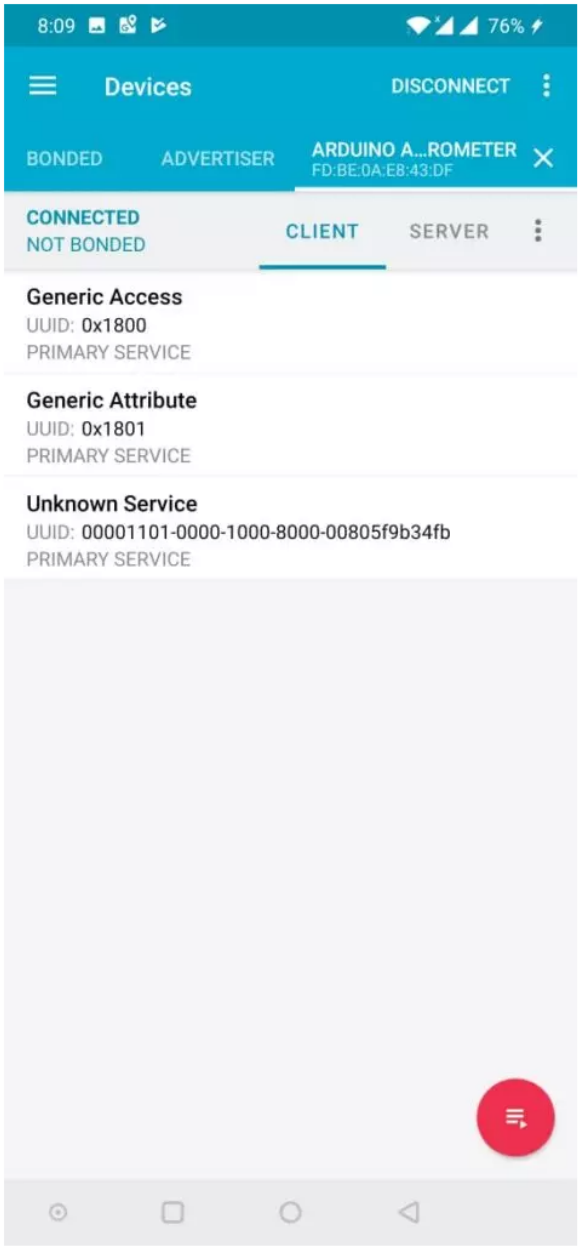


Figure 12.9.: Accelerometer Values Read from Arduino Using BLE

Tap on Unknown Service and you will see the accelerometer values being read from the Arduino.
In the next post, I will show you how you can send inbuilt sensor values such as accelerometer, gyroscope, color sensor and gesture sensor from the Arduino to your phone as well as another Arduino via BLE.

12.8. Arduino Library `BLEAK`

Low Energy (BLE) protocol. BLE allows you to exchange data between devices wirelessly and efficiently. You can use the bleak library in Python to interact with BLE devices.
To get started, you need to install the bleak library using pip:

```
pip install bleak
```

Then, you need to write a Python program that can scan for BLE devices, connect to the Arduino Nano 33 BLE 33 Sense, and read or write data to its characteristics. Characteristics are the attributes that define the data and behavior of a BLE device. For example, the Arduino Nano 33 BLE 33 Sense has a characteristic for each color of its RGB LED.
You can find an example of a Python program that can control the RGB LED of the Arduino Nano 33 BLE 33 Sense using bleak here3. You can also find the corresponding Arduino sketch that sets up the BLE service and characteristics for the Nano 33 BLE 33 Sense here4.

12.9. Test

12.10. Test with Bluetooth Module Connection

The same procedure we need to follow for making the successful bleutooth coonection, the one we follow for on-board sensors. Below figure 12.10 shows the ArduinoBLE library in the example section of Arduino IDE.

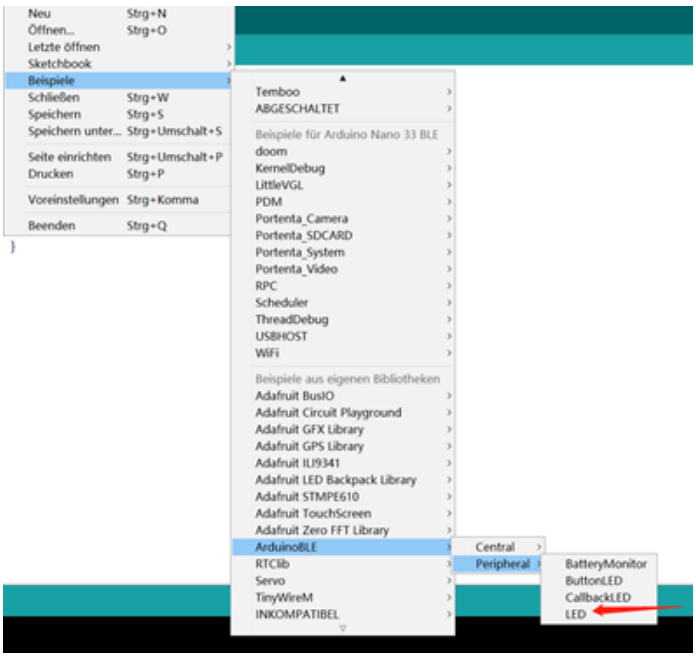
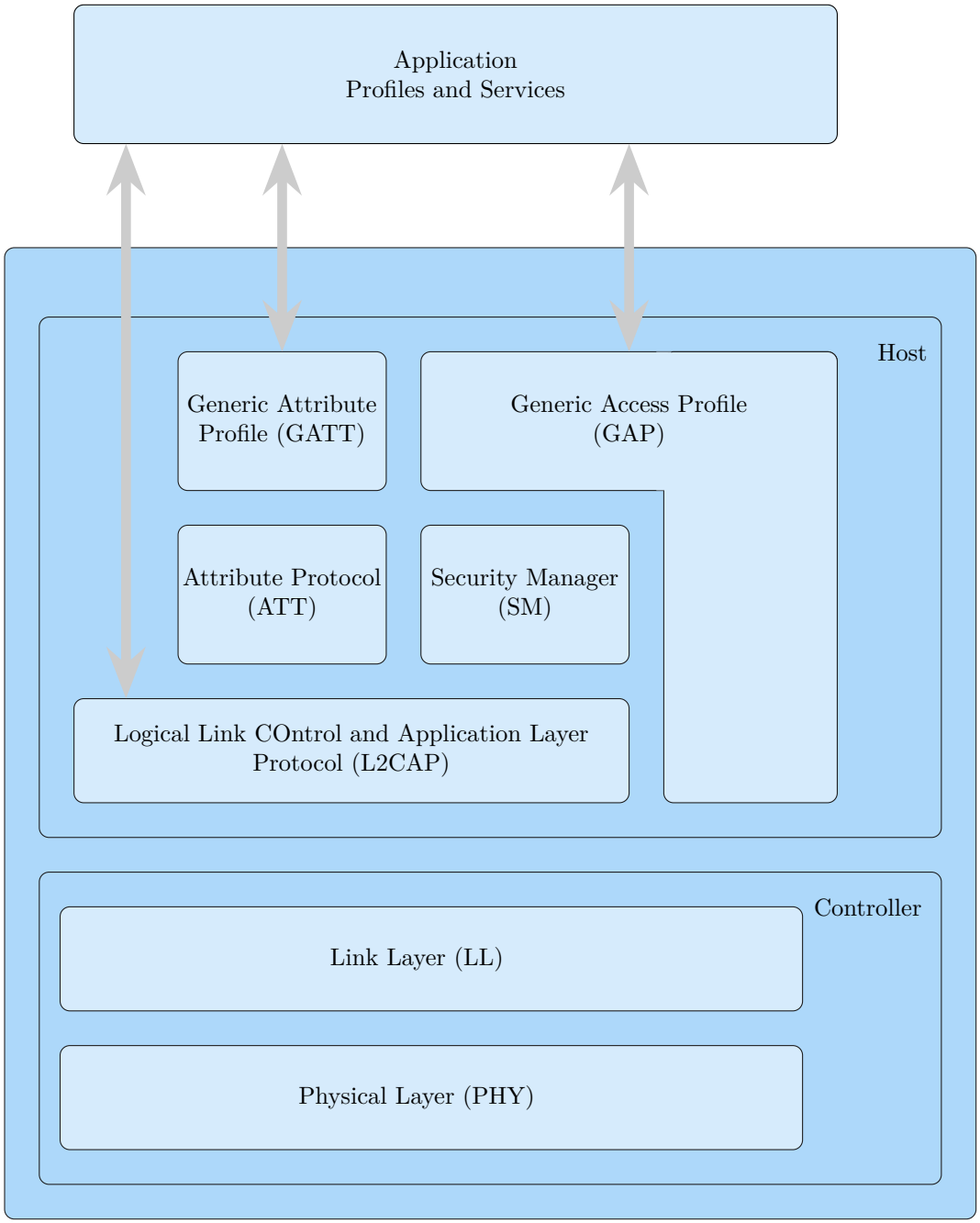


Figure 12.10.: Bluetooth Connection

Bluetooth wireless technology allows us to share the data, the voice, the music, the video, and a lot of information between paired devices, It is built into many products, from mobile phones, cars to medical devices and computers. It has lower power consumption. It is easily upgradeable. It has range better than Infrared communication. Bluetooth is used for voice and data transfer, we can communicate by recieving and sending the data with other bluetooth connected devices. It is also possible to output a value on the phone side or also on the laptop by making the proper pairing.

12.11. BLEStack



12.12. Control Arduino Nano BLE with Bluetooth & Python

https://www.hackster.io/sridhar-rajagopal/control-arduino-nano-ble-with-bluetoot
Demonstrated with the on-board RGB LED.

12.13. Peripheral Device aka Arduino Nano

The Generic Attribute Profile (or GATT) defines how Bluetooth LE devices can transfer data back and forth, using the Attribute Protocol (or ATT) which organizes

the data hierarchically into Service and Characteristics.
For our purpose, we define a top level service, as well as 3 characteristics, corresponding to each of the 3 colors of our RGB LED:

```
BLEService nanoService("13012F00-F8C3-4F4A-A8F4-15CD926DA146");
BLEByteCharacteristic redLEDCharacteristic("13012F01-F8C3-4F4A-A8F4-15CD926DA146"
BLEByteCharacteristic greenLEDCharacteristic("13012F02-F8C3-4F4A-A8F4-15CD926DA1
BLEByteCharacteristic blueLEDCharacteristic("13012F03-F8C3-4F4A-A8F4-15CD926DA146
```

The UUIDs used above have been randomly generated. They must match between the peripheral device (the Nano) and the central device (the Python program).
In our setup function, apart from setting up the pinMode for the RGB LEDs, we also setup our BLE service. Here, we set the local name for our BLE peripheral - central devices will see this when trying to connect to advertising services.
We add the service and characteristics we defined above, set their initial values, and start advertising. Here, at this point, central devices that are listening will see our advertising packets and know that we are ready to accept connections.

```
// set advertised local name and service UUID:
BLE.setLocalName("Arduino Nano 33 BLE Sense");
BLE.setAdvertisedService(nanoService);
// add the characteristic to the service
nanoService.addCharacteristic(redLEDCharacteristic);
nanoService.addCharacteristic(greenLEDCharacteristic);
nanoService.addCharacteristic(blueLEDCharacteristic);
// add service
BLE.addService(nanoService);
// set the initial value for the characeristic:
redLEDCharacteristic.writeValue(0);
greenLEDCharacteristic.writeValue(0);
blueLEDCharacteristic.writeValue(0);
// start advertising
BLE.advertise();
```

In our main loop(), we wait for a central device to connect. We then wait to see if any of the characteristics are written to, and the value of that characteristic. In our example, a non-zero value written to the redLEDCharacteristic, for example, will cause us to turn on the RED LED. When we encounter a zero value, we turn off the LED.

```
void loop() {
        BLEDevice central = BLE.central();
        if (central) {
                ...
                while (central.connected()) {
                        if (redLEDCharacteristic.written()) {
                                if (redLEDCharacteristic.value()) {
// any non-zero value
                                        Serial.println("RED LED on");
                                        digitalWrite(RED_PIN, LOW); // LOW will t
                                } else { // a zero value
                                        Serial.println(F("RED LED off"));
                                        digitalWrite(RED_PIN, HIGH); // HIGH will
                                }
                        }
                        .... do similar stuff for GREEN and BLUE
                }
```

```
                    // when the central disconnects, print it out:
                    Serial.print(F("Disconnected from central: "));
                    Serial.println(central.address());
            }
}
```

Here we have to note that the RGB LEDs on the Arduino Nano 33 BLE Sense follow a reverse logic, so setting the pin LOW turns ON the LED and setting it HIGH turns it OFF.
The onboard RGB LED also does not support PWM, so the states of the LEDs can either be ON or OFF, giving us 7 different colors:

R	G	B	
OFF	OFF	OFF	– OFF
ON	OFF	OFF	– RED
OFF	ON	OFF	– GREEN
OFF	OFF	ON	– BLUE
ON	ON	OFF	– ORANGE
ON	OFF	ON	– PURPLE
OFF	ON	ON	– CYAN
ON	ON	ON	– WHITE

The Central device can also read the characteristic's value. This is automatically handled by the BLE library and we don't need to do anything more. We initialize the value appropriately on startup, and maintain the value, so it can be read anytime.
Let's dive into the code - Central Device aka Python On the Python side, we utilize the Python bleak library for BLE support - of course, the machine we are running it on needs to have Bluetooth support as well!
We also use the asyncio asynchronous framework for our program. Our main loop here is the run() function. It goes into discover mode and checks the local names of the Bluetooth devices that are advertising, and connects to the device with the local name of "Arduino Nano 33 BLE Sense" (the local name we chose).
It then reads the RED, GREEN and BLUE Characteristics to know the initial states of the LEDs, and then goes into the setColor() loop:

```
async def run():
global RED, GREEN, BLUE
print('ProtoStax Arduino Nano BLE LED Peripheral Central Service')
print('Looking for Arduino Nano 33 BLE Sense Peripheral Device...')
found = False
devices = await discover()
for d in devices:
if 'Arduino Nano 33 BLE Sense'in d.name:
print('Found Arduino Nano 33 BLE Sense Peripheral')
found = True
async with BleakClient(d.address) as client:
print(f'Connected to {d.address}')
val = await client.read_gatt_char(RED_LED_UUID)
if (val == on_value):
print ('RED ON')
RED = True
else:
print ('RED OFF')
RED = False
... do similar stuff for GREEN and BLUE
while True:
await setColor(client)
```

```
if not found:
print('Could not find Arduino Nano 33 BLE Sense Peripheral')
```

In setColor(), it prompts for user input, and checks the user input to see if the string has any 'r', 'g' or 'b' values in it. It then toggles the color and writes the appropriate value to the characteristic, and waits for further user input again. This continues until the user hits Control-C to stop the program.

```
async def setColor(client):
global RED, GREEN, BLUE
val = input('Enter rgb to toggle red, green and blue LEDs :')
print(val)
if ('r' in val):
RED = not RED
await client.write_gatt_char(RED_LED_UUID, getValue(RED))
```

... do similar stuff for GREEN and BLUE

12.13.1. Taking the Project Further

The Arduino Nano has a whole bunch of sensors onboard. You can extend the project by adding characteristics for the other sensors - for example, reading the temperature, humidity and pressure values (there is no need to write these values, so the characteristics can be defined read-only). You'll also need to modify the program to handle the read command - polling the sensor and writing the data with the current value of the sensor.
On the central device (Python Program), you can choose the store the data along with the timestamp of when it was read, to a database or comma-separated file. This way, your central device can act like a data logger, and you can even utilize the data to perform actions, or send the data to a cloud service like AWS to process and run analytics on. Having the central device on Python allows you to utilize the numerous packages and libraries and capabilities of Python.
Happy Coding! If you have any questions, feel free to ask them in the comments below! We'll also be happy to hear how you have taken the project further and what wonderful creations you have made!

13. Ultraschallsensor HC-SR04

13.1. Einleitung

Ein Ultraschallsensor misst berührungslos den Abstand zwischen Sensor und einer Werkstückoberfläche. Aus dieser Distanz kann der aktuelle Abstand zu der Oberfläche berechnet werden. Der Sensor arbeitet nach dem Prinzip der Laufzeitmessung von Ultraschallwellen und bietet eine einfache, kostengünstige und präzise Möglichkeit zur Pegelüberwachung, Warenerkennung und Abstandsmessung.[AG24] [TR18] [HS23]

13.2. Ultraschallsensoren zur Abstandsmessung

Ultraschallsensoren zur Abstandsmessung dienen der Ermittlung von Objektabständen über die Laufzeit von Ultraschallimpulsen und die Schallgeschwindigkeit. Da die Schallgeschwindigkeit temperatur- und luftdruckabhängig ist und zusätzlich durch die Luftzusammensetzung beeinflusst wird, ist bei der Auslegung auf solche Einflussfaktoren Rücksicht zu nehmen. Bei Nichtbeachtung sind Messungenauigkeiten in der Entfernungsmessung zu erwarten. [HS09] Umwelteinflüsse wie Feuchte, Staub und Rauch beeinflussen die Messgenauigkeit nicht. Einige Gase und Flüssigkeiten können die Sensorfunktion jedoch beeinträchtigen oder gar untüchtig machen. Beispiele hierfür sind CO2 und Dämpfe von Lösungsmitteln. [HS+12] Ultraschallsensoren können nach ihrem Sensorprinzip in Tastbetrieb und Schrankenbetrieb unterschieden werden. Der Tastbetrieb basiert dabei auf der Auswertung der Laufzeitmessung zwischen dem Senden und Empfangen des Schalls. Der Schrankenbetrieb dahingegen beruht auf der Kontrolle, ob das gesendete Signal empfangen wurde [HLG19] Der Ultraschallsensor zur Abstandsmessung beruht auf ersterem Prinzip. Im Folgenden soll die Funktionsweise näher erläutert werden. Zunächst aktiviert die Steuerelektronik den Leistungsverstärker, welcher den Schallwandler mit einer elektrischen Leistung und einer Sinusspannung ansteuert. Dadurch arbeitet der Schallwandler als Lautsprecher und sendet einen Ultraschallimpuls, auch Burst genannt, aus. Bis der Schallwandler ausgeschwungen ist, dauert es zwei- bis dreimal so lang wie die Sendezeit. Anschließend leitet die Steuerelektronik den Empfangsbetrieb ein und der Schallwandler dient nun als Mikrofon. Sollte sich in der Schallkeule ein Objekt befinden, wird der Ultraschallimpuls zum Sender zurückreflektiert. Der Schallwandler beginnt zu schwingen und erzeugt eine Sinusschwingung, die auf einen Verstärker übertragen wird. Durch Autokorrelation wird kontrolliert, ob es sich bei dem reflektierten Schall wirklich um das Echo der ausgesandten Ultraschallwellen handelt. [HS09] Eine ganze Bandbreite an Objekten unabhängig von der Oberflächenbescha‘enheit, Farbe, Transparenz [HS+12] und Form [HS09] kann durch den Ultraschall erfasst werden. Der Ultraschall findet seine Anwendung in sehr vielen verschiedenen Bereichen 13.3, näher soll auf ihre Anwendung in den Bereichen der Medizin und Industrie eingegangen werden. In der Medizin gehört der Ultraschall mit zu den wichtigsten und häufigsten genutzten Diagnoseverfahren. Er lässt sich vielfältig einsetzen. So dient es zur Erkennung und Darstellung von Organen, gefüllten Hohlräumen, Gefäßen, Knochen, Sehnen und anderen Objekten im menschlichen Körper. Kennzeichnend für die Anwendung des Ultraschalls in der Medizin ist außerdem sein geringes Gesundheitsrisiko für Patienten. In der Industrie gibt es vielfältige Einsatzbereiche von Ultraschallsensoren. Sie können der Überwachung von Stellplätzen in der Lagerhaltung, der Füllstandbestimmung von Schüttgütern [HS09] oder der Detektion von Objekten und Grenzflächen dienen.

Beispiel für letztere Erwähnung ist die Rückfahrhilfe bei Fahrzeugen. Eine weitere sehr wichtige Anwendung findet sich in der zerstörungsfreien Prüfung von Werkstücken und WerkstoNen wieder. Vorrangig wird dabei das Materialgefüge auf Unebenheiten und Au'älligkeiten überprüft. Solche Untersuchungen werden häufig für sicherheitsrelevante Bauteile wie Eisenbahnräder oder Druckbehälter für Atomanlagen verwendet [MM17].

13.3. Grundlagen zur Verwendung eines Ultraschallsensors

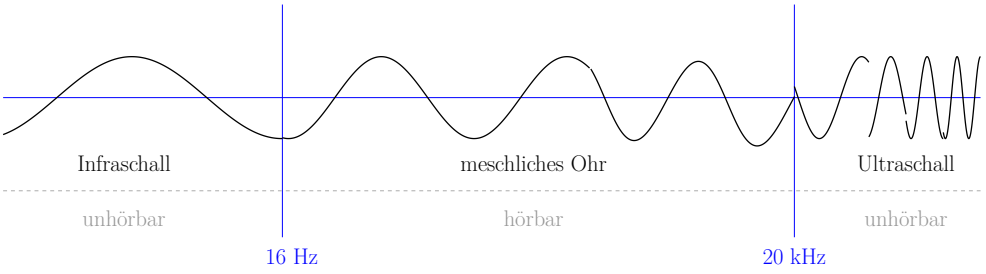
Um einen Ultraschallsensor gezielt und effizient einsetzen zu können, ist ein grundlegendes Verständnis über die Funktionsweise, technischen Eigenschaften und Anwendungsvoraussetzungen abstandsmessender Ultraschallsensoren notwendig. Die folgenden Unterkapitel bieten eine strukturierte Einführung in die physikalischen und technischen Grundlagen des Sensors. Ziel ist es, ein tiefergehendes Verständnis für den Aufbau, die Signalverarbeitung und die praktische Handhabung eines Ultraschallsensors zu vermitteln. [AG24]

13.3.1. Grundlagen Schall

Als Schall bezeichnet man die Ausbreitung von lokalen Druckschwankungen in einem elastischen Medium. Die Schallwelle breitet sich dabei longitudinal aus, die Auslenkung der Moleküle erfolgt also in Ausbreitungsrichtung [TM24]. Die Einteilung der Schallbereiche erfolgt auf Grundlage der Frequenz, siehe außerdem 13.1:

- *Infraschall:* bis 16 Hz,
- *Hörbarer Schall:* von 16 Hz bis 20 kHz,
- *Ultraschall:* von 20 kHz bis 1 GHz,
- *Hyperschall:* über 1 GHz [HS23].

Figure 13.1.: Frequenz der verschiedenen Schall-Arten



13.3.2. Schallgeschwindigkeit und Wellenlänge

Die Geschwindigkeit, mit der sich Schall in Luft ausbreitet, ist von der Lufttemperatur T und der Luftfeuchtigkeit φ abhängig. Da die Luftfeuchtigkeit einen erheblich geringeren Einfluss auf den Wert hat

$$\Delta v_{Schall,\varphi} \approx 1,2\,\frac{\mathrm{m}}{\mathrm{s}};\ 0 < \varphi < 1,\ T_C = 20\,^{\circ}\mathrm{C}, \tag{13.1}$$

[HS23] lässt sich die Schallgeschwindigkeit mit ausreichender Genauigkeit mit der Formel 13.2

$$v_{Schall} = \sqrt{\kappa R T} \tag{13.2}$$

und der Annahme $\varphi = 0$ berechnen [TM24]. Dabei ist $\kappa = 1{,}4$ der Adiabatenexponent von Luft, $R = 287\ \frac{\mathrm{J}}{\mathrm{kg\ K}}$ die spezifische Gaskonstante von Luft und T die absolute Temperatur in Kelvin [TM24; Wil22]. Für eine angenommene Temperatur von $T_C = 20$℃ ergibt sich damit eine Geschwindigkeit von $v_{Schall} = 343{,}2\,\frac{\mathrm{m}}{\mathrm{s}}$. In Abb. 13.2 ist die Schallgeschwindigkeit in Luft im Temperaturbereich von $T_C = -10\,$℃ bis $60\,$℃ dargestellt.

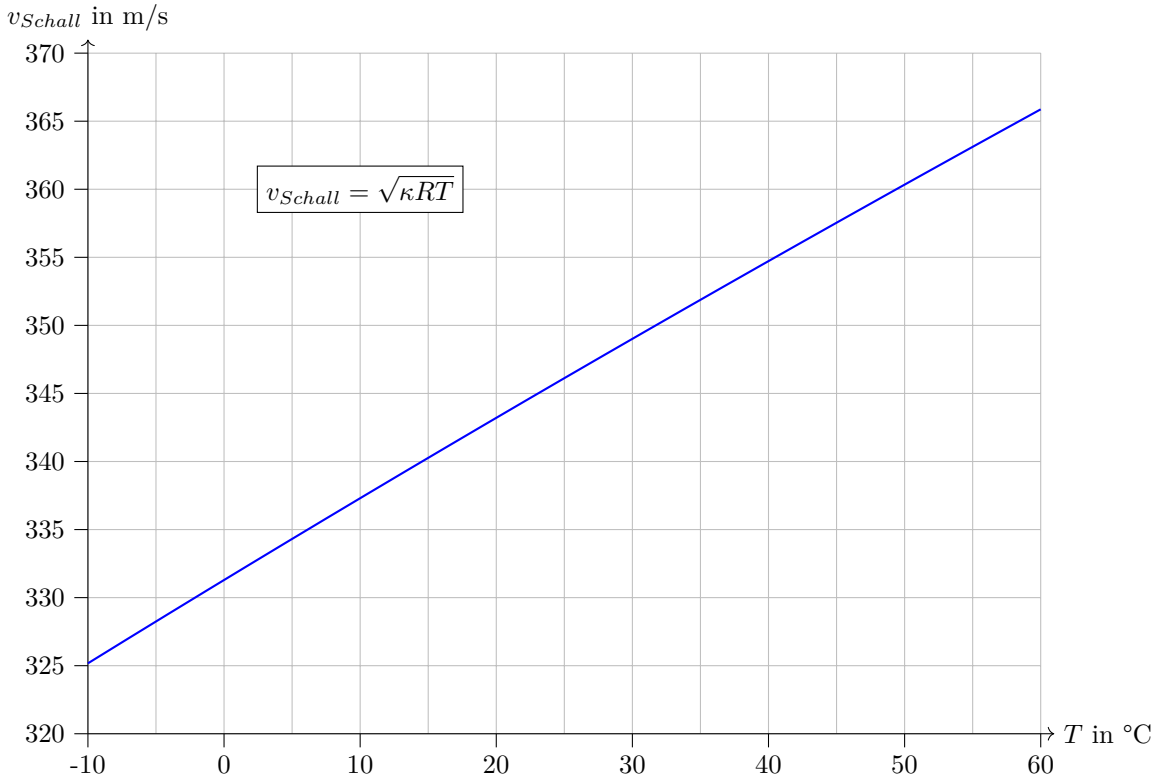


Figure 13.2.: Ausbreitungsgeschwindigkeit von Schall in Luft

Über den Formelzusammenhang 13.3

$$v_{Schall} = \lambda f \tag{13.3}$$

lässt sich bei gegebener Frequenz f und mit der berechneten Schallgeschwindigkeit v_{Schall} die Wellenlänge des Schalls berechnen [Wil22]. Diese beträgt bei einer angenommenen Frequenz von $f = 40$kHz und der in Abschnitt 13.3.2 berechneten Schallgeschwindigkeit $\lambda = 8{,}58 \cdot 10^{-3}\,$m.

13.3.3. Weg- und Abstandsmessung mit Ultraschall

Um eine Entfernung oder einen Abstand eines Objekts zu dem Sensor zu ermitteln, wird der Sensor im sog. *Tastbetrieb mit Echo-Laufzeit-Messung*, wie in Abb. 13.3 zu sehen, verwendet. Dabei sendet der Sensor zunächst ein Schallpuls in Richtung des zu messenden Objekts aus (Lautsprecher), welche zum Teil von diesem reflektiert wird und somit zurück zum Sensor gelangt (Echo). Dieses Echo kann der Sensor detektieren

(Mikrofon) und über die gemessene Laufzeit des Echos sowie die Schallgeschwindigkeit im dem den Sensor umgebenden Medium die Distanz berechnen [HS23].

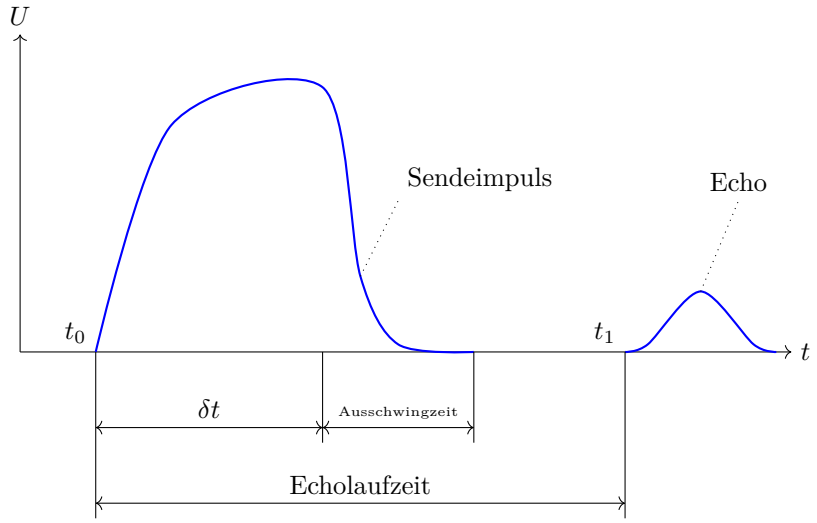


Figure 13.3.: Spannungsverlauf Schallwandler bei einer Echo-Laufzeit-Messung, nach [HS23]

13.4. Ultraschall-Abstandsensor HC-SR04

Der Ultraschallabstandssensor in Abbildung 13.4 enthält den Sensor HC-SR04. Er ist als zweiköpfiger Sensor ausgeführt, wodurch er über je einen Lautsprecher und ein Mikrofon verfügt und ist zur Erfassung von Abständen in einem Bereich von 3 bis 400 cm spezifiziert. Dafür nutzt er eine Schallfrequenz von 40 kHz. Er ist als zweiköpfiger Sensor ausgeführt, wodurch er über je einen Lautsprecher und ein Mikrofon verfügt. Die Abmessungen des Sensors betragen in der Breite 43 mm, in der Höhe 15 mm und in der Tiefe 20 mm. Die Auflösung des Sensors beträgt ±1 mm. Angeschlossen wird der Abstandssensor über den VCC-Pin, Trig/Serial Clock (SCL)-Pin, Echo/Serial Data (SDA)-Pin und dem GND-Pin. Der Sensor kann über die drei Schnittstellen General Purpose Input Output (GPIO), Universal Asynchronous Receiver Transmitter (UART) und Inter-Integrated Circuit (I^2C) verbunden werden [Sim].

13.5. Spezifikation

Der Sensor HC-SR04 ist zur Erfassung von Abständen in einem Bereich von 3 bis 400 cm mit einer Auflösung von ±1mm spezifiziert. Dafür nutzt er eine Schallfrequenz von 40 kHz. Die Abmessungen des Sensors betragen in der Breite 43 mm, in der Höhe 15 mm und in der Tiefe 20 mm.
Angeschlossen wird der Abstandssensor über den SCL-Pin für das Triggersignal, den SDA-Pin fürs Echo, den VCC-Pin zur Stromversorgung und dem GND-Pin.
Der Sensor kann über die drei Schnittstellen GPIO, UART und I^2C verbunden werden [Sim].
Bei den folgenden Beispielprogrammen wird der Arduino Nano 33 BLE Sense verwendet. In den Beispielen werden die Pins D6 und D9 für die Datenübertragung verwendet. In der Tabelle 13.1 ist dies für die Grove-Schnittstelle festgehalten und zudem auch in der Abbildung 13.4 zu sehen.

Figure 13.4.: Ultraschallsensor mit Pin-Belegung

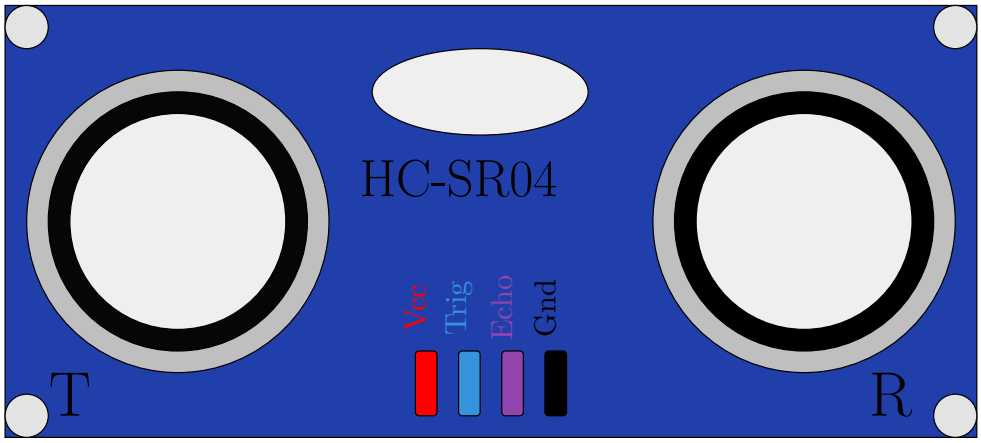


Table 13.1.: Pinbelegung für den Ultraschallabstandssensor [Sim]

Arduino Pin	Ultraschall Abstandssensor
D9	Trig
D6	Echo
3V3	VCC
GND	GND

13.5.1. Anwendungen

Ultraschallsensoren finden in der Praxis Anwendung in verschiedenen Bereichen, die auf die Ausbreitung des Schalls über unterschiedliche Medien basieren [HS23]. Es lassen sich drei Hauptkategorien unterscheiden: In der ersten Kategorie breitet sich der Schall über Luft aus. Typische Anwendungen sind hier Füllstandmessungen, Durchmessererfassungen bei Auf- und Abwickelsteuerungen sowie Kollisionsschutz und Objekterkennung [HS23]. Die zweite Kategorie umfasst Sensoren, bei denen sich der Schall über Flüssigkeiten ausbreitet. Diese finden vor allem im maritimen Bereich Verwendung, beispielsweise in Sonarsystemen zum Orten von Unterseebooten oder Fischschwärmen sowie in Echoloten zur Kartografie des Meeresbodens [HEG21]. Die dritte Kategorie umfasst Körperschallsensoren, die in der Industrie zur zerstörungsfreien Materialprüfung sowie in der Medizin zur Ultraschalldiagnostik verwendet werden. [HEG21]

13.6. Bibliothek

13.6.1. Beschreibung

Zur Ansteuerung des Ultraschallsensors HC-SR04 wird eine Arduino-kompatible Bibliothek `NewPing` verwendet, die die Initialisierung des Sensors sowie das Senden und Empfangen von Ultraschallsignalen abstrahiert. Diese Bibliothek kapselt die Funktionen zum Triggern, zur Laufzeitmessung und zur Berechnung der Distanz [AG24].

13.6.2. Installation

Die Installation der Bibliothek `NewPing`, welche die Funktionalität des Ultraschallsensors HC-SR04 unter Arduino abstrahiert, erfolgt über den Bibliotheksverwalter der

Arduino IDE. Diese bietet eine benutzerfreundliche Oberfläche zur Integration externer Bibliotheken. [AG24]
Schrittweise Anleitung:

1. Öffne die Arduino IDE (Version 1.8.x oder neuer empfohlen).
2. Navigiere zu: Sketch > Bibliothek einbinden > Bibliotheken verwalten...
3. Im Suchfeld oben rechts den Begriff `NewPing` eingeben
4. Wähle den Eintrag `NewPing` by Tim Eckel aus (Version 1.9.7 oder neuer empfohlen).
5. Klicke auf „Installieren “.

13.6.3. Funktionen

Die NewPing-Bibliothek bietet eine Vielzahl von Funktionen, die die Nutzung von Ultraschallsensoren erleichtern und erweitern. Die wichtigsten Funktionen sind: [AG24]

`NewPing()`: Initialisierung eines Sensors durch Angabe der Pins für Trigger- und Echo-Signale sowie der maximalen Messdistanz.
`ping()`: Senden eines Ultraschallimpulses und Rückgabe der gemessenen Laufzeit in Mikrosekunden.
`getDistance()`: Direkte Ausgabe der gemessenen Entfernung in Zentimetern basierend auf der Standard-Schallgeschwindigkeit in Luft.
`pingMedian()`: Durchführung mehrerer Messungen und Rückgabe des Medianwertes zur Reduktion von Ausreißern.

Diese Funktionen ermöglichen eine einfache und präzise Nutzung von Ultraschallsensoren in verschiedenen Anwendungen, von der Einzelmessung bis hin zur Durchführung mehrerer Messungen zur Reduzierung von Ausreißern.

13.7. Kalibrierung

Damit der HC-SR04 präzise misst, muss er kalibriert werden. Die wichtigste Einflussgröße ist die Schallgeschwindigkeit, da sie von der Umgebungstemperatur abhängt. Eine Kalibrierung erfolgt durch den Vergleich mit einer bekannten Referenzdistanz. Ergibt sich eine systematische Abweichung, kann diese durch einen festen Korrekturwert (Offset) oder durch Anpassung der Schallgeschwindigkeit ausgeglichen werden. [TR14] Der Ultraschallabstandssensor wird für die Verwendung bei der Temperatur von 20 °C ausgelegt. Die berechnete Schallgeschwindigkeit $v_{Schall} = 343,2\,\frac{\mathrm{m}}{\mathrm{s}}$ bei 20 °C wird für die Umrechnung der Schalldauer in die Distanz zur Reflexionsfläche verwendet.

13.8. Simple Code

Zur Überprüfung der grundlegenden Funktionalität des Ultraschallsensors wurde ein Simple Code 13.1 verwendet. Das Programm dient zum einfachen Test mithilfe der NewPing-Bibliothek. Es misst die Laufzeit (Echozeit) des Schallsignals . Die Werte werden anschließend über den Seriellen Monitor ausgegeben.

Listing 13.1.: Einfacher Code zum Testen des Ultraschallsensors

```
1000 /**
 * @file laufzeitMessung.ino
1002 * @brief Simple test program for measuring the echo time of an HC-SR04
     ultrasonic sensor using the NewPing library.
 *
1004 * This program initializes the ultrasonic sensor and performs a basic
     runtime measurement (ping)
 * to determine the echo time in microseconds. The result is printed to
     the Serial Monitor.
1006 */

1008 #include <NewPing.h>

1010 // Define pins and maximum distance
#define TRIGGER_PIN  9    ///< Pin connected to the trigger pin of the
     HC-SR04
1012 #define ECHO_PIN     10   ///< Pin connected to the echo pin of the HC-
     SR04
#define MAX_DISTANCE 200  ///< Maximum distance to measure (in cm)
1014
/// Create a NewPing object named 'sonar'
1016 NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);

1018 /**
 * @brief Arduino setup function.
1020 *
 * Initializes serial communication for output.
1022 */
void setup() {
1024   Serial.begin(9600);
   Serial.println("Ultrasonic runtime measurement started...");
1026 }

1028 /**
 * @brief Arduino main loop function.
1030 *
 * Measures the echo time using the ping() function and prints it in
     microseconds.
1032 */
void loop() {
1034   delay(1000); // Wait 1 second between measurements

1036   // Measure echo time in microseconds
   unsigned int echoTime = sonar.ping();
1038
   // Print result
1040   Serial.print("Echo time: ");
   Serial.print(echoTime);
1042   Serial.println("mikro-sekunden");
}
```

../../Code/Arduino/UltraSonicHCSR04/laufzeitMessung/laufzeitMessung.ino

13.9. Einfache Anwendung

In der Industrie ist es oft wichtig zu wissen, wann ein bestimmter Pegel einer Flüssigkeit unterschritten wird. Dafür können Ultraschallsensoren eingesetzt werden. Sie messen kontinuierlich den Pegel und können dem Programm so permanent einen Messwert bereitstellen. Wird ein bestimmter Messwert über- oder unterschritten, kann das Programm eine Warnmeldung erzeugen. Dieses Arduino-Programm 13.2 nutzt einen HC-SR04 Ultraschallsensor, um die Entfernung zu einem Objekt kontinuierlich zu überwachen. Es erkennt, ob eine kritische Schwelle (25 cm) unterschritten wurde und gibt in diesem Fall eine Warnmeldung im Seriellen Monitor aus.

13.10. Tests

13.10.1. Einfacher Funktionstest

Beim einfachen Funktionstest wird überprüft, ob der Sensor korrekt anspricht und brauchbare Messwerte liefert. Dazu wird der HC-SR04 an einen Mikrocontroller (z. B. Arduino Nano 33 BLE Sense Lite) angeschlossen und ein einfacher Sketch 13.3 aufgespielt, welcher die gemessene Entfernung über die serielle Schnittstelle ausgibt. [HS23] Ein Testobjekt wird in einem bekannten Abstand, typischerweise 20 cm, orthogonal vor den Sensor gehalten. Die gemessene Entfernung wird mit einem Maßband überprüft. Stimmen beide Werte überein, ist die Grundfunktion des Sensors sichergestellt. Dieser Test eignet sich insbesondere zur ersten Inbetriebnahme oder zur Überprüfung von Lötverbindungen, Spannungsversorgung und Signalverarbeitung.

13.10.2. Test aller Funktionen

Ein umfassender Funktionstest geht über die reine Abstandsmessung hinaus. Hierbei werden systematisch verschiedene Einflüsse auf die Messgenauigkeit überprüft: [HS23]

- **Oberflächenbeschaffenheit:** Glatte, harte Oberflächen liefern bessere Reflexionen als weiche oder absorbierende Materialien wie Schaumstoff oder Stoff
- **Ausrichtungswinkel:** Objekte, die schräg zur Sensorachse stehen, reflektieren das Signal schlechter oder gar nicht. Der Test prüft, in welchem Winkelbereich zuverlässige Ergebnisse erzielt werden.
- **Entfernungsbereich:** Es wird getestet, ob der Sensor sowohl im Nahbereich (unter 10 cm) als auch im Fernbereich (bis 400 cm) verlässlich misst.
- **Umwelteinflüsse:** Staub, leichte Luftströmungen oder Temperaturveränderungen werden simuliert, um die Stabilität des Sensors unter realistischen Bedingungen zu bewerten.
- **Reproduzierbarkeit:** Mehrfache Messungen unter gleichen Bedingungen sollten ähnliche Ergebnisse liefern. Eine Standardabweichung der Messwerte kann berechnet werden, um die Präzision zu bewerten.

13.10.3. Lösungsansätze

Die Genauigkeit von Ultraschall-Abstandsmessungen hängt unter anderem von der Schallgeschwindigkeit ab, welche wiederum temperaturabhängig ist. Wird die Geschwindigkeit im Auswertungsprozess als konstanter Wert angenommen, kann es bei Abweichungen von der tatsächlichen Umgebungstemperatur zu Messfehlern kommen. Eine Möglichkeit zur Korrektur besteht darin, die aktuelle Temperatur zu erfassen und die Schallgeschwindigkeit entsprechend anzupassen. In vielen Systemen wird hierfür

ein interner Temperatursensor verwendet. Liegt dieser jedoch direkt auf der Platine und ist in einem Gehäuse untergebracht, kann es durch Eigenwärme und Isolation zu einer zeitverzögerten oder verfälschten Temperaturmessung kommen. Die Verwendung eines separaten Temperatursensors außerhalb des Gehäuses wäre in solchen Fällen eine mögliche Lösung, ist jedoch mit zusätzlichem Aufwand verbunden. Für das vorliegende Projekt, bei dem ein visueller Effekt eines scheinbar schwebenden Wasserhahns erzeugt werden soll, ist eine sehr hohe Messgenauigkeit nicht zwingend erforderlich. Der Wasserstand im Behälter wird zusätzlich durch Bohrungen geregelt, die ein Überlaufen verhindern. Die Ultraschallmessung dient primär der groben Überwachung des Füllstandes, beispielsweise zur rechtzeitigen Nachfüllung. Geringe Messabweichungen durch temperaturbedingte Schwankungen der Schallgeschwindigkeit sind daher tolerierbar und haben keinen wesentlichen Einfluss auf die Funktionalität der Installation.

13.11. Weiterführende Anwendungen

Ultraschallsensoren wie der HC-SR04 finden in folgenden Gebieten ihre Anwendung:

- **Robotik:** Hinderniserkennung und Kollisionsvermeidung bei autonomen Systemen [TR14]
- **Industrieautomation:** In Tanks und Silos, auch bei Staub oder Dampf [HS23].
- **Fahrzeugtechnik:** Einparkhilfen und Abstandswarnsysteme [HS23].
- **Medizin:** Kontaktlose Abstandsmessung in sterilen Umgebungen, z. B. bei Desinfektionsstationen [HS23]

Die Stärken des Sensors liegen in der einfachen Integration, dem günstigen Preis und der Robustheit gegenüber Lichtverhältnissen oder Verschmutzung. Die genannten Eigenschaften werden auch in der Fachliteratur als Vorteile von Ultraschallsensoren gegenüber optischen Verfahren hervorgehoben [HS23]

13.12. Further Readings

Tränkler, H.-R.:mSensortechnik – Handbuch für Praxis und Wissenschaft. 2. Aufl., Springer Vieweg, 2018 [TR18]

- Ein schöner Einstieg und Überblick über die Sensortechnik und ihre Anwendung heutzutage.

Hering, E.: Sensoren in Wissenschaft und Technik – Funktionsweise und Einsatzgebiete. 3. Aufl., Springer Vieweg, 2023 [HS23]

- Ein tiefer Einblick in die Sensortechnik, besonders schöne Abbildungen und Erklärungen. Von Grund auf an eine Einführung für Ultraschallsensoren

Arduino AG: NewPing – Library Documentation, Arduino.cc, abgerufen 2025. [AG24]

- Auf der Arduino Website kann man sich viele Extra-Informationen zum Arduino und weiterführende Anwendungen, Beispiele und Projekte anschauen.

Listing 13.2.: Einfache Anwendung als Schwellenwert-Melder

```
/**
 * @file schwellenwertMeldung.ino
 * @brief Simple application using an HC-SR04 ultrasonic sensor to monitor a distance threshold.
 *
 * This program continuously measures the distance using the HC-SR04 sensor and triggers a warning
 * when a critical threshold of 25 cm is reached or undershot. The distance is measured using
 * pingMedian() for stable values, and getDistance() is also demonstrated for direct measurement.
 */

#include <NewPing.h>

#define TRIGGER_PIN     9    ///< Pin connected to the trigger pin of the HC-SR04
#define ECHO_PIN        10   ///< Pin connected to the echo pin of the HC-SR04
#define MAX_DISTANCE    100  ///< Maximum distance to measure (in cm)

#define TARGET_DISTANCE     30  ///< Reference distance (in cm)
#define CRITICAL_THRESHOLD  25  ///< Critical threshold for warning (in cm)

/// Create a NewPing object named 'sonar'
NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);

/**
 * @brief Arduino setup function.
 *
 * Initializes serial communication and outputs startup message.
 */
void setup() {
  Serial.begin(9600);
  Serial.println("Distance threshold monitoring started...");
}

/**
 * @brief Arduino main loop function.
 *
 * Continuously measures the distance using pingMedian().
 * If the measured distance is less than or equal to the critical threshold,
 * a warning message is printed.
 */
void loop() {
  delay(500); // Measurement interval

  // Use pingMedian() for stable distance reading (average of multiple pings)
  unsigned int distance = sonar.pingMedian();

  // Print measured distance
  Serial.print("Measured distance: ");
  Serial.print(distance);
  Serial.println(" cm");

  // Check if distance is at or below critical threshold
  if (distance > 0 && distance <= CRITICAL_THRESHOLD) {
    Serial.println("WARNING: Critical distance reached!");
  }

  // Optional: demonstrate use of getDistance() (less accurate, uses default single ping)
  // unsigned int quickDistance = sonar.getDistance();
  // Serial.print("Quick distance (getDistance): ");
  // Serial.print(quickDistance);
  // Serial.println(" cm");
}
```

../../Code/Arduino/UltraSonicHCSR04/schwellenwertMeldung/schwellenwertMeldung.ino

Listing 13.3.: Code für die Messung einer Strecke

```
/**
 * @file entfernungMessung.ino
 * @brief Simple program to measure and display distance in centimeters
    using the HC-SR04 ultrasonic sensor.
 *
 * This program initializes the HC-SR04 sensor and continuously measures
     the distance to an object.
 * The distance is displayed in centimeters on the Serial Monitor using
    the NewPing library.
 */

#include <NewPing.h>

// Define pins and maximum measurement distance
#define TRIGGER_PIN  9     ///< Pin connected to the trigger pin of the
     HC-SR04
#define ECHO_PIN     10    ///< Pin connected to the echo pin of the HC
    -SR04
#define MAX_DISTANCE 200   ///< Maximum distance to measure (in cm)

/// Create a NewPing object for distance measurement
NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);

/**
 * @brief Arduino setup function.
 *
 * Initializes serial communication and prints startup message.
 */
void setup() {
  Serial.begin(9600);
  Serial.println("Simple distance measurement started...");
}

/**
 * @brief Arduino main loop function.
 *
 * Measures the distance using getDistance() and prints it in
    centimeters.
 */
void loop() {
  delay(1000); // Wait 1 second between measurements

  // Measure distance in cm (automatically calculated by NewPing)
  unsigned int distance = sonar.getDistance();

  // Print result
  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");
}
```

../../Code/Arduino/UltraSonicHCSR04/entfernungMessung/entfernungMessung.ino

Part III.

Applikation: Magic Faucet Fountain

14. Hardware-Beschreibung

14.1. Sensor: HC-SR04

siehe oben...?

14.2. Aktor: Pumpe

14.2.1. Einleitung

14.2.2. Grundlegende Informationen

14.2.3. Pumpe <Modell...>

14.2.4. Spezifikation

14.2.5. Einfacher Code

14.2.6. Tests

14.2.7. Einfache Anwendung

14.2.8. Further Readings

14.3. Netzteil

14.3.1. Einleitung

Netzteile sind elektronische Bauteile welche dazu genutzt werden, eine Eingangsspannung (oder auch Strom) in eine gewünschte Ausgangsspannung (oder Strom) umzuwandeln. Sie sind besonders wichtig für eine stabile und sichere Versorgungsspannung elektrischer Bauteile. Übergreifend lassen sich Netzteile zu den Spannungswandlern zuordnen. [Her+24]

14.3.2. Grundlegende Informationen

Ein Netzteil wandelt in den meisten Fällen eine bekannte Wechselspannung, wie die Netzspannung im deutschen Netz (230 V / 50 Hz), in eine gewünschte Gleichspannung um. Wechselspannungen lassen sich leichter als Gleichspannungen über weite Strecken transportieren, jedoch wird für die meisten elektrischen Geräte eine Gleichspannung benötigt. Man teilt Netzteile in 2 Gruppen ein: [Her+24]

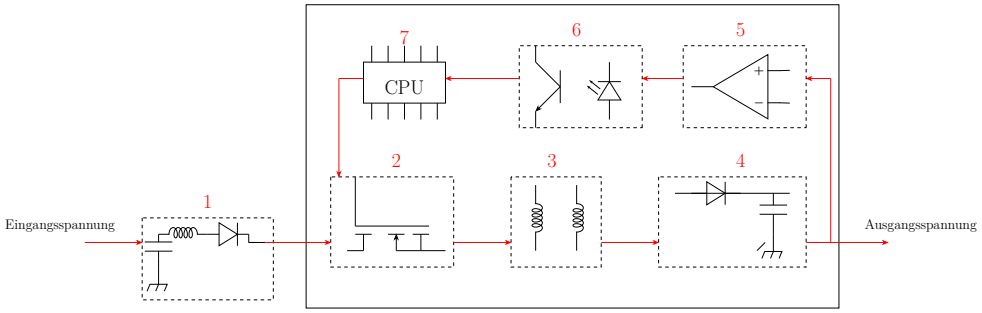
1. **Trafonetzteile:** groß und schwer, bestehen aus Transformator, Gleichrichter und Glättungskondensator

2. **Schaltnetzteile:** klein und leicht, sehr effizient, nutzen Halbleitertechnik

Für die meisten elektronischen Bauteile werden heutzutage nur noch Schaltnetzteile eingesetzt. Sie sind leichter, effizienter, günstiger und deutlich kleiner. Im Folgenden wird daher vertieft auf das Schaltnetzteil eingegangen.[HI18] Der Aufbau eines Schaltnetzteils ist wie folgt: (vgl. 14.1) Die Eingangs-Netzfrequenz von 230 V wird im Netzgleichrichter gleichgerichtet und gesiebt. Der Schalter ist ein Halbleiterbauelement

(Hochfrequenzschalter wie MOSFET) welcher aus der Gleichspannung eine Rechteckwechselspannung mit hoher Schaltfrequenz umwandelt. Der Transformator ist ein klassischer Ferritkern-Trafo und dient zur Spannungsübersetzung und zur galvanischen Trennung der Stromkreise. Im Ausgangsgleichrichter wird aus der hochfrequenten Wechselspannung eine eine kleine Gleichspannung gleichgerichtet und gesiebt. Der Regelverstärker dient zur Verstärkung der Ausgangsspannung und führt diese in den eingebauten Regelkreis des Schaltnetzteils. Um den Regelkreis und die empfindliche Elektronik vom Ausgangsstromkreis zu trennen, wird eine Potentialtrennung (Optokoppler) verwendet. Die Steuer- und Überwachungsschaltung wird mit einem Microcontroller (CPU) realisiert. Dieser überwacht und steuert den Regelkreis und sorgt somit für eine konstante und stabile Ausgangsspannung. [HI18]

Figure 14.1.: Aufbau eines Schaltnetzteils



14.3.3. Schaltnetzteil: MW RD-50A

Das Schaltnetzteil in unserem Projekt ist das „MeanWell RD-50A“. Dabei handelt es sich um ein Schaltnetzteil mit variabler Eingangsspannung und zwei Ausgangsspannungen: 5 V und 12 V. Als Eingangsspannung wird die Netzspannung von 230 V und 50 Hz genutzt, das Netzteil stellt daraufhin die Versorgungsspannung für Arduino und Sensor (5 V) her und ebenfalls die Lastspannung von 12 V für die Pumpe. [Mea] [Rei25]

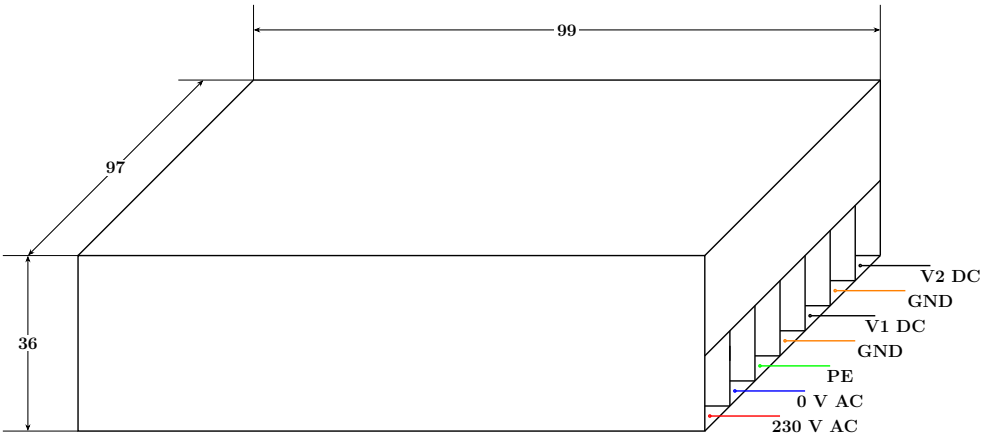
14.3.4. Spezifikation

Das MW RD-50A, zu sehen in 14.2, verfügt über eine variable Ausgangsspannung von 5/12 V und eine Ausgangsleistung von 54 W bei einem Ausgangsstrom von 6 A. Das Schaltnetzteil hat einen Wirkungsgrad von 79% und verfügt über einen integrierten Kurzschlussschutz, Überlastschutz und einen Überspannungsschutz. Es hat mit seiner kompakten Bauform ein Gewicht von 410 g und ist verfügt über Schraubanschlüsse. In der Tabelle 14.1 ist die Pinbelegung zu sehen. Die Pins 1-3 sind die L1 Phase vom AC Netz, Neutralleiter und PE. Sie bilden die Versorgungsspannung. Die Pins 4 und 6 sind für die Erdung der beiden Gleichstromkreise zuständig. Pin 5 sorgt für die 5 V DC Spannung und Pin 7 ist für das 12 V DC Netz zuständig.[Mea] [Rei25]

Table 14.1.: Pinbelegung des Schaltnetzteils

1	L1 – 230 V AC
2	N – 0 V AC
3	PE
4	GND (DC)
5	V1 – 5 V DC
6	GND DC
7	V2 – 12 V (DC)

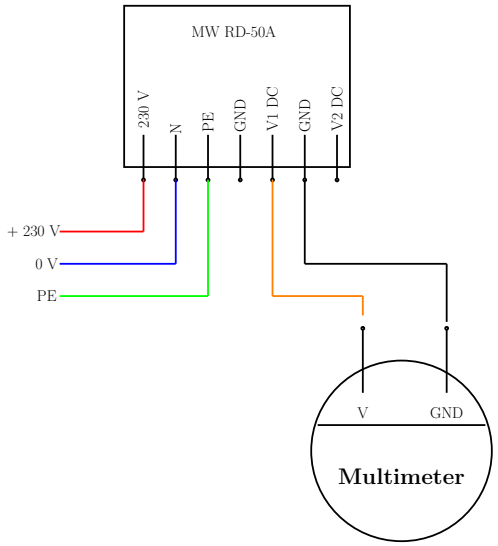
Figure 14.2.: Skizze des Netzteils



14.3.5. Tests

Mit einem einfachen Multimeter kann man die Ausgangsspannungen des Schaltnetzteils überprüfen. Dazu schließt man das Netzteil am AC-Netz an und verbindet die Messspitzen des Multimeters mit dem GND und jeweils dem V1 und V2 Anschluss, sieh dazu 14.3. Am Multimeter muss jedoch das Wahlrad auf DC 20 V eingestellt werden. Zu beachten ist, dass das Netzteil eine Toleranz der Ausgangsspannung von ± 2% hat und das Multimeter auch eine interne Messabweichung hat. Dies ist zu berücksichtigen, daher empfiehlt es sich, eine Versuchsreihe von mindestens 3 Versuchen durchzuführen und das arithmetische Mittel der 3 Anzeigewerte zu nehmen.

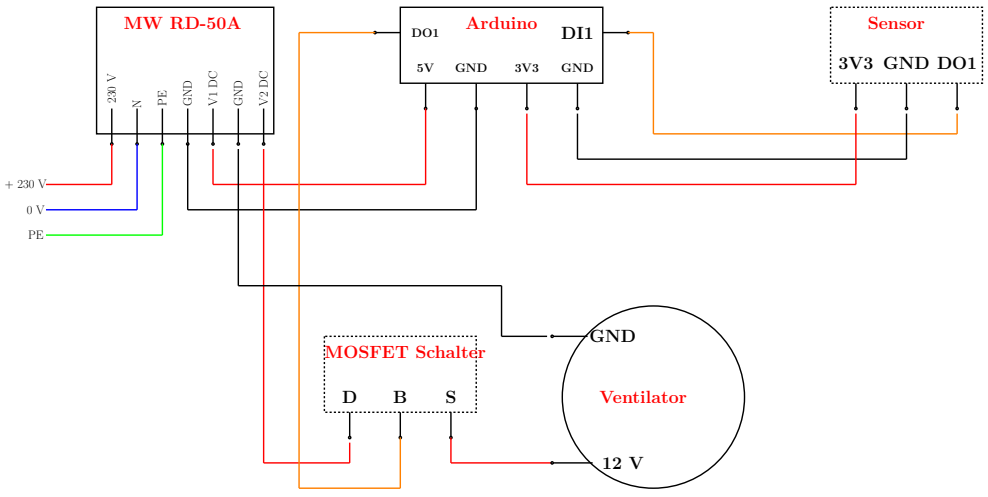
Figure 14.3.: Spannungsmessung des Netzteils



14.3.6. Einfache Anwendung

Als eine einfache Anwendung kann man zum Beispiel das Netzteil verwenden, um einen Steuer- und einen Laststromkreis aufzubauen. Mit dem Steuerstromkreis werden Sensoren und Mikrocontroller versorgt und der Laststromkreis ist für größere Aktoren wie ein Ventilator zuständig. In diesem Anwendungsfall (14.4) ist der Mikrocontroller ein Arduino und der Sensor ein Luftfeuchtigkeitssensor. Sinkt die Luftfeuchtigkeit im Raum registriert dies der Sensor. Der Arduino schaltet dann den Laststromkreis mit einem MOSFET frei und der Ventilator beginnt sich zu drehen. Mit folgendem Code lässt sich das realisieren: 14.1

Figure 14.4.: Anwendung des Netzteils für 2 seperate Stromkreise



Listing 14.1.: Einfacher Code zum Verwenden des Netzteils

```
1000 /**
 * @file anwendungSchaltnetzteil.ino
1002 * @brief Controls a 12V fan via a MOSFET based on room humidity.
 *
1004 * If the room humidity drops below 40%, the Arduino activates DO1,
 * which switches a MOSFET to power a 12V fan.
1006 *
 * @date 2025-04-28
1008 */

1010 #include <DHT.h>

1012 /// Digital pin connected to the DHT sensor
#define DHT_PIN 2
1014
/// Digital pin to control the MOSFET (DO1)
1016 #define FAN_CONTROL_PIN 3

1018 /// DHT sensor type: DHT11 or DHT22
#define DHT_TYPE DHT22
1020
/// Humidity threshold in percent
1022 #define HUMIDITY_THRESHOLD 40.0

1024 /// DHT sensor instance
DHT dht(DHT_PIN, DHT_TYPE);
1026
/**
1028 * @brief Arduino setup function.
 * Initializes serial communication, DHT sensor, and fan control pin.
1030 */
void setup() {
1032   Serial.begin(9600);
   dht.begin();
1034   pinMode(FAN_CONTROL_PIN, OUTPUT);
   digitalWrite(FAN_CONTROL_PIN, LOW); // Ensure fan is off at startup
1036 }

1038 /**
 * @brief Arduino main loop.
1040 * Reads humidity and activates fan if below threshold.
 */
1042 void loop() {
   float humidity = dht.readHumidity();
1044
   // Check if sensor reading is valid
1046   if (isnan(humidity)) {
     Serial.println("Failed to read humidity from sensor.");
1048     return;
   }
1050
   Serial.print("Humidity: ");
1052   Serial.print(humidity);
   Serial.println(" %");
1054
   // Control fan based on humidity
1056   if (humidity < HUMIDITY_THRESHOLD) {
     digitalWrite(FAN_CONTROL_PIN, HIGH);  ///< Turn on fan
1058     Serial.println("Fan ON (humidity below threshold)");
   } else {
1060     digitalWrite(FAN_CONTROL_PIN, LOW);   ///< Turn off fan
     Serial.println("Fan OFF (humidity above threshold)");
1062   }

1064   delay(2000); // Wait before next reading
}
```

../../Code/Arduino/Schaltnetzteil/anwendungSchaltnetzteil/anwendungSchaltnetzteil.ino

14.3.7. Further Readings

Tränkler, H.-R.:mSensortechnik – Handbuch für Praxis und Wissenschaft. 2. Aufl., Springer Vieweg, 2018 [TR18]

- Ein schöner Einstieg und Überblick über die Sensortechnik und ihre Anwendung heutzutage.

E. Hering et al. *Elektrotechnik und Elektronik in Maschinenbau und Mechatronik*. 5. Auflage. Springer, 2024. ISBN: 978-3-662-67538-0.[Her+24]

- Eine sehr spannende Einführung in die Elektrotechnik für Maschinenbauer. Viele grundlegende Informationen und gute Anwenderbeispiele.

Haeberle and Isele. Tabellenbuch Elektrotechnik. 28th ed. Europa Lehrmit- tel, 2018. isbn: 978-3-808-53490-8.[HI18]

- Ein tolles Tabellenbuch in dem viel Grundlagenwissen vermittelt wird. Zudem sind die einfache Skizzen und Erklärungen hervorzuheben.

14.4. Bluetooth Modul

Beispieltext und so uodsafOSIFsdoifhSDOFHsdkljfhSKDJFHksljdfhKSJDFKjlsdfkjSDFHKJsdfhkjSDFHKjsdfhksLDFHKjsdfhklSDF SDFOJsdhflSDHFLKsdflkSDHFLKsdhflksDFHLSDKFLSFLSDHFSD DSJLFHsdlfhSLKDFHlskdfhlksdfhlKSDFHlskdflkSDFLFSDFGGGGGGGGGGGGGGGGGGGGGGGGSDFGSDGDSGDFSGADFGadsfgadgadfgadgdafgadfg Beispieltext und so uodsafOSIFsdoifhSDOFHsdkljfhSKDJFHksljdfhKSJDFKjlsdfkjSDFHKJsdfhkjSDFHKjsdfhksLDFHKjsdfhklSDF SDFOJsdhflSDHFLKsdflkSDHFLKsdhflksDFHLSDKFLSFLSDHFSD DSJLFHsdlfhSLKDFHlskdfhlksdfhlKSDFHlskdflkSDFLFSDFGGGGGGGGGGGGGGGGGGGGGGGGSDFGSDGDSGDFSGADFGadsfgadgadfgadgdafgadfg Beispieltext und so uodsafOSIFsdoifhSDOFHsdkljfhSKDJFHksljdfhKSJDFKjlsdfkjSDFHKJsdfhkjSDFHKjsdfhksLDFHKjsdfhklSDF SDFOJsdhflSDHFLKsdflkSDHFLKsdhflksDFHLSDKFLSFLSDHFSD DSJLFHsdlfhSLKDFHlskdfhlksdfhlKSDFHlskdflkSDFLFSDFGGGGGGGGGGGGGGGGGGGGGGGGSDFGSDGDSGDFSGADFGadsfgadgadfgadgdafgadfg Beispieltext und so uodsafOSIFsdoifhSDOFHsdkljfhSKDJFHksljdfhKSJDFKjlsdfkjSDFHKJsdfhkjSDFHKjsdfhksLDFHKjsdfhklSDF SDFOJsdhflSDHFLKsdflkSDHFLKsdhflksDFHLSDKFLSFLSDHFSD DSJLFHsdlfhSLKDFHlskdfhlksdfhlKSDFHlskdflkSDFLFSDFGGGGGGGGGGGGGGGGGGGGGGGGSDFGSDGDSGDFSGADFGadsfgadgadfgadgdafgadfg Beispieltext und so uodsafOSIFsdoifhSDOFHsdkljfhSKDJFHksljdfhKSJDFKjlsdfkjSDFHKJsdfhkjSDFHKjsdfhksLDFHKjsdfhklSDF SDFOJsdhflSDHFLKsdflkSDHFLKsdhflksDFHLSDKFLSFLSDHFSD DSJLFHsdlfhSLKDFHlskdfhlksdfhlKSDFHlskdflkSDFLFSDFGGGGGGGGGGGGGGGGGGGGGGGGSDFGSDGDSGDFSGADFGadsfgadgadfgadgdafgadfg

14.5. Kippschalter

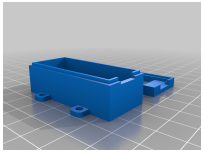
14.6. MOSFET

14.7. Externe LED

Part IV.

Anhang

15. Materialliste

Anzahl	Bezeichnung	Link	Preis
1	ARD NANO 33BLE H - Arduino Nano 33 BLE with header 	www.reichelt.de - ARD NANO 33BLE H	29,50 €
1	ARD NANO 33BSH2 - Arduino Nano 33 BLE Sense Rev.2 with Header 	www.reichelt.de - ARD NANO 33BSH2	44,80 €
1	ARD KIT TINYML - Arduino Lern-Kit Tiny Machine 	www.reichelt.de - ARD Kit TinyML	49,24 €
1	Arducam B0226 Lens Calibration Tool 	www.welectron.com - Arducam B0226 Lens Calibration Tool	11,90 €
1	3D Printed Case - A protective case for Arduino Nano 33 BLE Sense, customizable and available at Thingiverse 	Thingiverse	./.

Stand: 03.02.2025

Literaturverzeichnis

[AG24] A. AG. *NewPing – Library Documentation*. 2024. URL: https://docs.arduino.cc/libraries/newping/.

[Arda] *Arduino ABX00031 Nano 33 BLE Sense Module Benutzerhandbuch*. [pdf]. 2022. URL: https://de.manuals.plus/arduino/abx00031-nano-33-ble-sense-module-manual (visited on 06/26/2023).

[Ardb] *Arduino Reference – Interrupt*. 2019. URL: https://www.arduino.cc/reference/de/language/functions/external-interrupts/attachinterrupt/ (visited on 06/26/2024).

[Ardc] *Arduino Tiny Machine Learning Kit*. [pdf]. 2022. URL: https://store.arduino.cc/products/arduino-tiny-machine-learning-kit (visited on 06/23/2023).

[Ard21] Arduino, ed. *Arduino Nano 33 BLE Sense Rev2 with headers*. 2021. URL: https://store.arduino.cc/products/nano-33-ble-sense-rev2-with-headers.

[Ard21] Arduino, ed. *Check out Arduino Docs!* 2021. URL: https://www.arduino.cc/en/Guide.

[Ard23a] Arduino, ed. *Arduino Nano 33 BLE Sense Rev1 - Product Reference Manual*. [pdf]. 2023. URL: https://docs.arduino.cc/resources/datasheets/ABX00031-datasheet.pdf.

[Ard23b] Arduino, ed. *Arduino Nano 33 BLE Sense Rev2 - Product Reference Manual*. [pdf]. 2023. URL: https://docs.arduino.cc/resources/datasheets/ABX00069-datasheet.pdf.

[Ard24a] Arduino, ed. *Getting started with the Arduino Nano 33 BLE Sense*. 2024. URL: https://wiki-content.arduino.cc/en/Guide/NANO33BLESense.

[Ard24b] Arduino, ed. *Language Reference*. 2024. URL: https://docs.arduino.cc/language-reference/.

[BN11] H. Babovsky and W. Neundorf. *Numerische Approximation von Funktionen*. Tech. rep. TU Ilmenau, 2011. URL: https://www.tu-ilmenau.de/fileadmin/media/num/neundorf/Dokumente/Preprints/NumApp1.pdf (visited on 01/13/2017).

[Bab+23] N. R. Babu et al. "Case Study on Ni-MH Battery". In: *2023 2nd International Conference on Applied Artificial Intelligence and Computing (ICAAIC)* (2023), pp. 1559–1564.

[Ber18] H. Bernstein. *Elektrotechnik/Elektronik für Maschinenbauer - Einfach und praxisgerecht*. 3rd ed. Berlin Heidelberg New York: Springer-Verlag, 2018, pp. 235–236. ISBN: 978-3-658-20838-7.

[Box21] J. Boxall. *Arduino Workshop - A Hands-On Introduction with 65 Projects*. 2. No Starch Press, 2021. ISBN: 9781593274481.

[Din] *DIN EN ISO 13850: Sicherheit von Maschinen - Not-Halt-Funktion - Gestaltungsleitsätze (ISO 13850:2015)*. 2016. URL: https://www.din.de/de/mitwirken/normenausschuesse/nam/veroeffentlichungen/wdc-beuth:din21:233572513.

[FD22] P. Filipi and Dozie. *A Difference between Arduino Nano 33 BLE Sense vs. Arduino Nano 33 BLE Sense Lite*. [pdf]. 2022. URL: https://forum.arduino.cc/t/a-difference-between-a-n-33-ble-sense-vs-sense-lite/1030305.

[Far02] G. Farin. *Curves and Surfaces for CAGD*. 5. [pdf]. San Diego, CA: Academic Press, 2002.

[Gua21] A. Guadalupi. *Machine LEarning Carrier V1.0*. [pdf]. 2021.

[HEG21] E. Hering, J. Endres, and J. Gutekunst. *Elektronik für Ingenieure und Naturwissenschaftler*. 8. [pdf]. Springer Vieweg, 2021. ISBN: 978-3-662-62697-9. DOI: 10.1007/978-3-662-62698-6.

[HI18] Haeberle and Isele. *Tabellenbuch Elektrotechnik*. 28th ed. Europa Lehrmittel, 2018. ISBN: 978-3-808-53490-8.

[HLG19] B. Heinrich, P. Linke, and M. Glöckler. *Grundlagen Automatisierung: Erfassen-Steuern-Regeln*. Springer-Verlag, 2019.

[HS09] S. Hesse and G. Schnell. *Sensoren für die Prozess-und Fabrikautomation*. Vol. 5. Springer, 2009.

[HS+12] E. Hering, G. Schönfelder, et al. *Sensoren in Wissenschaft und Technik*. Springer, 2012.

[HS23] E. Hering and G. Schönfelder. *Sensoren in Wissenschaft und Technik*. 3. [pdf]. Springer Vieweg, 2023. ISBN: 978-3-658-39490-5. DOI: 10.1007/978-3-658-39491-2978-3-662-62697-9.

[Her+24] E. Hering et al. *Elektrotechnik und Elektronik in Maschinenbau und Mechatronik*. 5th ed. Springer, 2024. ISBN: 978-3-662-67538-0.

[Hil22] G. Hiller. *Color measurement - the CIE color space*. [pdf]. Datacolor, 2022.

[Jsta] *Corporate Profile*. 2021. URL: https://www.jst-mfg.com/product/index.php?series=262.

[Jstb] *List of JST Connectors Series*. 2020. URL: http://www.jst.com/home8.html.

[Jstc] *PH connector*. [pdf]. J.S.T. Deutschland GmbH, 2022. URL: https://www.jst-mfg.com/product/index.php?series=199.

[Jstd] *VH connector*. [pdf]. J.S.T. Deutschland GmbH, 2021. URL: https://www.jst-mfg.com/product/index.php?series=262.

[KKV14] K. Karvinen, T. Karvinen, and V. Valtokari. *Sensoren – messen und experimentieren mit Arduino und Raspberry Pi*. dpunkt, 2014. ISBN: 97833864901607.

[Küh24] C. Kühnel. *Arduino - Das umfassende Handbuch*. 3rd ed. [pdf]. Rheinwerk Verlag GmbH, 2024. ISBN: 978-3-367-10279-2.

[Kur21] A. Kurniawan. *IoT Projects with Arduino Nano BLE Sense: Step-By-Step Projects for Beginners*. [pdf]. Berkeley, CA: Apress, 2021. ISBN: 978-1-4842-6457-7. DOI: 10.1007/978-1-4842-6458-4. URL: https://doi.org/10.1007/978-1-4842-6458-4.

[LP18] R. Lukac and K. N. Plataniotis. *Color Image Processing: Methods and Applications*. Image Processing Series. CRC Press, 2018. ISBN: 9781420009781. URL: https://books.google.de/books?id=oD8qBgAAQBAJ.

[MM17] G. Müller and M. Möser. *Ultraschall in Medizin und Technik*. Springer, 2017.

[Mal15] Mallon, Edward and Beddows, Patricia. *How to calibrate a compass (and accelerometer) with Arduino*. accessed date 19.05.2022. 2015. URL: https://thecavepearlproject.org/2015/05/22/calibrating-any-compass-or-accelerometer-for-arduino/.

[Mea] *Handbuch MW RD-50A Series*. [pdf]. 2014.

[Raj19] A. Raj. *Arduino Nano 33 BLE Sense Review - What's New and How to Get Started?* 2019. URL: https://circuitdigest.com/microcontroller-projects/arduino-nano-33-ble-sense-board-review-and-getting-started-guide.

[Rei] *Batterieclip für 9-Volt-Block*. 11445. Das Datenblatt liefert allgemeine Informationen zum Batterieclip. Die Höhe des Batterieclips ist nicht aufgeführt. Reichelt Elektronik GmbH. July 2011.

[Rei24a] Reichelt Elektronik GmbH, ed. *Arduino - Grove Universal-Kabel, 4-Pin, 5cm, fixiert (5er-Pack)*. Online; accessed 17.04.2024. 2024. URL: https://www.reichelt.de/arduino-grove-universal-kabel-4-pin-5cm-fixiert-5er-pack--grv-cable4pin5f-p191140.html.

[Rei24b] Reichelt Elektronik GmbH, ed. *Batterieclip für 9-Volt-Block, vertikal*. Online; accessed 17.04.2024. 2024. URL: https://www.reichelt.de/batterieclip-fuer-9-volt-block-vertikal-clip-9v-p6665.html.

[Rei24c] Reichelt Elektronik GmbH, ed. *GRV 4PIN F2GRV Arduino - Grove 4-Pin Female Jumper zu Grove 4-Pin (5er-Pack)*. Online; accessed 06.06.2024. 2024. URL: https://www.reichelt.de/arduino-grove-4-pin-female-jumper-zu-grove-4-pin-5er-pack--grv-4pin-f2grv-p191166.html.

[Rei25] Reichelt Online, ed. *Schaltnetzteil - MW HRP-75*. [pdf]. 2025. URL: https://www.reichelt.de/de/de/shop/produkt/schaltnetzteil_geschlossen_50_w_5_12_v_6_a-137098#closemodal (visited on 05/02/2025).

[SS14] B. Sencer and E. Shamoto. "Curvature-continuous sharp corner smoothing scheme for Cartesian motion systems". In: *2014 IEEE 13th International Workshop on Advanced Motion Control (AMC)*. [pdf] und [Version 2 - pdf]. Piscataway, NJ: IEEE, 2014, pp. 374–379. ISBN: 978-1-4799-2323-6. DOI: \url{10.1109/AMC.2014.6823311}.

[Sch07] J. Schanda. *Colorimetry: Understanding the CIE System*. Wiley, 2007. ISBN: 9780470175620. URL: https://books.google.de/books?id=uZadszSGe9MC.

[Sch22] T. Scherer. "Batteriemanagement". In: *Elektor special: Stromversorgung und Batterien* (2022), pp. 6–8.

[See12] Seeed Technology Co.,Ltd., ed. *Introduction to Grove*. [pdf]. Shenzhen Headquarters, 2012.

[See13] Seeed Technology Co.,Ltd., ed. *Preface - Getting Started*. [pdf]. Shenzhen Headquarters, 2013.

[See16] Seeed Technology Co.,Ltd., ed. *Grove System*. [pdf]. Shenzhen Headquarters, 2016.

[She] *Voltage Sensor / 170640 - Data Sheet*. 170640. [pdf]. Shenzhen Global Tech Co. 2015.

[Sim] *Joy-IT® Ultrasonic Distance Sensor*. SEN-US01. [pdf]. Simac Electronics GmbH. Aug. 2017.

[Smy21a] R. J. Smythe. *Advanced Arduino Techniques in Science - Refine Your Skills and Projects with PCs or Python-Tkinter*. [pdf]. Berkeley, CA: Apress, 2021. ISBN: 978-1-4842-6786-8. DOI: 10.1007/978-1-4842-6784-4. URL: https://doi.org/10.1007/978-1-4842-6784-4.

[Smy21b] R. J. Smythe. *Arduino in Science - Collecting, Displaying, and Manipulation Sensor Data*. [pdf]. Berkeley, CA: Apress, 2021. ISBN: 978-1-4842-6777-6. DOI: 10.1007/978-1-4842-6777-6.

[Stma] *LSM9DS1 Data Sheets*. 2015. URL: https://www.st.com/resource/en/datasheet/lsm9ds1.pdf (visited on 06/11/2022).

[Stmb] *MP34DT06J – MEMS audio sensor omnidirectional digital microphone*. [pdf]. 2021. URL: https://www.st.com/resource/en/datasheet/mp34dt06j.pdf (visited on 06/23/2023).

[TM24] P. A. Tipler and G. Mosca. *Tipler Physik*. 9th ed. Springer Spektrum Berlin, May 2024, pp. 423–427. ISBN: 978-3-662-67936-4.

[TR14] H.-R. Tränkler and L. Reindl, eds. *Sensortechnik: Handbuch für Praxis und Wissenschaft*. 2nd ed. VDI-Buch. Published: 13 January 2015. eBook Packages: Computer Science and Engineering (German Language). Berlin, Heidelberg: Springer Vieweg Berlin, Heidelberg, 2014, pp. XIV, 1596. ISBN: 978-3-642-29942-1. DOI: 10.1007/978-3-642-29942-1.

[TR18] H.-R. Tränkler and L. M. Reindl, eds. *Sensortechnik Handbuch für Praxis und Wissenschaft*. 2nd ed. Berlin, Heidelberg: Springer Vieweg, 2018. ISBN: 978-3-642-29942-1.

[Voš24] A. Voš. “Volumio mit Drehgebern erweitern”. In: *Make Magazin* (2024). [pdf].

[Wil22] W. M. Willems. *Lehrbuch der Bauphysik*. 9th ed. Springer Vieweg Wiesbaden, Nov. 2022, pp. 567–568. ISBN: 978-3-658-34093-3.

Index